MASTER DSA - 1

Version 1.0

Contents

I. Arrays:
   A. 2 POINTER
      1. Non Repeating Character
      2. Find the duplicate No
      3. 2 Sum
      4. 3 Sum
      5. 4 Sum
      6. 3Sum With Multiplicity
      7. 3Sum Closest
      8. Boats to Save People
      9. Merge two Sorted Arrays Without Extra Space
      10. Remove duplicates in place from sorted array
      11. Container With Most Water
      12. Find Median from Data Stream(A)
   B. PREFIX SUM
      1. Rain Water Trapping
      2. Ways to Split Array Into Three Subarrays
      3. Car Pooling
      4. Subarray Sums Divisible by K
      5. Continuous Subarray Sum
      6. Contiguous Array
      7. Corporate Flight Bookings
      8.
   C. SPECIAL PROBLEMS (2-d matrix, others)
      1. Set Matrix Zeroes
      2. Pascal's Triangle
      3. Next Permutation
      4. Maximum Subarray (KADANES' ALGO)
      5. Sort an array of 0s, 1s and 2s
      6. Rotate matrix
      7. Merge Intervals
      8. Missing Number
      9. Search in 2d matrix
      10. Implement Pow(x,n) | X raised to the power N
      11. Find the Majority Element that occurs more than N/2 times
      12. Find the Majority Element that occurs more than N/3 times
      13. Longest Consecutive Sequence
      14. Subarray Sum Equals K

15. **Count the number of subarrays with given xor K**
16. **Length of Longest Substring without any Repeating Character**
17. **Spiral Matrix II**

II. **STRINGS:**

1. **Longest Substring Without Repeating Characters**
2. **ZigZag Conversions**
3. **Valid Palindrome II**
4. **Print Anagrams Together (A)**
5.

III. **SLIDING WINDOW**

1. **Longest Substring Without Repeating Characters**
2. **Find All Anagrams in a String**
3. **Minimum Size Subarray Sum**
4. **Contains Duplicate II**
5. **Permutation in String**
6. **Max Consecutive Ones III**
7.

IV. **STACKS, QUEUES, DeQue AND HEAPS(PRIORITY QUEUE):**

1. **Sliding window maximum**
2. **Implement STACKS USING ARRAYS**
3. **Implement Queues Using Arrays**
4. **Implement Stack using Queue**
5. **Implement Queue using Stack**
6. **Check for balanced parentheses**
7. **Next greater element**
8. **Next smaller element**
9. **Sort a stack**
10. **Largest rectangle histogram**
11. **Implement min stack**
12. **Maximum Number of Eaten Apples**
13. **Maximum Product After K Increments**
14. **Find K Pairs with Smallest Sums**
15. **Rotten oranges**
16. **Stock Span problem**
17. **Find maximum of minimum of every window size**
18. **The celebrity problem**

V. **Greedy Algorithms**

A.

VI. **SORTING ALGOS:**

A. **Binary Search**

1. **Ways to Split Array Into Three Subarrays**

B. Merge Sort, Bubble Sort,..

I. ARRAYS

A. 2 POINTER

1. Non Repeating Character

Given a string **S** consisting of **lowercase** Latin Letters. Find the first non-repeating character in S.

Example 1:

Input:
S = hello
Output: h
Explanation: In the given string, the first character which is non-repeating is h, as it appears first and there is no other '**h**' in the string.

Example 2:

Input:

S = zxvczbtxyzvy
Output: c
Explanation: In the given string, 'c' is the character which is non-repeating.

Method - 1 (using 26 array)

```
char nonrepeatingCharacter(string S) {
        int n = S.length();
        int[] ans = new int[26];
        for (int i = 0; i < n; i++) {
            ans[S[i]-'a']++;
       }
       for (int i = 0; i < n; i++) {
           if(ans[S[i] - 'a'] == 1){
```

```
                return S[i];
            }
        }
        return "";
    }
```

Explanation: **Take the array of 26 and keep the track of num of each char by traversing the given array to the 26 array. As you get the non duplicate you'll check ans[given[i] - 'a'] == 1 and return given[i].**

2. Find the Duplicate Number

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive. There is only one repeated number in `nums`, return *this repeated number*. You must solve the problem without modifying the array `nums` and use only constant extra space.

Takeaway: Whenever there's constraint of range in values it could be turned into the graph of indices. Which we can apply graph operations like fast-slow pointer, dfs, bfs and such algo's to see if we can arrive at the soln.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

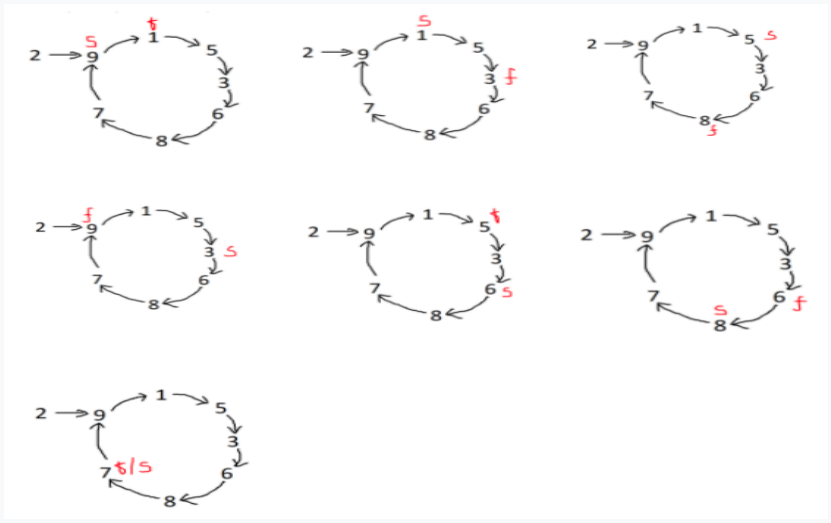
Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

```
METHOD - 1 FAST AND SLOW PTR - Floyd's Tortoise and Hare algorithm 4 cyc. detect.

class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[0], fast = nums[0];
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while(slow!=fast);

        fast = nums[0];
        while(slow!=fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;}}
```



Method - 2 → sorting and seeing the upcoming element is same as the present
Method - 3 → using the concept of hashmap or hashset by lookup
Method - 4 →

3. Two Sum problem

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have *exactly* one solution, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

```
Method - 1 using hashmap
```

```
class Solution{
    public int[] twoSum(int[] nums, int target) {
```

```
        HashMap<Integer, Integer> tracker = new HashMap<Integer, Integer>();
        int len = nums.length;
        for(int i = 0; i < len; i++){
            if(tracker.containsKey(nums[i])){
                int left = tracker.get(nums[i]);
                return new int[]{left+1, i+1};
            }else{
                tracker.put(target - nums[i], i);
            }
        }
        return new int[2]; }}
```

4. 3 Sum

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`. Notice that the solution set must not contain duplicate triplets.

Example 1:

```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
```

```
Method - 1 TC = N**2 16ms
```

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> lis = new ArrayList();
        Arrays.sort(nums);
        for (int i = 0; i < nums.length - 2; i++){
            if (i > 0 && nums[i] == nums[i - 1]) continue; //  /
            // binary search problem.
            int j = i + 1, k = nums.length - 1, target = -nums[i];
            while (j < k){
                if (nums[i] > 0) break; // since sorted so removing redundancy
                int req = nums[k] + nums[j];
                if (req == target){
                    lis.add(Arrays.asList(nums[i],nums[j],nums[k]));
                    j++; k--;
                    while (j < k && nums[j] == nums[j - 1]) j++;
                    while (j < k && nums[k] == nums[k + 1]) k--;}
                else if (req > target) k--;
```

```
                else j++;}}
        return lis;}}
```

```
Method - 2 two ptr (TC = n**2) 29 ms
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        int target = 0;
        ArrayList<List<Integer>> sol = new ArrayList<List<Integer>>();
        if(nums == null || nums.length == 0) return sol;
        Arrays.sort(nums);
        for(int i = 0 ; i<=nums.length-1 ;i++){

                int remaining  = target - nums[i] ;
                int front = i+1;
                int back = nums.length-1;

                while(front < back){
                    int twoSum = nums[front] + nums[back];
                    if(twoSum < remaining)front++;
                    else if(twoSum >remaining)back--;
                    else{
                        List<Integer> triplet = new ArrayList<>();
                        triplet.add(nums[i]);
                        triplet.add(nums[front]);
                        triplet.add(nums[back]);
                        sol.add(triplet);

                        while(front < back && nums[front] == triplet.get(1))front++;
                        while(front < back && nums[back] == triplet.get(2))back--;
                    }
                }
            while( i+1<=nums.length-1 && nums[i] == nums[i+1]) i++;
        }
        return sol; }}
```

5. 4 Sum

Given an array `nums` of `n` integers, return *an array of all the unique quadruplets* `[nums[a], nums[b], nums[c], nums[d]]` such that:

- `0 <= a, b, c, d < n`
- `a`, `b`, `c`, and `d` are distinct.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in any order.

Method - 1 using hashmap

```
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
```

```
        int n = nums.length;
        Arrays.sort(nums);
        HashSet<Integer> set = new HashSet<>();
        List<List<Integer>> ans = new ArrayList<>();
        for (int i = 0; i < n; i++){
            int a = nums[i];
            for (int j = i + 1; j < n; j++){
                int b = nums[j];
                for (int k = j + 1; k < n; k++){
                    int c = nums[k];
                    int d = target - a - b - c;
                    if (set.contains(d)){
                        List<Integer> list = new ArrayList();
                        list.add(a); list.add(b);
                        list.add(c); list.add(d);
                        if (! ans.contains(list))
                            ans.add(list);

                    }
                }
            }
            set.add(a);
        }
        return ans; }}
```

Method - 2 better approach

Intuition: In the previous approach we fixed three-pointers and did a binary search to find the fourth. Over here we will fix two-pointers and then find the remaining two elements using two pointer technique as the array will be sorted at first.

Approach: Sort the array, and then fix two pointers, so the remaining sum will be **(target – (nums[i] + nums[j]))**. Now we can fix two pointers, one front, and another back, and easily use a two-pointer to find the remaining two numbers of the quad. In order to avoid duplications, we jump the duplicates, because taking duplicates will result in repeating quads. We can easily jump duplicates, by skipping the same elements by running a loop.

Dry-run: In case you want to see the dry run of this approach, please watch the video at the bottom.

```
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> res = new ArrayList<>();
        Arrays.sort(nums);
        for(int i = 0; i < nums.length; i++){
            if(i != 0 && nums[i] == nums[i-1]) continue;
            for(int j = i+1; j < nums.length; j++){
                if(j != i+1 && nums[j] == nums[j-1]) continue;
                int t = target - nums[i] - nums[j];
```

```
                int m = j+1, n = nums.length-1;
                while(m < n){
                    if(nums[m]+nums[n] == t){
                        Integer[] temp = {nums[i], nums[j], nums[m], nums[n]};
                        res.add(Arrays.asList(temp));
                        m++;
                        n--;
                        while(m < nums.length && nums[m] == nums[m-1]) m++;
                        while(n >= 0 && nums[n] == nums[n+1]) n--;
                    } else if(nums[m]+nums[n] < t){
                        m++;
                    } else n--;
                }
            }
        }
        return res; }}
```

6. 3Sum With Multiplicity

Given an integer array `arr`, and an integer `target`, return the number of tuples `i, j, k` such that `i < j < k` and `arr[i] + arr[j] + arr[k] == target`.

As the answer can be very large, return it modulo `10^9 + 7`.

Example 1: `Input: arr = [1,1,2,2,3,3,4,4,5,5], target = 8  Output: 20`
`Explanation:  Enumerating by the values (arr[i], arr[j], arr[k]):`
`(1, 2, 5) occurs 8 times; (1, 3, 4) occurs 8 times;`
`(2, 2, 4) occurs 2 times; (2, 3, 3) occurs 2 times.`

Example 2: `Input: arr = [1,1,2,2,2,2], target = 5  Output: 12`
`Explanation:  arr[i] = 1, arr[j] = arr[k] = 2 occurs 12 times:`
`We choose one 1 from [1,1] in 2 ways, and two 2s from [2,2,2,2] in 6 ways.`

Using HashMap

```
class Solution {
    public int threeSumMulti(int[] arr, int target) {
        int cnt = 0, n = arr.length, mod = 1000_000_007;
        for (int i = 0; i < n; i++){
            int a = arr[i];
            HashMap<Integer, Integer> freq = new HashMap();
            for (int j = i + 1; j < n; j++){
                int b = arr[j];
                int c = target - a - b;
                if (freq.containsKey(c)){
                    cnt += freq.get(c);
                    cnt %= mod; }
                freq.put(b, freq.getOrDefault(b, 0) + 1);  } }
        return cnt;  }}
```

Using permutations n freq array

```
class Solution {
    public int threeSumMulti(int[] arr, int target) {
```

```
        long cnt = 0, n = arr.length, mod = 1000_000_007;
        long[] freq = new long[101];
        for (int a: arr) freq[a]++;
        for (int i = 0; i <= 100; i++){
            for (int j = i; j <= 100; j++){
                int k = target - i - j;
                if (k < 0 || k > 100) continue;
                if (i == j && j == k){
                    cnt += freq[i]*(freq[i] - 1)*(freq[i] - 2)/6;
                } else if (i == j && j != k){
                    cnt += (freq[j]*(freq[j] - 1)/2)*freq[k];
                } else if (i < j && j < k){
                    cnt += freq[i]*freq[j]*freq[k];
                }
                cnt %= mod;
            } }
        return (int) cnt;  }}
```

7. 3Sum Closest

Given an integer array nums of length n and an integer target, find three integers in nums such that the sum is closest to target.

Return *the sum of the three integers*.

You may assume that each input would have exactly one solution.

Example 1: `Input: nums = [-1,2,1,-4], target = 1 Output: 2`

`Explanation: The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).`

Example 2:

`Input: nums = [0,0,0], target = 1 Output: 0`

```
Using 2 ptr method
class Solution {
    public int threeSumClosest(int[] nums, int target) {
        int ans = 10001, n = nums.length;
        Arrays.sort(nums);
        for (int i = 0; i < n; i++){
            int left = i + 1, right = n - 1;
            while (left < right){
                int s = nums[i] + nums[left] + nums[right];
                if (s == target){
                    return target;
                }
                if (Math.abs(ans - target) > Math.abs(s - target)){
                    ans = s;
                }
                if (s > target){
                    right--;
                }else {
                    left++;
```

```
                }
            }
        }
        return ans;    }}
```

8. Boats to Save People

You are given an array of people where people[i] is the weight of the ith person, and an infinite number of boats where each boat can carry a maximum weight of limit. Each boat carries at most two people at the same time, provided the sum of the weight of those people is at most limited. Return *the minimum number of boats to carry every given person*. Example 1:

Input: people = [1,2], limit = 3
Output: 1
Explanation: 1 boat (1, 2)

Example 2:

Input: people = [3,2,2,1], limit = 3

Output: 3

Explanation: 3 boats (1, 2), (2) and (3)

```
class Solution {
    public int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people);
        int ans = 0, pL = 0, pR = people.length - 1;

        while (pL <= pR){
            if (people[pL] + people[pR] <= limit){
                pL++;
            }
            pR--;
            ans++;
        }
        return ans;}}
```

9. Merge two Sorted Arrays Without Extra Space

You are given two integer arrays nums1 and nums2, sorted in non-decreasing order, and two integers m and n, representing the number of elements in nums1 and nums2 respectively.
Merge nums1 and nums2 into a single array sorted in non-decreasing order.
The final sorted array should not be returned by the function, but instead be *stored inside the array* nums1. To accommodate this, nums1 has a length of m + n, where the first m elements

denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n.

Example 1: `Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3`

```
Output: [1,2,2,3,5,6]
Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.
```

Example 2:`Input: nums1 = [1], m = 1, nums2 = [], n = 0`

```
Output: [1]
Explanation: The arrays we are merging are [1] and [].
The result of the merge is [1].
```

```
Method - 1 two pointer approach giving SC = 1 and TC = n + m
```

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m + n - 1;
        m--; n--;
        while (m >= 0 && n >= 0){
            if (nums1[m] >= nums2[n]) nums1[i--] = nums1[m--];
            else nums1[i--] = nums2[n--];
        }
        while (n>=0) nums1[i--] = nums2[n--];
    }}
```

```
Method - 2 to make the arr of size m + n and then sort that and retrieve back the elements back to the original matrices TC = ((n + m) * log(n + m)) and SC = n + m;
```

```
Method - 3 using gap concept TC = log (n) (do research)
```

10. Remove duplicates in place from sorted array

Problem Statement: Given an integer array sorted in non-decreasing order, remove the duplicates in place such that each unique element appears only once. The relative order of the elements should be kept the same. If there are k elements after removing the duplicates, then the first k elements of the array should hold the final result. It does not matter what you leave beyond the first k elements.

Note: Return k after placing the final result in the first k slots of the array.

Example 1:

Input: arr[1,1,2,2,2,3,3] **Output:** arr[1,2,3,_,_,_,_]

Explanation: Total number of unique elements are 3, i.e[1,2,3] and Therefore return 3 after assigning [1,2,3] in the beginning of the array.

Example 2:

Input: arr[1,1,1,2,2,3,3,3,3,4,4] **Output:** arr[1,2,3,4,_,_,_,_,_,_,_]

Explanation: Total number of unique elements are 4, i.e[1,2,3,4] and Therefore return 4 after assigning [1,2,3,4] in the beginning of the array.

Method 1: Brute Force

Intuition: We have to think of a data structure that does not store duplicate elements. So can we use a Hash set? Yes! We can. As we know HashSet only stores unique elements.

Approach:

- Declare a HashSet.
- Run a for loop from starting to the end.
- Put every element of the array in the set.
- Store size of the set in a variable K.
- Now put all elements of the set in the array from the starting of the array.
- Return K.

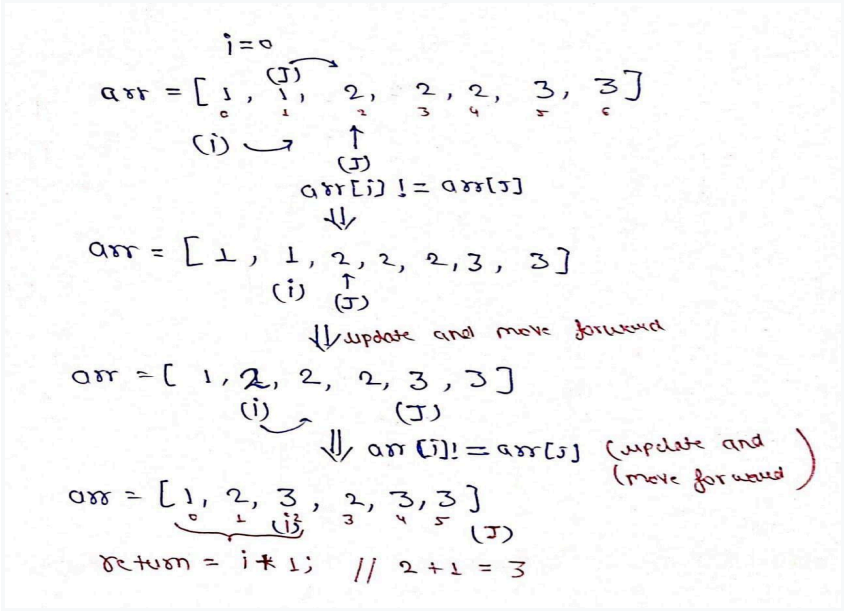
```
static int removeDuplicates(int[] arr) {
        HashSet < Integer > set = new HashSet < > ();
        for (int i = 0; i < arr.length; i++) {
            set.add(arr[i]);
        }
        int k = set.size();
        int j = 0;
        for (int x: set) {
            arr[j++] = x;
        }
        return k; }
Method - 2 Two  pointer
```

Intuition: We can think of using two pointers ‘i’ and ‘j’, we move ‘j’ till we don’t get a number arr[j] which is different from arr[i]. As we got a unique number we will increase the i pointer and update its value by arr[j].

Approach:

- Take a variable i as 0;
- Use a for loop by using a variable ‘j’ from 1 to length of the array.
- If arr[j] != arr[i], increase ‘i’ and update arr[i] == arr[j].

- After completion of the loop return i+1, i.e size of the array of unique elements.




```
static int removeDuplicates(int[] arr) {
        int i = 0;
        for (int j = 1; j < arr.length; j++) {
            if (arr[i] != arr[j]) {
                i++;
                arr[i] = arr[j];
            }
        }
        return i + 1;
    }
```

11. Container With Most Water

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `ith` line are `(i, 0)` and `(i, height[i])`. Find two lines that together with the x-axis form a container, such that the container contains the most water. Return *the maximum amount of water a container can store*. Notice that you may not slant the container.

Example 1:



Input: height = `[1,8,6,2,5,4,8,3,7]`
Output: 49
Explanation: The above vertical lines are represented by array

[1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

```
class Solution {
    public int maxArea(int[] arr) {
        int n = arr.length, ans = 0, i = 0, j = n - 1, area = 0;
        while (i < j){
            if (arr[i] > arr[j]){
                area = (j - i)*arr[j];
                j--;
            }else{
                area = (j - i)*arr[i];
                i++;
            }
            ans = Math.max(ans, area);
        }
        return ans;
    }}
```

12. Find Median from Data Stream(A)

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value and the median is the mean of the two middle values.

- For example, for arr = [2,3,4], the median is 3.
- For example, for arr = [2,3], the median is (2 + 3) / 2 = 2.5.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1: Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output [null, null, null, 1.5, null, 2.0]
Explanation
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1);    // arr = [1]
medianFinder.addNum(2);    // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3);    // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

Solution for main question

```
/**
 * Using two heaps
 *
 * Time Complexity:
 * 1) addNum -> O(5 * log (N/2)) = O(log N)
 * 2) findMedian -> O(1)
 *
 * Space Complexity: O(N)
 * N = Count of numbers in the data stream.
class MedianFinder {
    PriorityQueue<Integer> smallNums; // Max Heap
    PriorityQueue<Integer> largeNums; // Min Heap

    public MedianFinder() {
        smallNums = new PriorityQueue<>(Collections.reverseOrder());
        largeNums = new PriorityQueue<>(); }
    public void addNum(int num) {
        // Add to maxHeap first (Group of smaller numbers)
        smallNums.offer(num);
        // Balance the heaps
        largeNums.offer(smallNums.poll());
        if (largeNums.size() > smallNums.size()) {
            smallNums.offer(largeNums.poll());
        }}
    public double findMedian() {
        if (smallNums.size() != largeNums.size()) {
            return smallNums.peek();
        }
        return (smallNums.peek() + largeNums.peek()) / 2.0; } }
```

Solution for Follow-Up

```
 * Follow-Up 1 - Numbers are between [0, 100]
 * Maintain a cumulative count array.
 * Time Complexity:
 * 1) addNum -> O(101) = O(1)
 * 2) findMedian -> O(log(101)) = O(1)
 * Space Complexity: O(101) = O(1)
class MedianFinder {
    int[] count;

    public MedianFinder() {
        count = new int[101]; }
    public void addNum(int num) {
        for (int i = num; i < 101; i++) {
            count[i]++;
        }}
    public double findMedian() {
        int totalNums = count[100];
        int medianCount = totalNums % 2 == 0 ? totalNums / 2 : (totalNums + 1) / 2;
        int idx = binarySearch(count, medianCount);
        if (totalNums % 2 == 0) {
```

```
            if (medianCount < count[idx]) {return idx; }
            for (int i = idx + 1; i < 101; i++) {
                if (count[i] > count[idx]) { return (idx + i) / 2.0;}
            } }
        return idx; }
    private int binarySearch(int[] nums, int target) {
        int start = 0;
        int end = nums.length - 1;
        while (start < end) {
            int mid = start + (end - start) / 2;
            if (nums[mid] >= target)  end = mid;
            else start = mid + 1;
        }
        return start;}}
```

Optimized addNum() function to true O(1)

```
 * Follow-Up 1 - Numbers are between [0, 100]
 * Maintain a count array. Here Add num is optimized to true O(1)
 * Time Complexity:
 * 1) addNum -> O(1)
 * 2) findMedian -> O(101) = O(1)
 * Space Complexity: O(101) = O(1)
class MedianFinder {
    int[] count;
    int totalNums;
    public MedianFinder() {
        count = new int[101]; }
    public void addNum(int num) {
        count[num]++;
        totalNums++;}
    public double findMedian() {
        int medianCount = totalNums % 2 == 0 ? totalNums / 2 : (totalNums + 1) / 2;
        int countSum = 0, idx = 0;
        while (idx <= 100) {
            countSum += count[idx];
            if (medianCount <= countSum) {
                break;
            }
            idx++;
        }
        if (totalNums % 2 == 0) {
            if (medianCount < countSum)  return idx;
            for (int i = idx + 1; i < 101; i++) {
                if (count[i] != 0)  return (idx + i) / 2.0;
            }
        }
        return idx;}}
```

Solution for Follow-Up 2

```
 * Follow-Up 2 - 99% Numbers are between [0, 100]
 * Until a threshold is met use two heaps to find the median. Once the threshold is
met
 * use the cumulative count array.
```

```
 *
 * Time Complexity:
 * 1) addNum -> Until threshold O(101 + 5*log(threshold/2)). After threshold O(101)
 * 2) findMedian -> Until threshold O(1). After threshold O(101)
 *
 * Space Complexity: O(101+threshold)
 */
class MedianFinder {
    int[] count;
    int lessThanZero, totalNums, threshold;
    PriorityQueue<Integer> smallNums; // Max Heap
    PriorityQueue<Integer> largeNums; // Min Heap

    public MedianFinder() {
        count = new int[101];
        threshold = 100;
        smallNums = new PriorityQueue<>(Collections.reverseOrder());
        largeNums = new PriorityQueue<>();
    }

    public void addNum(int num) {
        if (num < 0) { lessThanZero++;}
        for (int i = Math.max(num, 0); i < 101; i++) {
            count[i]++; }
        totalNums++;
        if (totalNums <= threshold) {
            smallNums.offer(num);
            largeNums.offer(smallNums.poll());
            if (largeNums.size() > smallNums.size()) {
                smallNums.offer(largeNums.poll());
            }} }

    public double findMedian() {
        if (totalNums <= threshold) {
            if (smallNums.size() != largeNums.size()) {
                return smallNums.peek();
            }
            return (smallNums.peek() + largeNums.peek()) / 2.0;
        }
        int medianCount = totalNums % 2 == 0 ? totalNums / 2 : (totalNums + 1) / 2;
        int idx = binarySearch(count, medianCount);
        if (totalNums % 2 == 0) {
            if (medianCount < count[idx]) {
                return idx;
            }
            for (int i = idx + 1; i < 101; i++) {
                if (count[i] > count[idx]) {
                    return (idx + i) / 2.0;
                }
            }
        }
        return idx;
    }

    private int binarySearch(int[] nums, int target) {
```

```
        int start = 0;
        int end = nums.length - 1;
        while (start < end) {
            int mid = start + (end - start) / 2;
            if (nums[mid] >= target) end = mid;
            else  start = mid + 1;
        }
        return start;
    } }
```

B. Prefix Sum

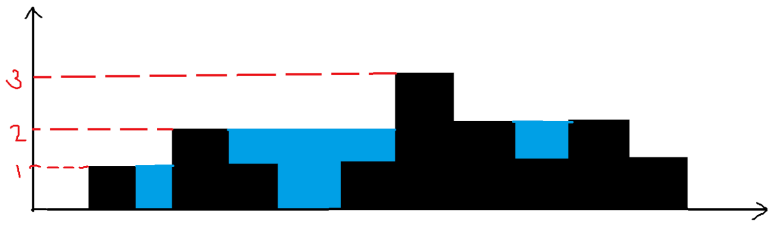
1. Rain Water Trapping

Problem Statement: Given an array of non-negative integers representation elevation of ground. Your task is to find the water that can be trapped after raining.

Example 1:

Input: height= [0,1,0,2,1,0,1,3,2,1,2,1] **Output:** 6

Explanation: As seen from the diagram 1+1+2+1+1=6 unit of water can be trapped



Example 2:
Input: [4,2,0,3,2,5] **Output:**9

Method -3 using 2 ptr approach

```
   public int trap(int[] height) {
   int l_max = 0, r_max = 0, l = 0, r = height.length - 1, ans = 0;
   while (l < r) {
       l_max = Math.max(l_max, height[l]);
       r_max = Math.max(r_max, height[r]);
```

```
        ans += (l_max < r_max) ? l_max - height[l++] : r_max - height[r--];
    }
    return ans;}
```

Method 1: Brute force

Approach: For each index, we have to find the amount of water that can be stored and we have to sum it up.If we observe carefully the amount the water stored at a particular index is the minimum of maximum elevation to the left and right of the index minus the elevation at that index.

```
static int trap(int[] arr) {
        int n = arr.length;
        int waterTrapped = 0;
        for (int i = 0; i < n; i++) {
            int j = i;
            int leftMax = 0, rightMax = 0;
            while (j >= 0) {
                leftMax = Math.max(leftMax, arr[j]);
                j--;
            }
            j = i;
            while (j < n) {
                rightMax = Math.max(rightMax, arr[j]);
                j++;
            }
            waterTrapped += Math.min(leftMax, rightMax) - arr[i];
        }
        return waterTrapped; }
```

Solution 2:Better solution

Intuition: We are taking O(N) for computing leftMax and rightMax at each index. The complexity can be boiled down to O(1) if we precompute the leftMax and rightMax at each index.

Approach: Take 2 array prefix and suffix array and precompute the leftMax and rightMax for each index beforehand.Then use the formula min(prefix[i],suffix[i])-arr[i] to compute water trapped at each index.

```
static int trap(int[] arr) {
        int n = arr.length;
        int prefix[] = new int[n], suffix[] = new int[n];
        prefix[0] = arr[0];
        for (int i = 1; i < n; i++) {
            prefix[i] = Math.max(prefix[i - 1], arr[i]);
        }
        suffix[n - 1] = arr[n - 1];
        for (int i = n - 2; i >= 0; i--) {
            suffix[i] = Math.max(suffix[i + 1], arr[i]);
        }
        int waterTrapped = 0;
        for (int i = 0; i < n; i++) {
            waterTrapped += Math.min(prefix[i], suffix[i]) - arr[i];
```

```
        }
        return waterTrapped;
    }
```

2. Ways to Split Array Into Three Subarrays

A split of an integer array is good if:

- The array is split into three non-empty contiguous subarrays - named `left`, `mid`, `right` respectively from left to right.
- The sum of the elements in `left` is less than or equal to the sum of the elements in `mid`, and the sum of the elements in `mid` is less than or equal to the sum of the elements in `right`.

Given `nums`, an array of non-negative integers, return *the number of good ways to split* `nums`. As the number may be too large, return it modulo $10^9 + 7$.

Example 1: `Input: nums = [1,1,1]  Output: 1`
`Explanation: The only good way to split nums is [1] [1] [1].`

Example 2: `Input: nums = [1,2,2,2,5,0] Output: 3`
`Explanation: There are three good ways of splitting nums:`
`[1] [2] [2,2,5,0]`
`[1] [2,2] [2,5,0]`
`[1,2] [2,2] [5,0]`

Example 3: `Input: nums = [3,2,1] Output: 0`
`Explanation: There is no good way to split nums.`

```
class Solution {
    public int waysToSplit(int[] nums) {
        int MOD = (int) (1e9 + 7), N = nums.length;

        int[] A = new int[N];
        A[0] = nums[0];
        for (int i = 1; i < N; ++i) A[i] = A[i - 1] + nums[i];

        int res = 0;
        for (int i = 1; i < N - 1; ++i) {
            if (A[i - 1] > (A[N - 1] - A[i - 1]) / 2) break;  // early termination

            int left = helper(A, A[i - 1], i, true);
            int right = helper(A, A[i - 1], i, false);

            if (left == -1 || right == -1) continue;  // none is satisfied

            res = (res + (right - left + 1) % MOD) % MOD;
        }
        return res;
    }

    private int helper(int[] A, int leftSum, int index, boolean searchLeft) {
        int N = A.length, l = index, r = N - 2, res = -1;
        while (l <= r) {
```

```
            int m = l + (r - l)/2;
            int midSum = A[m] - A[index - 1];
            int rightSum = A[N - 1] - A[m];

            if (leftSum <= midSum && midSum <= rightSum) {
                res = m;
                if (searchLeft) r = m - 1;
                else l = m + 1;
            } else if (leftSum > midSum) {  // shrink left
                l = m + 1;
            } else {  // shrink right
                r = m - 1;
            }
        }
        return res;
    }}
```

3. Car Pooling

There is a car with `capacity` empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west). You are given the integer `capacity` and an array `trips` where `trips[i] = [numPassengersi, fromi, toi]` indicates that the `ith` trip has `numPassengersi` passengers and the locations to pick them up and drop them off are `fromi` and `toi` respectively. The locations are given as the number of kilometers due east from the car's initial location.

Return `true` *if it is possible to pick up and drop off all passengers for all the given trips, or* `false` *otherwise*.

Example 1: `Input: trips = [[2,1,5],[3,3,7]], capacity = 4 Output: false`

Example 2: `Input: trips = [[2,1,5],[3,3,7]], capacity = 5 Output: true`

Using prefix sum

```
class Solution {
    public boolean carPooling(int[][] trips, int capacity) {
        int[] stops = new int[1001];
        for (int[] t: trips){
            int from = t[1], to = t[2], cap = t[0];
            stops[from] += cap; // Increment when passenger is on board
            stops[to] -= cap; // decrement when they depart
        }
        int load = 0;
        for (int i = 0; i <= 1000; i++){
            load += stops[i];
            if (load > capacity){
                return false;
            }
        }
        return true;
    }}
```

Using priority Queue

```
public boolean carPooling(int[][] trips, int capacity) {
        Arrays.sort(trips, (a,b) -> a[1]-b[1]);  // sort array from start time in ascending order
        PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[2]-b[2]);  // sort PriorityQueue from end time in ascending order.

        for(int[] trip : trips) {
            while(!pq.isEmpty() && pq.peek()[2] <= trip[1]) {
                capacity += pq.poll()[0];
            }
            pq.offer(trip);
            capacity -= trip[0];
            if(capacity < 0)
                return false;
        }
        return true; }
```

4. Subarray Sum Equals K

Given an array of integers `nums` and an integer `k`, return *the total number of subarrays whose sum equals to* `k`.

Example 1: `Input: nums = [1,1,1], k = 2 Output: 2`

Example 2: `Input: nums = [1,2,3], k = 3 Output: 2`

Thinkings

1. Use an array to store the sum accumulated from the beginning to a certain position.

   Example:

   ```
   nums = [1,   2,   3  ]
    sum  = [1, 1+2, 1+2+3]
   ```

2. How to create an array "sum" ? `sum[i] = sum[i - 1] + nums[i]`

Q : If i == 0, the index is out of range. How to solve this problem ?

A : Set the first element of the array "sum" to 0, and initialize the array "sum" from index 1 rather than 0.

```
nums = [1,   2,   3  ]
sum  = [0,   1,   1+2, 1+2+3] // Also, the length of "sum" is one more than "nums"

sum[i] = sum[i - 1] + nums[i - 1]

// Java Version
int[] sum = new int[nums.length + 1];
```

```
sum[0] = 0;
for (int i = 1; i < (nums.length + 1); i++)
  sum[i] = sum[i - 1] + nums[i - 1];
```

1. Using array "sum" to calculate the sum of a subarray

   `sumOfSubarray = sum[end] - sum[start];`

   For example : Calculate the sum of "nums" means using the last element of "sum" minus the first element of "sum" which is 0.

   `nums[0] + nums[1] + nums[2] = sum[3] - sum[0] = 6 - 0`

   Using array "sum" to calculate all possibilities .

```
for (int start = 0; start < sum.length; start++) {
            for (int end = start + 1; end < sum.length; end++) {
                if (sum[end] - sum[start] == k)
                    count++;
            }
        }
```

Code

```
public class Solution {
    public int subarraySum(int[] nums, int k) {
        int count = 0;
        int[] sum = new int[nums.length + 1];
        sum[0] = 0;
        for (int i = 1; i <= nums.length; i++)
            sum[i] = sum[i - 1] + nums[i - 1];

        for (int start = 0; start < sum.length; start++) {
            for (int end = start + 1; end < sum.length; end++) {
                if (sum[end] - sum[start] == k)
                    count++;
            }
        }

        return count;
    }
}
```

Complexity Analysis Time complexity : $O(n2)$ Space complexity : $O(n)$.

Optimization by Hashmap

Thinkings

1. In the previous method

   Step 1. The "nums" array is traversed to calculate all the elements of the sum array

   Step 2. Use the nested loop to judge.

   key : Can we judge when the array is traversed(Step 1) ?

Transposition

```
int[] sum = new int[nums.length + 1];

sum[0] = 0;
for (int end = 1; end < (nums.length + 1); end++)
  sum[end] = sum[end - 1] + nums[end - 1];
```

2.
    1. Put each element of "sum" array into hashmap according to this format : (*sumi*, *number of occurence*)
    2. When constructing the "sum" array, we take the currently constructed element as sum[end], then all the elements before "end" which have been calculated can be regarded as all sum[start] for this "end".

       Transform the judgment condition

       Obviously, when sum[end] is calculated, all its possible sum[start] are already in the map.

       ```
       sum[end] - sum[start] == k

       sum[end] - k == sum[start]
       ```
    3. When *sumend* is calculated, we only need to determine whether there is key == *sumend* - k in the hashmap and add the number of occurence to the answer.

       Attention : In the previous method, we set the first element of sum to 0. Similarly, we put it in the hashmap, which is (0, 1).

Code:

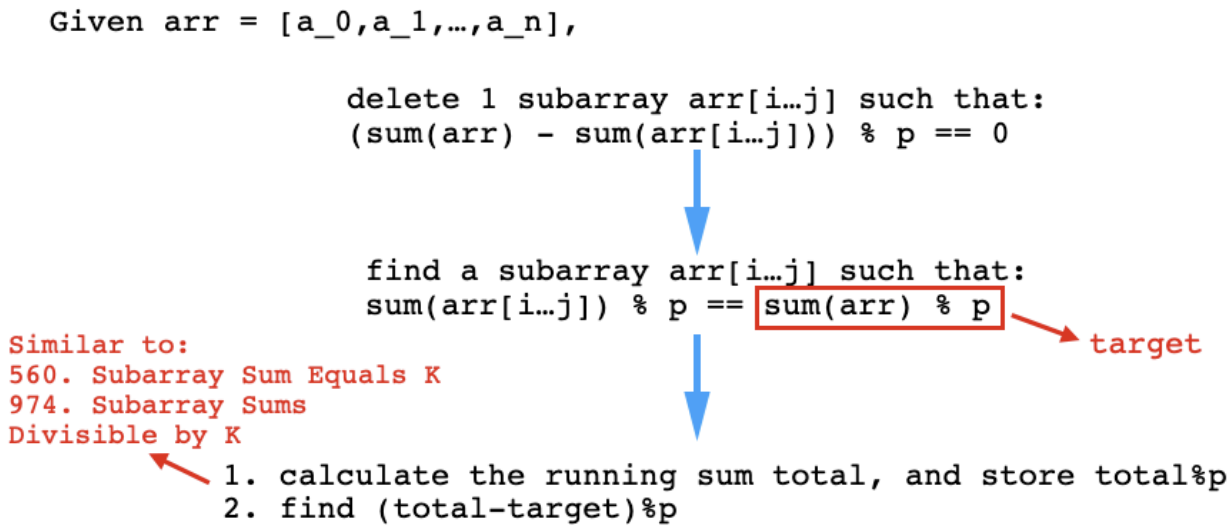
```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int cnt = 0, n = nums.length, sum = 0;
        HashMap<Integer, Integer> map = new HashMap();
        map.put(0 ,1 );
        for (int i = 0; i < n; i++){
            sum += nums[i];
            if (map.containsKey(sum - k)){
                cnt += map.get(sum - k);
            }
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        return cnt; }}
```

5. Make Sum Divisible by P

Given an array of positive integers nums, remove the smallest subarray (possibly empty) such that the sum of the remaining elements is divisible by p. It is not allowed to remove the whole array.
Return *the length of the smallest subarray that you need to remove, or -1 if it's impossible*.

A subarray is defined as a contiguous block of elements in the array.

Example 1: Input: nums = [3,1,4,2], p = 6 **Output**: 1
Explanation: The sum of the elements in nums is 10, which is not divisible by 6. We can remove the subarray [4], and the sum of the remaining elements is 6, which is divisible by 6.
Example 2: Input: nums = [6,3,5,2], p = 9 **Output**: 2
Explanation: We cannot remove a single element to get a sum divisible by 9. The best way is to remove the subarray [5,2], leaving us with [6,3] with sum 9.
Example 3: Input: nums = [1,2,3], p = **3 Output**: 0
Explanation: Here the sum is 6. which is already divisible by 3. Thus we do not need to remove anything.
Logic



The question asks for removing a subarray from arr such that the sum of the remaining array is divisible by p. As shown above, this question could be converted to another question: find one subarray such that the sum of it equals to the sum(arr)%p.

there are O(n^2) subarrays and to reduce the time complexity to O(n), we have to borrow some space. While we are calculating the running/accumulated sum cur==sum(arr[0...j)), we hope there is some prefix sum prev=sum(arr[0...i])) such that: sum(arr[i...j])==(cur-prev)%p==sum(arr)%p==target

Therefore, all we need to find is simply (cur-target)%p.

Code

```
class Solution {
```

```
    public int minSubarray(int[] nums, int p) {
        long remainder = 0, prefixSum = 0;
        int n = nums.length, ans = n;
        for (int N: nums){
            remainder = (remainder + N) % p;
        }
        if (remainder == 0){
            return 0;
        }
        HashMap<Long, Integer> map = new HashMap<Long, Integer>();
        map.put(prefixSum, -1);

        for (int i = 0; i < n; i++){
            prefixSum = (prefixSum + nums[i]) % p;
            long key = (prefixSum - remainder + p) % p;
            if (map.containsKey(key)){
                ans = Math.min(ans, i - map.get(key));
            }
            map.put(prefixSum, i);
        }
        return (ans % (n) == 0) ? -1 : ans;    }}
```

6. Subarray Sums Divisible by K

Given an integer array `nums` and an integer `k`, return *the number of non-empty subarrays that have a sum divisible by* `k`.A subarray is a contiguous part of an array.

Example 1:`Input: nums = [4,5,0,-2,-3,1], k = 5 Output: 7`
`Explanation: There are 7 subarrays with a sum divisible by k = 5:`
`[4, 5, 0, -2, -3, 1], [5], [5, 0], [5, 0, -2, -3], [0], [0, -2, -3], [-2, -3]`
Example 2: `Input: nums = [5], k = 9 Output: 0`

```
class Solution {
    public int subarraysDivByK(int[] nums, int k) {
        int cnt = 0, sum = 0, n = nums.length;
        HashMap<Integer, Integer> freqMap = new HashMap();
        map.put(0, 1);
        for (int i = 0; i < n; i++){
            sum += nums[i];
            int rem = sum % k < 0? sum % k + k : sum % k;
// or sum = (sum + nums[i]% K + K) % K; //for above 2 steps
            if (map.containsKey(rem)){ cnt += freqMap.get(rem);}
```

```
            freqMap.put(rem, freqMap.getOrDefault(rem, 0) + 1);
           // System.out.println(map + " sum " + sum + " cnt : " + cnt) ;
        }
        return cnt;
    }}
{0=1, 4=1} sum 4 cnt : 0
{0=1, 4=2} sum 9 cnt : 1
{0=1, 4=3} sum 9 cnt : 3            // debug o/p's for nums = [4,5,0,-2,-3,1]  k = 5   o/p = 7
{0=1, 2=1, 4=3} sum 7 cnt : 3
{0=1, 2=1, 4=4} sum 4 cnt : 6
{0=2, 2=1, 4=4} sum 5 cnt : 7
```

7. Continuous Subarray Sum

Given an integer array `nums` and an integer `k`, return `true` *if* `nums` *has a continuous subarray of size at least two whose elements sum up to a multiple of* `k`*, or* `false` *otherwise*. An integer `x` is a multiple of `k` if there exists an integer `n` such that `x = n * k`. `0` is always a multiple of `k`.

Example 1: `Input: nums = [23,2,4,6,7], k = 6 Output: true`
`Explanation: [2, 4] is a continuous subarray of size 2 whose elements sum up to 6.`
Example 2: `Input: nums = [23,2,6,4,7], k = 6 Output: true`
`Explanation: [23, 2, 6, 4, 7] is a continuous subarray of size 5 whose elements sum up to 42. 42 is a multiple of 6 because 42 = 7 * 6 and 7 is an integer.`
Example 3: `Input: nums = [23,2,6,4,7], k = 13 Output: false`

```
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        int N = nums.length, sum = 0;
        HashMap<Integer, Integer> map = new HashMap();
        map.put(0, -1);
        for (int i = 0; i < N; i++){
            sum += nums[i];
            sum %= k;
            if (map.containsKey(sum)){
                if (i - map.get(sum) > 1){
                    return true;
                }
            }else{
                map.put(sum, i);
            }    }
        return false;
    }}
```

Math behind the solutions

Haven't seen anyone post the math or theory behind the solutions yet, so I'm sharing mine. Let me know if there is any better one. In short, start with mod =0, then we always do mod =

(mod+nums[i])%k, if mod repeats, that means between these two mod = x occurences the sum is multiple of k.
Math: c = a % k, c = b % k, so we have a % k = b % k.
Where a is the mod at i and b is the mod at j and a <= b, i < j, because all nums are non-negative. And c is the mod that repeats.
Suppose b-a=d, then we have b % k = ((a+d) % k)%k = (a%k + d%k)%k
In order to make the equation valid: a % k = (a%k + d%k)%k
d%k has to be 0, so d, the difference between b and a, is a multiple of k
Example: [23, 2, 1, 6, 7] k=9 mod = 5, 7, 8, 5 <-- at here we found it
Here's my **intuition** about why we need to find repeating mods using an analogy: let's say you're taking out the trash in a new trash can every day and putting it outside your house. However the guy doing the garbage collection tells you he will only take away the most recent number of trash cans whose total weight is divisible by k. He gives you a scale to measure but the scale can only display the remainder weight of all the trash after dividing by k. He also puts a tarp over the trash cans and says you can't lift it to see how many are there. You decide to jot down the excess number everyday but one day, you notice the excess weight returned to a previous day's value. You surmise you must have accumulated trash for the past few days whose weight is divisible by k, otherwise the excess weight could not have returned to a previously recorded value. Remember, the scale would read 0 if it were just the portion of the accumulated trash. You eagerly await the garbage collector and indeed, he comes to take away some of your trash the next day. But only some ;)

8. Contiguous Array

Given a binary array `nums`, return *the maximum length of a contiguous subarray with an equal number of* `0` *and* `1`.

Example 1: **Input:** `nums = [0,1]` **Output:** `2`
Explanation: `[0, 1] is the longest contiguous subarray with an equal number of 0 and 1.`
Example 2: **Input:** `nums = [0,1,0]` **Output:** `2`
Explanation: `[0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0 and 1.`

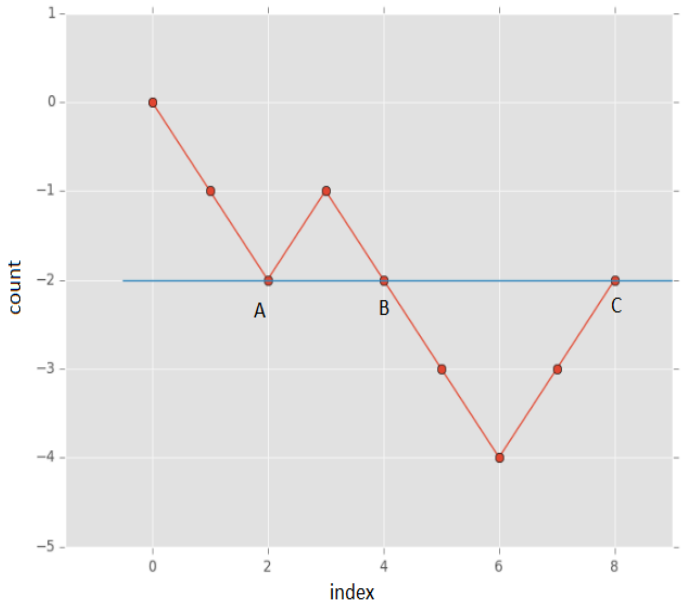
Brute force solution

```
public class Solution {
    public int findMaxLength(int[] nums) {
        int maxlen = 0;
        for (int start = 0; start < nums.length; start++) {
            int zeroes = 0, ones = 0;
```

```
            for (int end = start; end < nums.length; end++) {
                if (nums[end] == 0) {  zeroes++;
                } else {  ones++;  }
                if (zeroes == ones) {
                    maxlen = Math.max(maxlen, end - start + 1);
                }
            }
        }
        return maxlen;}}
```

Approach #2 Using Extra Array [Accepted]: In this approach, we make use of a *count* variable, which is used to store the relative number of ones and zeros encountered so far while traversing the array. The *count* variable is incremented by one for every 1 encountered and the same is decremented by one for every 0 encountered.

We start traversing the array from the beginning. If at any moment, the *count* becomes zero, it implies that we've encountered equal number of zeros and ones from the beginning till the current index of the array(i). Not only this, another point to be noted is that if we encounter the same *count* twice while traversing the array, it means that the number of zeros and ones are equal between the indices corresponding to the equal *count* values. The following figure illustrates the observation for the sequence [0 0 1 0 0 0 1 1]:



In the above figure, the subarrays between (A,B), (B,C) and (A,C) (lying between indices corresponding to *count*=2) have equal number of zeros and ones.Another point to be noted is that the largest subarray is the one between the points (A, C). Thus, if we keep a track of the indices corresponding to the same *count* values that lie farthest apart, we can determine the size of the largest subarray with equal no. of zeros and ones easily.

Now, the *count* values can range between -n to +n, with the extreme points corresponding to the complete array being filled with all 0's and all 1's respectively. Thus, we make use of an array *arr*(of size 2n+1to keep a track of the various *count*'s encountered so far. We make an entry containing the current element's index (*i*) in the *arr* for a new *count* encountered everytime. Whenever, we come across the same *count* value later while traversing the array, we determine the length of the subarray lying between the indices corresponding to the same *count* values.

```
public class Solution {
```

```
    public int findMaxLength(int[] nums) {
        int[] arr = new int[2 * nums.length + 1];
        Arrays.fill(arr, -2);
        arr[nums.length] = -1;
        int maxlen = 0, count = 0;
        for (int i = 0; i < nums.length; i++) {
            count = count + (nums[i] == 0 ? -1 : 1);
            if (arr[count + nums.length] >= -1) {
                maxlen = Math.max(maxlen, i - arr[count + nums.length]);
            } else {
                arr[count + nums.length] = i;
            }
        }
        return maxlen; }}
```

Prefix sum

```
class Solution {
    public int findMaxLength(int[] nums) {
        int max = 0, cnt = 0;
        HashMap<Integer, Integer> map = new HashMap();
        map.put(0, -1);
        for (int i = 0; i < nums.length; i++){
            if (nums[i] == 0) cnt--;
            else cnt++;
            if (map.containsKey(cnt))
                max = Math.max(max, i - map.get(cnt));
            else
                map.put(cnt, i);
        }
        return max;
    }}
```

9. Corporate Flight Bookings

There are `n` flights that are labeled from `1` to `n`.You are given an array of flight bookings `bookings`, where `bookings[i] = [firsti, lasti, seatsi]` represents a booking for flights `firsti` through `lasti` (inclusive) with `seatsi` seats reserved for each flight in the range.Return *an array* `answer` *of length* `n`*, where* `answer[i]` *is the total number of seats reserved for flight* `i`.

Example 1: `Input: bookings = [[1,2,10],[2,3,20],[2,5,25]], n = 5`
`Output: [10,55,45,25,25]`
`Explanation:`

```
Flight labels:        1   2   3   4   5
Booking 1 reserved:  10  10
Booking 2 reserved:      20  20
Booking 3 reserved:      25  25  25  25
Total seats:         10  55  45  25  25
Hence, answer = [10,55,45,25,25]
```

Example 2: `Input: bookings = [[1,2,10],[2,2,15]], n = 2`
`Output: [10,25]`
Explanation:

```
Flight labels:        1   2
Booking 1 reserved:  10  10
Booking 2 reserved:      15
Total seats:         10  25
Hence, answer = [10,25]
```

Brute approach

```
class Solution {
    public int[] corpFlightBookings(int[][] bookings, int n) {
        int[] ans = new int[n];
        for (int[] bk: bookings){
            for (int i = bk[0] - 1; i < bk[1]; i++){
                ans[i] += bk[2];
            } }
        return ans; }}
```

Optimal approach

Flights	1	2	3	4	5
Reservation #1	10		-10		
Reservation #2		20		-20	
Reservation #3			25		
All Reservations	10	20	15	-20	
		+10	+30	+45	+25
Result	**10**	**30**	**45**	**25**	**25**

Intuition Since ranges are continuous, what if we add reservations to the first flight in the range, and remove them after the last flight in range? We can then use the running sum to update reservations for all flights.This picture shows the logic for this test case: [[1,2,10],[2,3,20],[3,5,25]].

```
class Solution {
    public int[] corpFlightBookings(int[][] bookings, int n) {
        int[] ans = new int[n];
         for (int[] bk: bookings){
            ans[bk[0] - 1] += bk[2];
            if (bk[1] < n) ans[bk[1]] -= bk[2];
        }
        for (int i = 1; i < n; i++){   ans[i] += ans[i - 1];   }
        return ans;
```

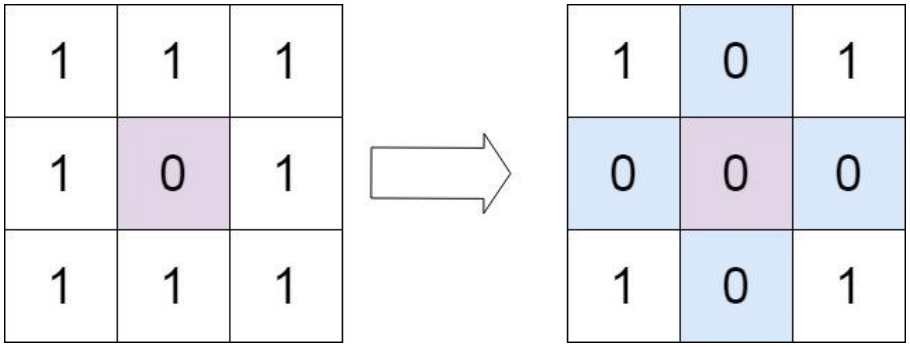
```
    }}
```

C. Special Problems

1. Set Matrix Zeroes

Given an `m x n` integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s. You must do it in place.

Example 1:




```
Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]
Output: [[1,0,1],[0,0,0],[1,0,1]]
```

Example 2:

```
Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]
```

```
Approach 1  Using storing the indicees
```

```
class Solution {
     public void setZeroes(int[][] matrix) {
         List<Integer> rowZero = new ArrayList<>();
         List<Integer> colZero = new ArrayList<>();

         for(int i=0;i<matrix.length;i++){
             for(int j=0;j<matrix[0].length;j++){
                 if(matrix[i][j] == 0){
                     rowZero.add(i);
                     colZero.add(j);
                 }}}

         for(int rowIndex : rowZero) makeRowZero(matrix, rowIndex);
```

```
        for(int colIndex : colZero) makeColZero(matrix, colIndex);

        return;
    }

    private void makeRowZero(int[][] matrix, int rIndex){
        for(int j=0;j<matrix[0].length;j++) {
            matrix[rIndex][j] = 0; }}
    private void makeColZero(int[][] matrix, int cIndex){
        for(int i=0;i<matrix.length;i++) {
            matrix[i][cIndex] = 0; } }}
```

Approach -2 in place

```
// TC : O(m*n)
// SC : O(1)
public class Solution {
    public void setZeroes(int[][] matrix) {
        if(matrix==null) return ;

        int m = matrix.length, n = matrix[0].length;

        boolean firstRowHasZero = false, firstColHasZero = false;

        for(int i=0; i<n; i++){
            if(matrix[0][i]==0){
                firstRowHasZero = true;
                break;  }
        }
        for(int i=0; i<m; i++){
            if(matrix[i][0]==0){
                firstColHasZero = true;
                break; }
        }

        for(int i=1; i<m; i++){
            for(int j=1; j<n; j++){
                if(matrix[i][j]==0){
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }  }
        }
        for(int j=1;j<n; j++){
            if(matrix[0][j]==0){
                nullifyCol(matrix, j, m, n); }
        }

        for(int i=1; i<m; i++){
            if(matrix[i][0]==0){
                nullifyRow(matrix, i, m, n);  }
        }

        if(firstRowHasZero) nullifyRow(matrix, 0, m, n);

        if(firstColHasZero) nullifyCol(matrix, 0, m, n);
```

```
    }

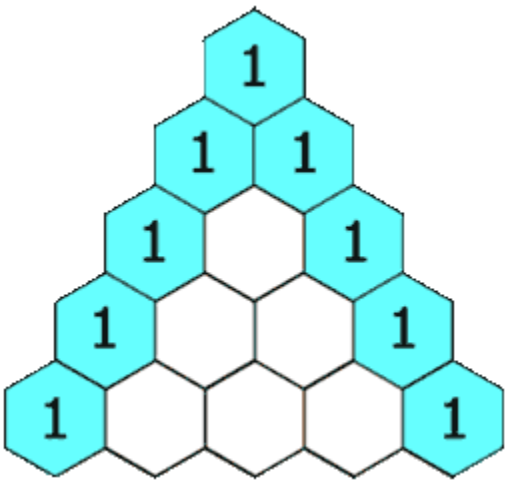
    public void nullifyRow(int[][] matrix, int i, int m, int n){
        for(int col=0; col<n; col++) matrix[i][col] = 0;
    }

    public void nullifyCol(int[][] matrix, int j, int m, int n){
        for(int row=0; row<m; row++) matrix[row][j] = 0;  }}
```

2. Pascal's Triangle

Given an integer `numRows`, return the first numRows of Pascal's triangle.

In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1:

```
Input: numRows = 5
Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]
```

Example 2:

```
Input: numRows = 1
Output: [[1]]
```

```
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> res = new ArrayList<List<Integer>>();
             List<Integer> row, pre = null;
             for (int i = 0; i < numRows; ++i) {
                    row = new ArrayList<Integer>();
                    for (int j = 0; j <= i; ++j)
                           if (j == 0 || j == i) row.add(1);
                           else row.add(pre.get(j - 1) + pre.get(j));
                    pre = row;
                    res.add(row);
             }
             return res; }}
```

3. Next Permutation

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1,2,3]`, the following are considered permutations of `arr`: `[1,2,3]`, `[1,3,2]`, `[3,1,2]`, `[2,3,1]`.

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such an arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of `arr = [1,2,3]` is `[1,3,2]`.
- Similarly, the next permutation of `arr = [2,3,1]` is `[3,1,2]`.
- While the next permutation of `arr = [3,2,1]` is `[1,2,3]` because `[3,2,1]` does not have a lexicographical larger rearrangement.

Given an array of integers `nums`, *find the next permutation of* `nums`. The replacement must be in place and use only constant extra memory.

Example 1: **`Input:`** `nums = [1,2,3]` **`Output:`** `[1,3,2]`

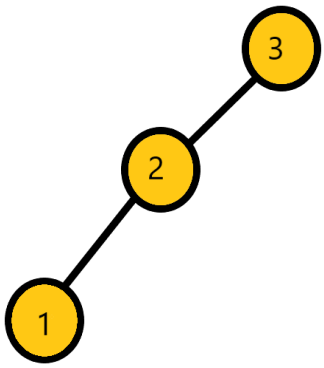
Example 2: **`Input:`** `nums = [3,2,1]` **`Output:`** `[1,2,3]`

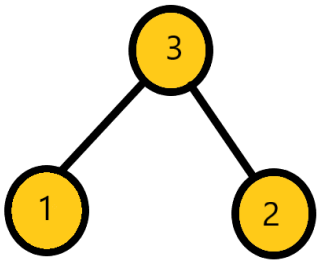
Example 3: **`Input:`** `nums = [1,1,5]` **`Output:`** `[1,5,1]`

`Intuition:` Intuition lies behind the lexicographical ordering of all possible permutations of a given array. There will always be an increasing sequence of all possible permutations when observed.

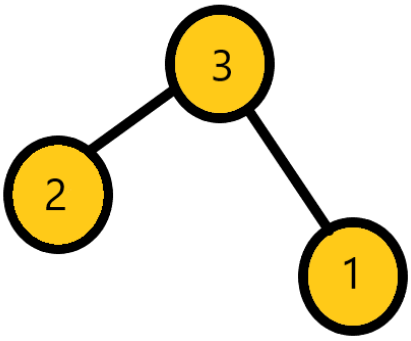
Let's check all sequences of permutations of {1,2,3}.

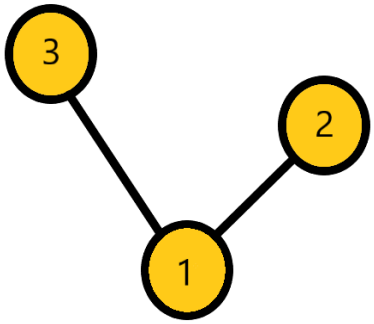
- {1,2,3} {1, 3, 2} {2,1,3}

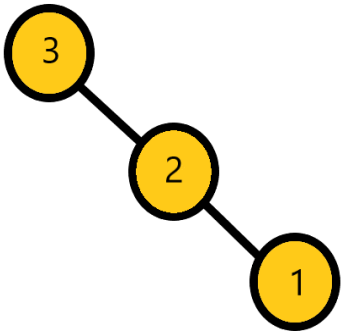




- {2,3,1} {3,1,2} {3,2,1}







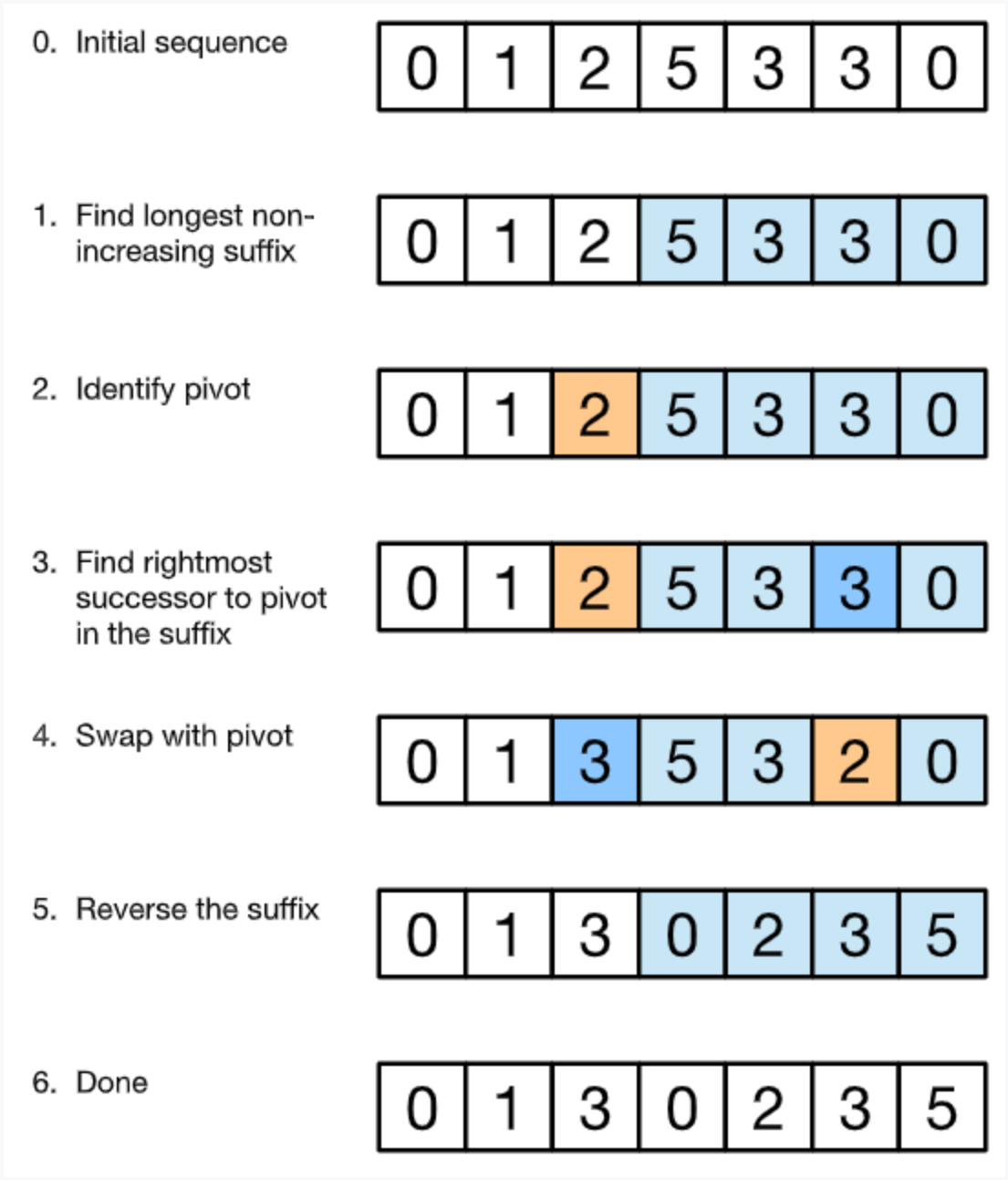
Thus, we can see every sequence has increasing order. Hence, our approach aims to get a peak from where the increasing sequence starts.
This is what we achieve from our first step of the approach.
Then, we need to get just a larger value than the point where the peak occurs. To make rank as few as possible but greater than input array, just perverse array from breakpoint achieved from the first step of the approach. We achieve these from all remaining steps of our approach.
Approach :
Step 1: Linearly traverse array from backward such that ith index value of the array is less than (i+1)th index value. Store that index in a variable.
Step 2: If the index value received from step 1 is less than 0. This means the given input array is the largest lexicographical permutation. Hence, we will reverse the input array to get the minimum or starting

permutation. Linearly traverse array from backward. Find an index that has a value greater than the previously found index. Store index another variable.
Step 3: Swap values present in indices found in the above two steps.
Step 4: Reverse array from index+1 where the index is found at step 1 till the end of the array.
Standard Approach

```
class Solution {
    public void nextPermutation(int[] A) {
        if(A == null || A.length <= 1) return;
        int i = A.length - 2;
        while(i >= 0 && A[i] >= A[i + 1]) i--;
        if(i >= 0) {
            int j = A.length - 1;
            while(A[j] <= A[i]) j--;
            swap(A, i, j);
        }
        reverse(A, i + 1, A.length - 1);
}

public void swap(int[] A, int i, int j) {
    int tmp = A[i];  A[i] = A[j]; A[j] = tmp;
}

public void reverse(int[] A, int i, int j) {
    while(i < j) swap(A, i++, j--);
}}
```



4. Maximum Subarray (KADANES' ALGO)

Problem Statement: Given an integer array arr, find the contiguous subarray (containing at least one number) which
has the largest sum and return its sum and print the subarray.

Example 1: Input: arr = [-2,1,-3,4,-1,2,1,-5,4] **Output:** 6

Explanation: [4,-1,2,1] has the largest sum = 6.

Examples 2: **Input:** arr = [1] **Output:** 1

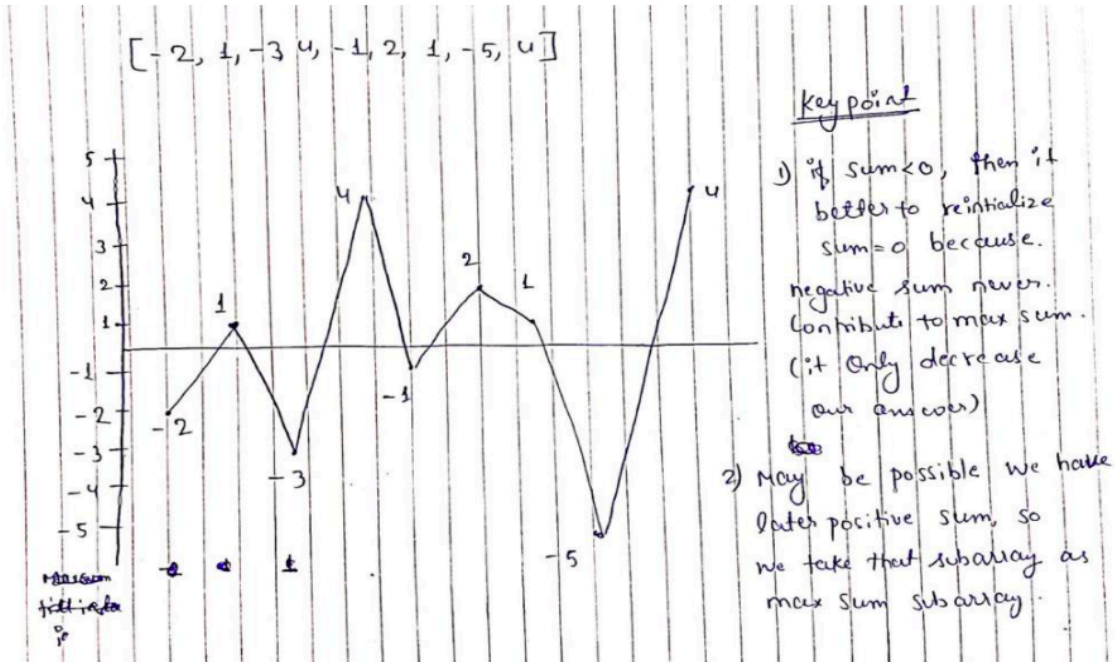
Explanation: Array has only one element and which is giving positive sum of 1.

```
Method - 1 basic method
class Solution {
    public int maxSubArray(int[] nums) {
        int ans = Integer.MIN_VALUE, sum = 0;
        for (int i = 0; i < nums.length; i++){
            sum += nums[i];
            ans = Math.max(ans, sum);
            sum = Math.max(sum, 0);
        }
        return ans; }}
```

```
Method - 2 using naive approach takes O(n**3) TC
```

```
Method - 3 uses better approach with O(n**2) TC
```

> Approch 3 : Kadane Algorithm (Best Approch) - O(n) time Complexity




```
Method - 4 taste of divide and conquer
class Solution {
   public int maxSubArray(int[] nums) {
        return Subarray(nums, 0 ,nums.length -1 );
    }
    public int Subarray(int[] A,int left, int right){
        if(left == right){return A[left];}
        int mid = left + (right - left) / 2;
        int leftSum = Subarray(A,left,mid);// left part
```

```
        int rightSum = Subarray(A,mid+1,right);//right part
        int crossSum = crossSubarray(A,left,right);// cross part
        if (leftSum >= rightSum && leftSum >= crossSum){// left part is max
            return leftSum;
        }
        if (rightSum >= leftSum && rightSum >= crossSum){// right part is max
            return rightSum;
        }
        return crossSum; // cross part is max
    }
    public int crossSubarray(int[] A,int left,int right){
        int leftSum = Integer.MIN_VALUE, rightSum = Integer.MIN_VALUE;
        int sum = 0, mid = left + (right - left) / 2;
        for(int i = mid; i >= left ; i--){
            sum = sum + A[i];
            if(leftSum < sum){
                leftSum = sum;
            }
        }
        sum = 0;
        for(int j = mid + 1; j <= right; j++){
            sum = sum + A[j];
            if(rightSum < sum){
                rightSum = sum;
            }
        }
        return leftSum + rightSum;
    }
}
```

5. Sort an array of 0s, 1s and 2s

Problem Statement: Given an array consisting of only 0s, 1s and 2s. Write a program to in-place sort the array without using inbuilt sort functions. (Expected: Single pass-O(N) and constant space)

Examples:
Input: nums = [2,0,2,1,1,0] **Output:** [0,0,1,1,2,2]

Input: nums = [2,0,1] **Output:** [0,1,2]

Solution 1: Sorting (even if it is not the expected solution here but it can be considered as one of the approach). Since the array contains only 3 integers, 0, 1, and 2. Simply sorting the array would arrange the elements in increasing order.
Time Complexity: O(N*logN) **Space Complexity:** O(1)
Solution 2: Keeping count of values
Intuition: Since in this case there are only 3 distinct values in the array so it's easy to maintain the count of all, Like the count of 0, 1, and 2. This can be followed by overwriting the array based on the frequency(count) of the values.
Approach:

1. Take 3 variables to maintain the count of 0, 1 and 2.
2. Travel the array once and increment the corresponding counting variables

(let's consider **count_0 = a, count_1 = b, count_2 = c**)

3. In the 2nd traversal of array, we will now over write the first 'a' indices / positions in array with '0', the next 'b' with '1' and the remaining 'c' with '2'.

Time Complexity: O(N) + O(N) **Space Complexity:** O(1)
Solution 3: 3-Pointer approach. This problem is a variation of the *popular* ***Dutch National flag algorithm***
Intuition: In this approach, we will be using 3 pointers named low, mid, and high. We will be using these 3 pointers to move around the values. The primary goal here is to move 0s to the left and 2s to the right of the array and at the same time all the 1s shall be in the middle region of the array and hence the array will be sorted.
Approach:

1. Initialize the 3 pointers such that low and mid will point to 0th index and high pointer will point to last index
   int low = arr[0], mid = arr[0], high = arr[n – 1]
2. Now there will 3 different operations / steps based on the value of arr[mid] and will be repeated until mid <= high.
   1. arr[mid] == 0:
      swap(arr[low], arr[mid])
      low++, mid++
   2. arr[mid] == 1: mid++
   3. arr[mid] == 2: swap(arr[mid], arr[high])
      high–;

The array formed after these steps will be a sorted array.

```
class Solution {
    public void sortColors(int[] nums) {
        int lo = 0, hi = nums.length - 1, mid = 0,  temp;
        while (mid <= hi) {
            switch (nums[mid]) {
                case 0: {
                    swap(nums, lo, mid);
                    lo++; mid++;
                    break;
                }
                case 1:
                    mid++;
                    break;
                case 2: {
                    swap(nums, mid, hi);
                    hi--;
                    break;
                }
            }
        }
    }
}

// write swap fn.
```

6. Rotate Matrix

You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly.

DO NOT allocate another 2D matrix and do the rotation.

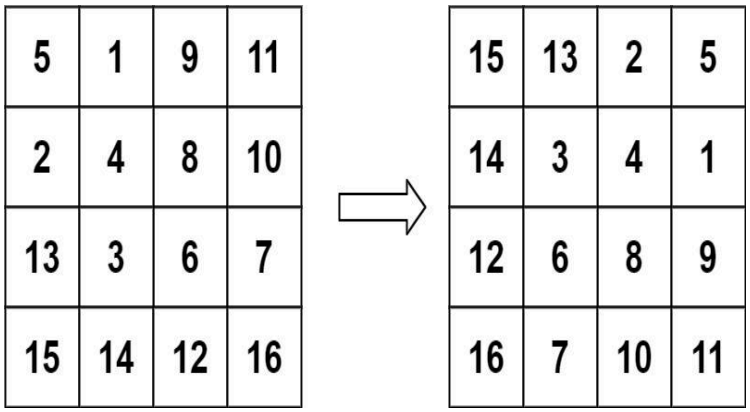
Example 1:



```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]
```

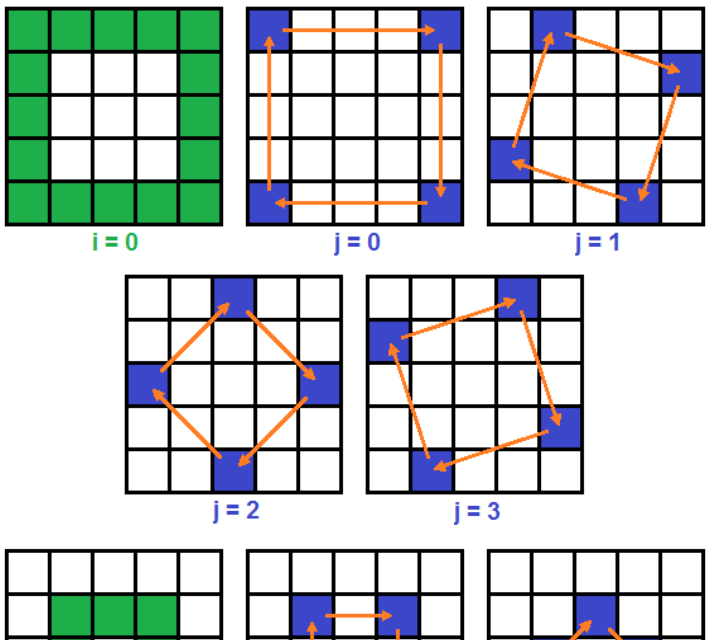
Example 2:

```
Input:matrix=[[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]
```




```
Method - 1 basic logic approach
class Solution {
    public void rotate(int[][] matrix) {
        for (int i = 0; i<matrix.length; i++){
            for(int j = i; j<matrix[0].length; j++){
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }  }
        for (int i =0 ; i<matrix.length; i++)
            for(int j = 0; j<matrix.length/2; j++){
                int temp = matrix[i][j];
                matrix[i][j] = matrix[i][matrix.length-1-j];
                matrix[i][matrix.length-1-j] = temp;  }  } }
```

```
Method - 2 Using trick for 1 pass
```



The trick here is to realize that cells in our matrix (M) can be swapped out in groups of four in a self-contained manner. This will allow us to keep our space complexity down to O(1).

The remaining difficulty lies in setting up our nested for loops to accomplish the entirety of these four-way swaps. If we consider each ring of data as a larger

iteration, we can notice that each successive ring shortens the length of its side by 2. This means that we will need to perform this process to a maximum depth of floor(n / 2) times. We can use floor here because the center cell of an odd-sided matrix will not need to be swapped.

For each ring, we'll need to perform a number of iterations equal to the length of the side minus 1, since we will have already swapped the far corner as our first iteration. As noticed earlier, the length of the side of a ring is shortened by 2 for each layer of depth we've achieved (len = n - 2 * i - 1).

Inside the nested for loops, we need to perform a four-way swap between the linked cells. In order to save on some processing, we can store the value of the opposite side of i (opp = n - 1 - i) as that value will be reused many times over.

Time Complexity: O(N^2) where N is the length of each side of the matrix & Space Complexity: O(1)

```
class Solution {
    public void rotate(int[][] M) {
        int n = M.length, depth = n / 2;
        for (int i = 0; i < depth; i++) {
            int len = n - 2 * i - 1, opp = n - 1 - i;
            for (int j = 0; j < len; j++) {
                int temp = M[i][i+j];
                M[i][i+j] = M[opp-j][i];
                M[opp-j][i] = M[opp][opp-j];
                M[opp][opp-j] = M[i+j][opp];
                M[i+j][opp] = temp;
            }
        }
    }}
```

7. Merge Intervals

Given an array of intervals where intervals[i] = [start$_i$, end$_i$], merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlaps, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

```
Method - 1 brute force
```

```
class Pair {
    int x, y;
    Pair(int x, int y) {
        this.x = x;
        this.y = y; }}

class TUF {
    public ArrayList<Pair> merge(ArrayList < Pair > arr) {

        int n = arr.size();
        Collections.sort(arr, (a, b) - > a.x - b.x);
        ArrayList < Pair > ans = new ArrayList < > ();

        for (int i = 0; i < n; i++) {
            int start = arr.get(i).x, end = arr.get(i).y;
            //since the intervals already lies
            //in the data structure present we continue
            if (!ans.isEmpty()) {
                if (start <= ans.get(ans.size() - 1).y) {
                    continue;
                }
            }

            for (int j = i + 1; j < n; j++) {
                if (arr.get(j).x <= end) {
                    end = arr.get(j).y;
                }
            }

            ans.add(new Pair(start, end));
        }

        return ans; }}
```

Method - 2 optimal approach

```
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length == 1) return intervals;

       Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

       List<int[]> li = new ArrayList();
       li.add(intervals[0]);
       int j = 1, t = 0;

       while (j < intervals.length){
           if (li.get(t)[1] < intervals[j][0]) {
               li.add(intervals[j]);
               t++;
            } else {
```

```
                li.get(t)[1] = Math.max(li.get(t)[1], intervals[j][1]);
                j++;
            }
        }

        return li.toArray(new int[0][]); }}
```

8. Missing Number and Repeating No

Given an array nums containing n distinct numbers in the range [0, n], return *the only number in the range that is missing from the array.*

Example 1:

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3].
2 is the missing number in the range since it does not appear in nums.
```

Example 2:

```
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2].
2 is the missing number in the range since it does not appear in nums.
```

```
Method - 1 using math
public int missingNumber(int[] nums) {
    int n = nums.length, sum = (n*(n+1))/2;
    for (int i = 0; i < n; i++){
        sum -= nums[i];
     }
   return sum; }
```

```
Method - 2 using xor
```

If anyone needs explanation for XOR solution,

→ a number XOR itself will become 0,

→ any number XOR with 0 will stay unchanged.

→ So if every number from 1...n XOR with itself except the missing number, the result will be the missing number.

Example:

1. 4 missing 3

```
 1^2  ^4
```

^

1^2^3^4

0^0^3^0 => 3

```
public int missingNumber(int[] nums) { //xor
    int res = nums.length;
    for(int i=0; i<nums.length; i++){
        res ^= i;
        res ^= nums[i];
    }
    return res; }
```

Method - 3 binary search

```
public int missingNumber(int[] nums) { //binary search
    Arrays.sort(nums);
    int left = 0, right = nums.length, mid= (left + right)/2;
    while(left<right){
        mid = (left + right)/2;
        if(nums[mid]>mid) right = mid;
        else left = mid+1;
    }
    return left;
}
```

For repeating part just
→ repeating = missingNumber - len(len + 1)/2;

9. Search in 2d matrix

Write an efficient algorithm that searches for a value `target` in an `m x n` integer matrix `matrix`. This matrix has the following properties:

- Integers in each row are sorted from left to right.
- The first integer of each row is greater than the last integer of the previous row.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Method - 1 concept of linear search with TC = O(M +N)

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int r = 0, c = matrix[0].length - 1;
        while (r < matrix.length && c >= 0) {
            int cur = matrix[r][c];
            if (cur == target) return true;
            else if (cur > target) c--;
            else r++;
        }
        return false;}}
```

METHOD -2 USING BINARY SEARCH

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int cols = matrix[0].length, start = 0, end = matrix.length * cols - 1;
        while (start <= end) {
            int mid = start + (end - start) / 2;
            int r = mid / cols,  c = mid % cols;
            if (target == matrix[r][c]) {
                return true;
            }
            if (target < matrix[r][c]) {
                end = mid - 1;
            } else {
                start = mid + 1;
            }
        }
        return false; }}
```

10. Implement Pow(x,n) | X raised to the power N

Implement pow(x, n), which calculates x raised to the power n (i.e., x^n).

Example 1: **Input:** x = 2.00000, n = 10 **Output:** 1024.00000

Example 2: **Input:** x = 2.10000, n = 3 **Output:** 9.26100

Example 3: **Input:** x = 2.00000, n = -2 **Output:** 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Method - 1

```
class Solution {
    public double myPow(double x, int n) {
        if (x == 0) return 0;
        if (n > 0) return powPos(x, n);
        else return (double)(1/powPos(x, n));
    }
    private double powPos(double x, int n){
```

```
        if (n == 0) return 1;
        double ans = powPos(x, n/2);
        ans *= ans;
        if (n % 2 != 0) ans = ans*x;
        return ans; }}
```

```
Method - 2 does not for for neg no.s
public double myPow(double x, int n) {
    double ans = 1.0;
    for(int i = 0; i<n; i++)   ans = ans * x;
    return ans;}
```

Method - 3 Time Complexity: O(log n)

```
class Solution {
    public double myPow(double x, int n) {
        double ans = 1.0;
        long nn = n;
        if (nn < 0) nn = -1 * nn;
        while (nn > 0) {
          if (nn % 2 == 1) {
            ans = ans * x;
            nn = nn - 1;
          } else {
            x = x * x;
            nn = nn / 2;
          }
        }
        if (n < 0) ans = (double)(1.0) / (double)(ans);
        return ans; }}
```

11. Find the Majority Element that occurs more than N/2 times

Given an array nums of size n, return *the majority element*.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1: `Input: nums = [3,2,3] Output: 3`

Example 2: `Input: nums = [2,2,1,1,1,2,2] Output: 2`

```
Method - 1 Optimal Moore's voting algo TC = N and SC = 1
class Solution {
    public int majorityElement(int[] nums) {
        int n = nums.length, cnt = 0, ans = 0;
        for (int i = 0; i < n; i++){
            if (cnt == 0) ans = nums[i];
            if (nums[i] == ans) cnt++;
```

```
            else cnt--;
        }
        return ans; }}
```

Method - 2 using hashmap TC = N and SC = N

```
class Solution {
    public int majorityElement(int[] nums) {
        int n = nums.length;
        HashMap<Integer, Integer> map = new HashMap();
        for (int i = 0; i < n; i++){
            map.put(nums[i], map.getOrDefault(nums[i], 0) + 1);
            if (map.get(nums[i]) > n/2)
                return nums[i];
        }
        return 0;}}
```

OTHER METHODS
3. SORTING AND COUNTING
4. USING FREQ. ARRAY

12. Find the Majority Element that occurs more than N/3 times

Given an integer array of size `n`, find all elements that appear more than `⌊ n/3 ⌋` times.

Example 1: `Input: nums = [3,2,3] Output: [3]`

`METHOD - 1 SAME AS THE METHOD OF HASHMAP IN PREV. INSTEAD USE N/3 OF N/2`

`METHOD - 2 EXTENDED MOORE'S ALGO`

Approach + Intuition: In our code, we start with declaring a few variables:
num1 and num2 will store our currently most frequent and second most frequent element.
c1 and c2 will store their frequency relatively to other numbers.
We are sure that there will be a max of 2 elements which occurs > N/3 times because there cannot be if you do a simple math addition.
Let, ele be the element present in the array at any index.

- if ele == num1, so we increment c1.
- if ele == num2, so we increment c2.
- if c1 is 0, so we assign num1 = ele.
- if c2 is 0, so we assign num2 = ele.
- In all the other cases we decrease both c1 and c2.

In the last step, we will run a loop to check if num1 or nums2 are the majority elements or not by running a for loop check.
Intuition: Since it's guaranteed that a number can be a majority element, hence it will always be present at the last block, hence, in turn, will be on nums1 and nums2. For a more detailed explanation, please watch the video below.

```
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int number1 = -1, number2 = -1, count1 = 0, count2 = 0, len = nums.length;
```

```
        for (int i = 0; i < len; i++) {
          if (nums[i] == number1)
            count1++;
          else if (nums[i] == number2)
            count2++;
          else if (count1 == 0) {
            number1 = nums[i];
            count1 = 1;
          } else if (count2 == 0) {
            number2 = nums[i];
            count2 = 1;
          } else {
            count1--;
            count2--;
          }
        }
        ArrayList <Integer> ans = new ArrayList <Integer> ();
        count1 = 0; count2 = 0;
        for (int i = 0; i < len; i++) {
          if (nums[i] == number1)
            count1++;
          else if (nums[i] == number2)
            count2++;
        }
        if (count1 > len / 3) ans.add(number1);
        if (count2 > len / 3) ans.add(number2);
        return ans; }}
```

13. Longest Consecutive Sequence

Given an unsorted array of integers nums, return *the length of the longest consecutive elements sequence.* You must write an algorithm that runs in O(n) time.

Example 1:

```
Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its
length is 4.
```

```
METHOD - 1 SORT AND CHECK
```

```
int longestConsecutive(vector<int> nums) {
        if(nums.size() == 0 ){ return 0; }
        sort(nums.begin(),nums.end());
        int ans = 1, prev = nums[0], cur = 1;
        for(int i = 1;i < nums.size();i++){
            if(nums[i] == prev+1){
                cur++;
            }
            else if(nums[i] != prev){
                cur = 1;
```

```
            }
            prev = nums[i];
            ans = max(ans, cur);
        }
        return ans;}
```

```
METHOD - 2 OPTIMAL LOOKING LEFT AND RIGHT
```

```
public int longestConsecutive(int[] num) {
  int max = 0;
  Set<Integer> set = new HashSet<Integer>();
  for (int i = 0; i < nums.length; i++) set.add(nums[i]);

  for (int i = 0; i < nums.length; i++) {
    int count = 1;
    // look left
    int num = nums[i];
    while (set.contains(--num)) {
      count++; set.remove(num);
    }
    // look right
    num = nums[i];
    while (set.contains(++num)) {
      count++; set.remove(num);
    }
    max = Math.max(max, count);
  }
  return max;}
```

14. Subarray Sum Equals K

Given an array of integers nums and an integer k, return *the total number of subarrays whose sum equals to k*.

Example 1:

```
Input: nums = [1,1,1], k = 2
Output: 2
```

Example 2:

```
Input: nums = [1,2,3], k = 3
Output: 2
```

```
METHOD - 1 BRUTE FORCE
```

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int  max = 0;
        for(int i = 0; i < a.length; ++i){
            int sum = 0;
```

```
            for(int j = i; j < a.length; ++j){
                sum += a[j];
                if(sum == 0){
                    max = Math.max(max, j-i+1);
                }
            }
        }
        return max; }}
```

METHOD - 2 USING HASHMAP

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        LinkedHashMap<Integer, Integer> map = new LinkedHashMap();
        int cnt = 0, sum = 0, N = nums.length;
        for (int i = 0; i < N; i++){
            sum += nums[i];
            if (sum == k) cnt++;
            cnt += map.getOrDefault(sum - k, 0);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        System.out.println(map.entrySet());
        return cnt; }}
```

15. Count the number of subarrays with given xor K

Problem Statement: Given an array of integers A and an integer B. Find the total number of subarrays having bitwise XOR of all elements equal to B.

Examples: **Input Format**: A = [4, 2, 2, 6, 4] , B = 6 **Result**: 4
Explanation: The subarrays having XOR of their elements as 6 are [4, 2], [4, 2, 2, 6, 4], [2, 2, 6], [6]

Input Format: A = [5, 6, 7, 8, 9], B = 5 **Result:** 2
Explanation: The subarrays having XOR of their elements as 2 are [5] and [5, 6, 7, 8, 9]

Method 1: Brute Force
Intuition: The brute force solution is to generate all possible subarrays. For each generated subarray we get the respective XOR and then check if this XOR is equal to B. If it is then we increment the count. In the end, we will get the count of all possible subarrays that have XOR equal to B.
Approach:
Generate subarrays: To generate all possible subarrays, we use the same old technique of nested loops. For each value of the outer loop (i loop), the inner loop(j loop) runs from i till the last element. Each iteration of the inner loop gives a new subarray.
Maintain XOR: Since each iteration of the inner loop gives a new subarray, we maintain a variable X, in which we keep the XOR of the current subarray.

Check and Count: Before moving to the next iteration of the inner loop (that is before going to the next subarray), we check if the current XOR is equal to B, if it is then we increment the counter (counter also has to be maintained).

```
public class Solution {
    public int solve(int[] A, int B) {
        int c=0;
        for(int i=0;i<A.length;i++){
            int current_xor = 0;
            for(int j=i;j<A.length;j++){
                current_xor^=A[j];
                if(current_xor==B) c++;
            }
        }
        return c; }}
```

Method 2: Prefix xor and map

Intuition: The main idea is to observe the prefix xor of the array. Prefix Xor is just another array, where each index contains XOR of all elements of the original array starting from index 0 up to that index. In other words prefix_xor[i] = XOR(a[0], a[1], a[2],……,a[I])

Once we have made the prefix xor array, we observe a fact:

P = xor(a[0], a[1], a[2],……, a[q], a[q+1],….., a[p])

Q = xor(a[0], a[1], a[2],……, a[q])

Therefore,

P^Q = xor(a[q+1],….., a[p]) = M

So, now we understand that from the prefix xor array when we XOR two elements at different indices we get the xor of the elements (in the original array) that were between those indices.

Let's say we did XOR(P, B) and we got Q (B is the integer given in question). What does this mean?

This means that the subarray between q and p is having xor = B. To understand this we just use simple equations:

P^B = Q

=> P^B^P = Q^P

=> B = Q^P

And we already know by fact 1 that Q^P will give xor of all elements between q and p. Therefore, the subarray between q and p has xor = B.

Suppose we did XOR(P, B) and we got Q (B is the integer given in question). But there is more than one Q before p. In this case, there are two subarrays that have XOR = B. Subarrays between q1 to p and q2 to p.

IMP NOTE: although we talk about prefix xor "array", it should be noted that at a time we need only one element of this array. Hence, we can just use a variable to maintain the prefix xor.

Approach: We need to traverse the array while we maintain variables for current_perfix_xor, counter, and also a map to keep track of visited xors. This map will maintain the frequency count of all previous visited current_prefix_xor values. If for any index current_prefix_xor is equal to B, we increment the counter. If for any index we find that Z is present in the visited map, we increment the counter by visited[Z] (Z=current_prefi_xor^B). At every index, we insert the current_prefix_xor into the visited map with its frequency.

```
public class Solution {
    public int solve(int[] A, int B) {
     HashMap<Integer,Integer> visited = new HashMap<Integer,Integer>();
        int c = 0, cpx = 0,  n = A.length;
        for(int i = 0;i<n;i++) {
            cpx = cpx ^ A[i];
            if(visited.get(cpx^B) != null)
                c += visited.get(cpx ^ B);
            if(cpx == B) {
                c++;
            }
```

```
            if(visited.get(cpx) != null)
                visited.put(cpx, visited.get(cpx) + 1);
            else
                visited.put(cpx, 1);
        }
        return c;  }}
```

16. Length of Longest Substring without any Repeating Character

Problem Statement: Given a String, find the length of longest substring without any repeating character.

Example 1:Input: s = ”abcabcbb” **Output:** 3 **Explanation:** The answer is abc with length of 3.

Method 1: Brute force Approach

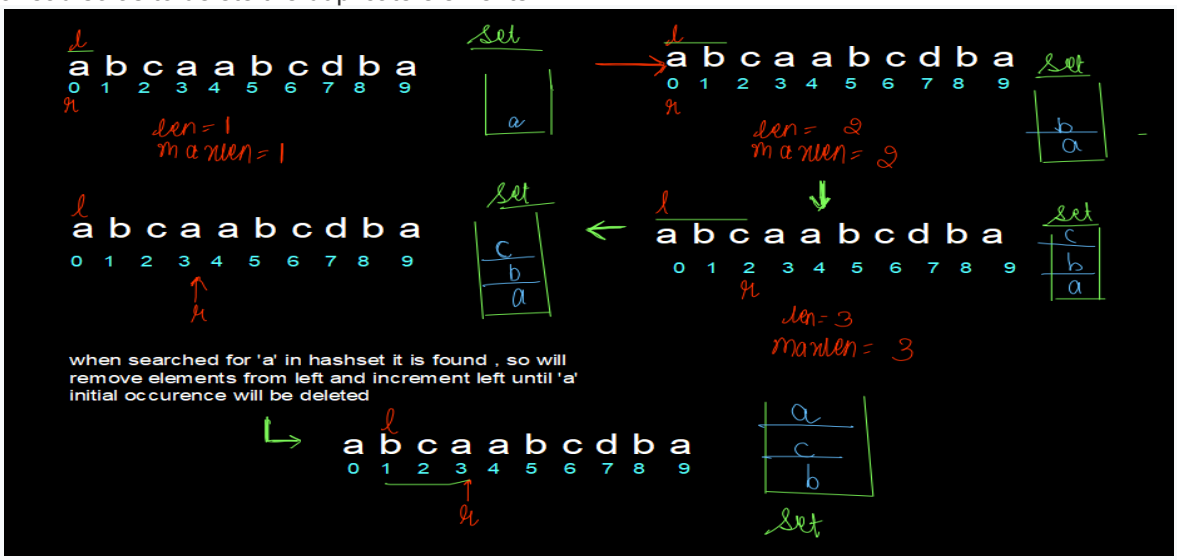
Approach: This approach consists of taking the two loops one for traversing the string and another loop – nested loop for finding different substrings and after that, we will check for all substrings one by one and check for each and every element if the element is not found then we will store that element in HashSet otherwise we will break from the loop and count it.

```
public int solve(String str) {
    int maxans = Integer.MIN_VALUE;
    for (int i = 0; i < str.length(); i++){ // outer loop for traversing the string
            Set < Character > se = new HashSet < > ();
            for (int j = i; j < str.length(); j++){ // nested loop for getting
different string starting with str[i]
                if (se.contains(str.charAt(j))){ // if element if found so mark it as
ans and break from the loop
                    maxans = Math.max(maxans, j - i);
```

```
                    break;
                }
                se.add(str.charAt(j));
            }
        }
        return maxans; }}
```

Method - 2: Optimized Approach

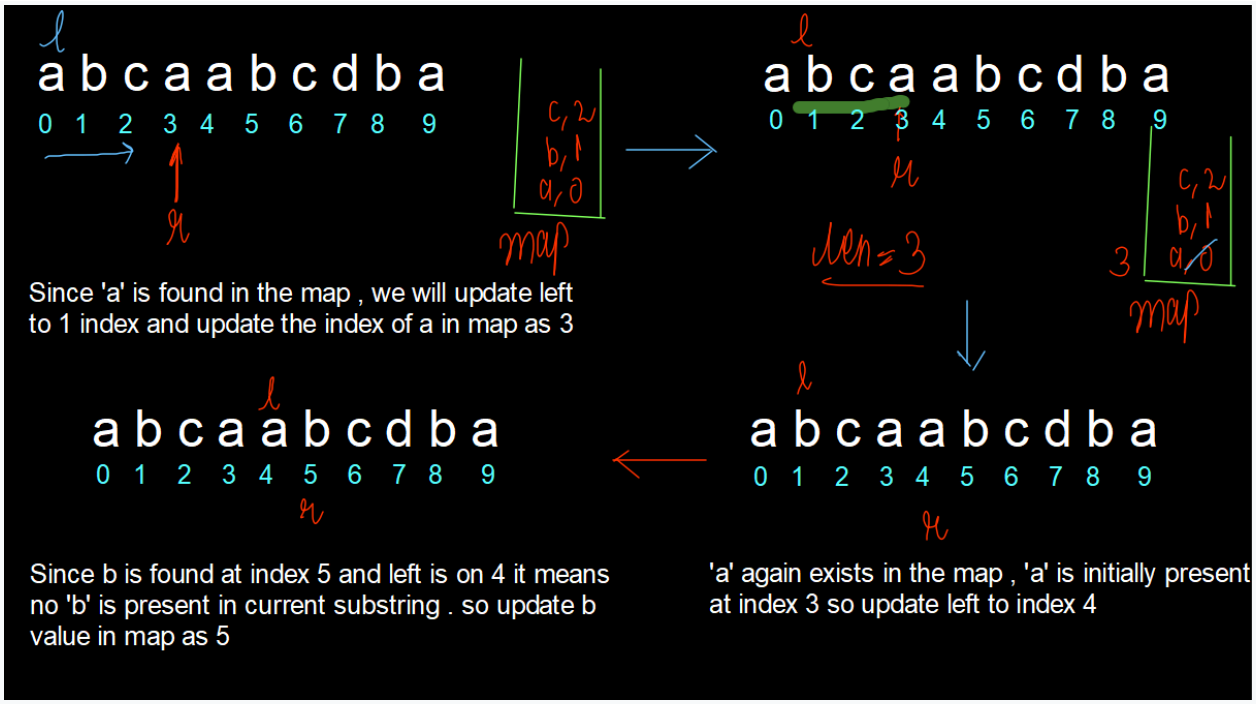
Approach: We will have two pointers left and right. Pointer 'left' is used for maintaining the starting point of the substring while 'right' will maintain the endpoint of the substring.' right' pointer will move forward and check for the duplicate occurrence of the current element; if found then the 'left' pointer will be shifted ahead so as to delete the duplicate elements.




```
public int solve(String str) {
        int maxans = Integer.MIN_VALUE;
        Set < Character > set = new HashSet < > ();
        int l = 0;
        for (int r = 0; r < str.length(); r++) // outer loop for traversing the
string
        {
            if (set.contains(str.charAt(r))) //if duplicate element is found
            {
                while (l < r && set.contains(str.charAt(r))) {
                    set.remove(str.charAt(l));
                    l++;
                }
            }
            set.add(str.charAt(r));
            maxans = Math.max(maxans, r - l + 1);
        }
        return maxans; }
```

Method 3: Optimized Approach 2

Approach: In this approach, we will make a map that will take care of counting the elements and maintaining the frequency of each and every element as a unity by taking the latest index of every element.




```
static int solve(String s) {
        HashMap < Character, Integer > mpp = new HashMap < Character, Integer > ();

        int left = 0, right = 0;
        int n = s.length();
        int len = 0;
        while (right < n) {
            if (mpp.containsKey(s.charAt(right))) left =
Math.max(mpp.get(s.charAt(right)) + 1, left);

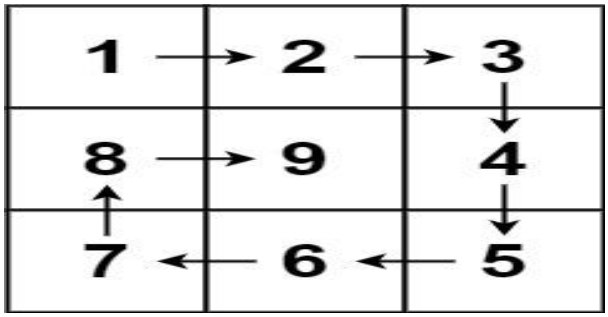
            mpp.put(s.charAt(right), right);

            len = Math.max(len, right - left + 1);
            right++;
        }
        return len; }
```

17. Spiral Matrix

Given a positive integer `n`, generate an `n x n` `matrix` filled with elements from `1` to n^2 in spiral order.

Example 1:




```
Input: n = 3
Output: [[1,2,3],[8,9,4],[7,6,5]]
```

Using 4 ptr approach

```
class Solution {
    public int[][] generateMatrix(int n) {
        int[][] M = new int[n][n];
        int top = 0, bottom = n - 1, right = n - 1, left = 0, N = 1;
        while (right >= left){
            for (int i = left; i <= right; i++, N++) M[top][i] = N;
            top++;

            for (int i = top; i <= bottom; i++, N++) M[i][right] = N;
            right--;

            for (int i = right; i >= left; i--, N++) M[bottom][i] = N;
            bottom--;

            for (int i = bottom; i >= top; i--, N++) M[i][left] = N;
            left++;
        }
        return M; }}
```

Similar q

II. STRINGS

1. Longest Substring Without Repeating Characters

Given a string `s`, find the length of the longest substring without repeating characters.

Example 1:

```
Input: s = "abcabcbb"   Output: 3
Explanation: The answer is "abc", with the length of 3.
```

Example 2:

```
Input: s = "bbbbb"    Output: 1

Explanation: The answer is "b", with the length of 1.
```

METHOD - 1 USING HASHSET

```
public int lengthOfLongestSubstring(String s) {
    int i = 0, j = 0, max = 0;
    Set<Character> set = new HashSet<>();

    while (j < s.length()) {
        if (!set.contains(s.charAt(j))) {
            set.add(s.charAt(j++));
            max = Math.max(max, set.size());
        } else {
            set.remove(s.charAt(i++));
        }
    }

    return max;
}
```

METHOD - 2 Using HashMap

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int ans = 0;
        Map<Character, Integer> map = new HashMap<>();
        for (int i = 0, j = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (map.containsKey(c)) {
                j = Math.max(j, map.get(c) + 1);
            }
            map.put(c, i);
            ans = Math.max(ans, i - j + 1);
        }
        return ans;
    }
}
```

METHOD - 3 USING ASCII ARRAY

```
public class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] mOccur = new int[256]; // IT MAY CONTAIN OTHER CHARS LIKE ' ' ETC..
        int maxL = 0;
        for (int i = 0, j = 0; i < s.length(); ++i){
            char ch = s.charAt(i);
            ++mOccur[ch];
            while (mOccur[ch] > 1){
                --mOccur[s.charAt(j++)];
            }
            maxL = Math.max(maxL, i - j + 1);
        }
        return maxL;
    }
}
```

METHOD - 4 USING Q DS

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
```

```
        Queue<Character> q = new LinkedList();
        int ans = 0;
        for (int i = 0; i < s.length(); i++){
            char cur = s.charAt(i);
            while (q.contains(cur)){
                q.poll();
            }
            q.add(s.charAt(i));
            ans = Math.max(ans, q.size());
        }
        return ans;
    }
}
```

2. ZigZag Conversions

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P   A   H   N
A P L S I I G
Y   I   R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows: `string convert(string s, int numRows);`

Example 1:

```
Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"
```

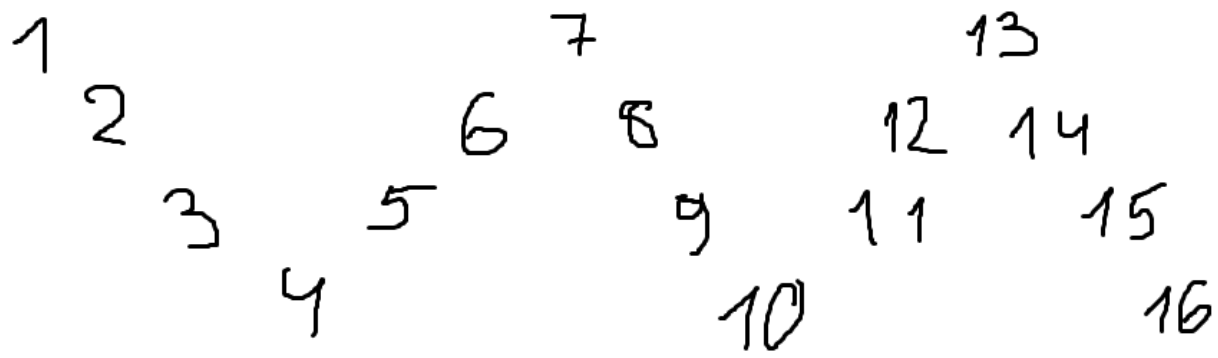
Example 2:

```
Input: s = "PAYPALISHIRING", numRows = 4
Output: "PINALSIGYAHRPI"
Explanation:
P     I    N
A   L S  I G
Y A   H R
P     I
```

`Solution - 1`

The idea is simple: I've calculated the formula that identifies the order of characters in zigzag.

Here is an example for numRows = 4 (just contiguous numbers instead letters):



1st row : 1,7,13 (or 1, 1+2*(4-1), 1+2*(4-1)+2*(4-1) = 1+k*2(n-1))

2nd row: 2,6,8,12,14 (or 2, 2+2*(4-2), 2+2*(4-2)+2*(2-1),2+2*(4-2)+2*(2-1)+2*(4-2), ... 2+2(n-2) + 2(2-1))

3th row:3,5,9,11,15 (or 3, 3+2(n-3), 3+2(n-3)+2*(3-1), ... 3+2(n-3) + 2(3-1))

4th row:4,10,16 (or 4, 4+2(4-1), 4+2(4-1)+2(4-1), ... 4+k*2(4-1))

So general formula will look like this:

$$P_n = \begin{cases} i=1 \Rightarrow P_{n-1} + 2*(n-1) \\ i=n \Rightarrow P_{n-1} + 2*(n-1) \\ \text{else: up} = \begin{cases} \text{true: } P_{n-1} + 2*(n-i) \\ \text{false: } P_{n-1} + 2*(i-1) \end{cases} \end{cases}$$

The whole code snippet:

ind - means what character (at what position in input string) we append to builder at current iteration.

```
class Solution {
    public String convert(String s, int numRows) {
        if (numRows == 1) return s;
        int n = s.length();
        String ans = "";
        for (int i = 1;i <= numRows;i++){
            int ind = i - 1;
```

```
            boolean up = true;
            while (ind < n){
                ans += s.charAt(ind);
                if (i == 1){
                    ind += 2*(numRows-i);
                } else if (i == numRows){
                    ind += 2*(i - 1);
                } else {
                    ind += up ? 2*(numRows - i) : 2*(i - 1);
                    up = !up;
                }
            }
        }
        return ans; }}
```

3. Valid Palindrome II

Given a string s, return true *if the* s *can be palindrome after deleting at most one character from it*.

Example 1:

```
Input: s = "aba" Output: true
Input: s = "abc" Output: false
```

Example 2:

```
Input: s = "abca" Output: true
Explanation: You could delete the character 'c'.
```

```
class Solution {
    public boolean validPalindrome(String s) {
        int n = s.length(), i = 0, j = n - 1;
        while (i < j){
            if (s.charAt(i) != s.charAt(j)){
                return isPalin(s, i, j - 1) || isPalin(s, i + 1, j);
            } i++; j--;
        }
        return true; }
    private boolean isPalin(String s, int i, int j){
        while (i < j){
            if (s.charAt(i) != s.charAt(j)){
                return false;
            }  i++; j--; }
        return true; } }
class Solution {
    public boolean validPalindrome(String s) {
        return isPalin(s, false);
    }
    private boolean isPalin(String s, boolean b){
        int n = s.length(), i = 0, j = n - 1;
```

```
        while (i < j){
            if (s.charAt(i) != s.charAt(j)){
             if (!b){
                   return isPalin(s.substring(0, i) + s.substring(i + 1, n), true) ||
                             isPalin(s.substring(0, j) + s.substring(j + 1, n), true);
                }
                else return false;
            }
            i++; j--;
        }
        return true;
    }
}
```

4. Print Anagrams Together (A)

Given an array of strings, return all groups of strings that are anagrams. The groups must be created in order of their appearance in the original array. Look at the sample case for clarification.

Note: The finial output will be in lexicographic order.

Example 1:

Input:
N = 5
words[] = {act,god,cat,dog,tac}
Output:
act cat tac
god dog
Explanation:
There are 2 groups of
anagrams "god", "dog" make group 1.
"act", "cat", "tac" make group 2.

```
class Solution {
    public List<List<String>> Anagrams(String[] arr) {
        List<List<String>>res=new ArrayList<>();
        HashMap<String,List<String>>map=new HashMap<>();
        for(int i=0;i<arr.length;i++) {
            String word=arr[i];
            char[] value=word.toCharArray();
            Arrays.sort(value);
            String b=new String(value);
            if(map.containsKey(b))  {
                map.get(b).add(word);
            } else {
                List<String>a=new ArrayList<>();
                a.add(arr[i]);
                map.put(b,a);
            }
```

```
        }
        for(String e: map.keySet())  {
            res.add(map.get(e));
        }
        return res;
    }
}
```

III. SLIDING WINDOW PROBLEMS

1. Longest Palindromic Substring

Given a string s, return *the longest palindromic substring* in s.

Example 1:

```
Input: s = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.
```

Example 2:

```
Input: s = "cbbd"
Output: "bb"
```

```
Method - 1 expanding(sliding) from middle

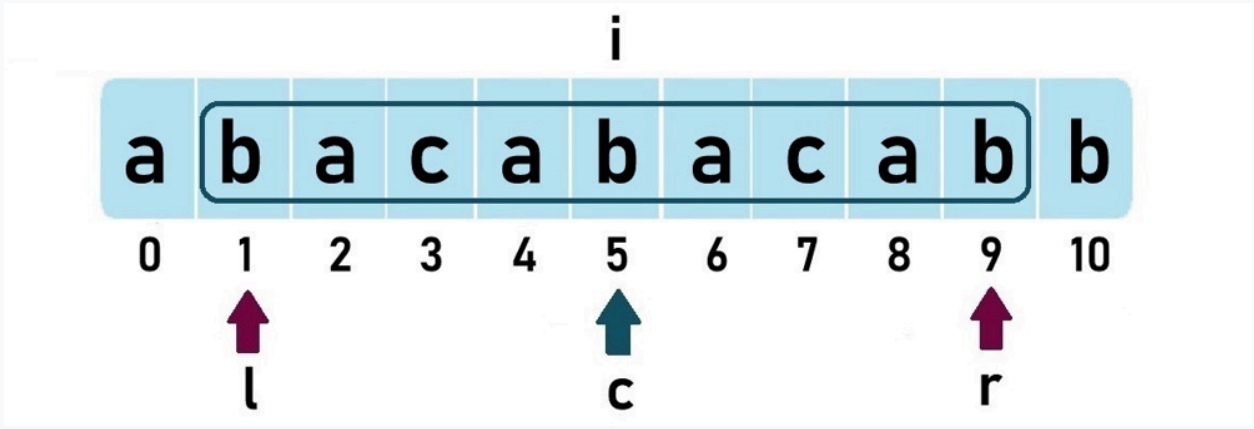
class Solution {
    public String longestPalindrome(String s) {
        String ans = "";
        for (int i = 0; i < s.length(); i++){
            String s1 = expandFromM(s, i, i); // odd
            if (ans.length() < s1.length()){
                ans = s1;
```

```
            }
            String s2 = expandFromM(s, i, i + 1); // even
            if (ans.length() < s2.length()){
                ans = s2;
            }
        }
        return ans;
    }
    private String expandFromM(String s, int lt, int rt){
        if (lt > rt) return "";
        while (lt >= 0 && rt < s.length() && s.charAt(lt) == s.charAt(rt)){
            lt--; rt++;
        }
        return s.substring(lt + 1,  rt);
    }
}
```

Approach 2: Manacher's Algorithm

In Approach 4, we do lots of comparisons which are unnecessary. Manacher's algorithm achieves the time complexity O(n) by skipping the unnecessary comparisons.

e.g.



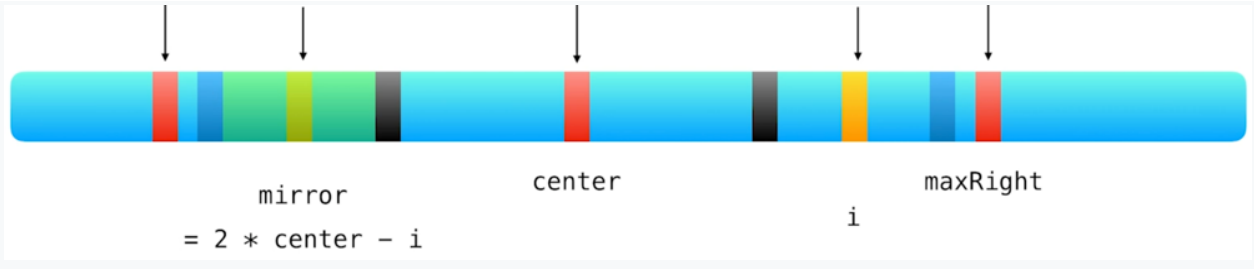
For indice 8 which is a, we don't need to do the expandAroundCenter since we have learned from indice 2, that the expand length is 0. And to make it true, the indice 8 and indice 2 should be in the same palindrome. Meanwhile they have the same distance to the center of the palindrome.

We could divide the problem into several cases.

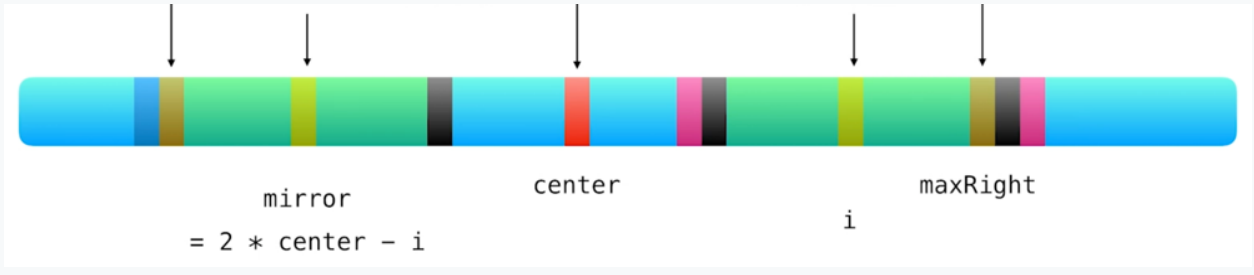
p[i] means the expanded length of substring with the center of indice i .

[a, a, b, c, b, a, d] => p[3] = 2

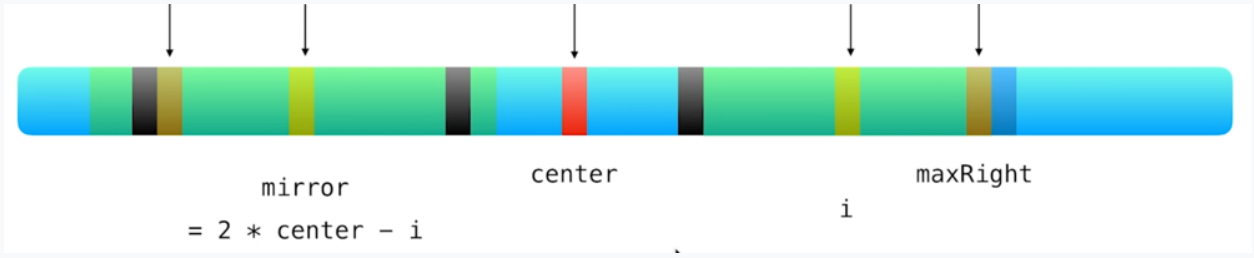
1. i + p[mirror] < maxRight



2. i + p[mirror] == maxRight



3. i + p[mirror] > maxRight




```
if i >= maxRight
    expand the substring with the center of i
else if i + p[mirror] < maxRight
```

```
    p[i] = p[mirror]
else if i + p[mirror] == maxRight
    p[i] = p[mirror] = maxRight - i
    expand the substring with the center of i, beginning from maxRight
else
    p[i] = maxRight - i
```

We actually could make the code more concise:

```
if i < maxRight
    p[i] = min(p[mirror], maxRight - i)

expand substring with center of i, beginning from i + p[i]
```

The algorithm only solve the string with odd size, so we add '#' to the string to make the size of string always be odd.

Approach - 2 (MANCHESTER ALGO)

```java
class Solution {
    public String longestPalindrome(String s) {
        // code line +8 to line +15
        int strLen = 2 * s.length() + 3;
        char[] sChars = new char[strLen];

        /*
         * To ignore special cases at the beginning and end of the array
         * "abc" -> @ # a # b # c # $
         * "" -> @#$
         * "a" -> @ # a # $
         */
        sChars[0] = '@';
        sChars[strLen - 1] = '$';
        int t = 1;
        for (char c : s.toCharArray()) {
            sChars[t++] = '#';
            sChars[t++] = c;
        }
        sChars[t] = '#';

        int maxLen = 0;
        int start = 0;
        int maxRight = 0;
```

```
        int center = 0;
        int[] p = new int[strLen]; // i's radius, which not includes i
        for (int i = 1; i < strLen - 1; i++) {
            if (i < maxRight) {
                p[i] = Math.min(maxRight - i, p[2 * center - i]);
            }

            // expand center
            while (sChars[i + p[i] + 1] == sChars[i - p[i] - 1]) {
                p[i]++;
            }

            // update center and its bound
            if (i + p[i] > maxRight) {
                center = i;
                maxRight = i + p[i];
            }

            // update ans
            if (p[i] > maxLen) {
                start = (i - p[i] - 1) / 2;
                maxLen = p[i];
            }
        }

        return s.substring(start, start + maxLen);
    }
}
```

Approach 3: Dynamic Programming

We define $P(i, len)$ as following:

$$P(i, len) = \begin{cases} \text{true}, & \text{if the substring } S_i \dots S_{i+len} \text{ is a palindrome} \\ \text{false}, & \text{otherwise} \end{cases}$$

Therefore,

$$P(i, len) = (P(i+1, len-2) \text{ and } Si == Si + len)$$

The base cases are:

$$P(i, 0) = \text{true}$$
$$P(i, 1) = (S_i == S_{i+1})$$

Table 1

		len				
	abbbc	0	1	2	3	4
i	0	TRUE	FALSE	FALSE	FALSE	FALSE
	1	TRUE	TRUE	TRUE	FALSE	
	2	TRUE	TRUE	FALSE		
	3	TRUE	FALSE			
	4	TRUE				
	currCol	0	1	0	1	0

```
class Solution {
```

```java
    public String longestPalindrome(String s) {
        char[] sChars = s.toCharArray();

        int m = s.length();

        // * dp[i][len + 1] means substring starting from i with length of len;
        boolean[][] dp = new boolean[m][2];
        int currCol = 0;

        int maxLen = 0;
        int ans = 0; // record start index of s

        for (int len = 0; len < m; len++) {
            for (int start = 0; start + len < m; start++) {
                int end = start + len;
                if (len == 0) {
                    dp[start][currCol] = true;
                } else if (len == 1) {
                    dp[start][currCol] = (sChars[start] == sChars[end]);
                } else {
                    dp[start][currCol] = (sChars[start] == sChars[end] && dp[start +
1][currCol]);
                }

                if (dp[start][currCol] && len + 1 > maxLen) {
                    ans = start;
                    maxLen = len + 1;
                }
            }

            currCol = 1 - currCol;
        }

        return maxLen == 0 ? "" : s.substring(ans, ans + maxLen);
    }
}
```

5. Find All Anagrams in a String

Given two strings `s` and `p`, return *an array of all the start indices of* `p`*'s anagrams in* `s`. You may return the answer in any order.

An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

`Input: s = "cbaebabacd", p = "abc"`

`Output: [0,6]`

`Explanation:`

`The substring with start index = 0 is "cba", which is an anagram of "abc".`

`The substring with start index = 6 is "bac", which is an anagram of "abc".`

`Approach - 1 (sliding window)`

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        int sLen = s.length(), pLen = p.length();
        if (sLen * pLen == 0 || sLen < pLen) {
            return result;
        }

        Map<Character, Integer> map = new HashMap<>();
        for (int i = 0; i < p.length(); i++) {
            map.put(p.charAt(i), map.getOrDefault(p.charAt(i), 0) + 1);
        }

        int toBeMatched = map.size(), start = 0, end = 0;

        while (end < sLen) {
            char eChar = s.charAt(end);
            if (map.containsKey(eChar)) {
                int count = map.get(eChar);
                if (count == 1) toBeMatched--;
                map.put(eChar, count - 1);
            }
            end++;
            if (end - start > pLen) {
                char sChar = s.charAt(start);
                if (map.containsKey(sChar)) {
                    int count = map.get(sChar);
                    if (count == 0) toBeMatched++;
                    map.put(sChar, count + 1);
                }
```

```
                start++;
            }

            if (toBeMatched == 0) result.add(start);
        }
        return result; }}
```

Approach - 2

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int freq1[] = new int[26];
        int freq2[] = new int[26];
        List<Integer> list = new ArrayList<>();
        if(s.length() < p.length())
            return list;

        for(int i = 0; i < p.length(); i++){
            freq1[s.charAt(i) - 'a']++;
            freq2[p.charAt(i) - 'a']++;
        }

        int start = 0, end = p.length();
        if (Arrays.equals(freq1,freq2))
            list.add(start);

        while(end < s.length()){

            freq1[s.charAt(start)-'a']--;
            freq1[s.charAt(end)-'a']++;

            if (Arrays.equals(freq1,freq2))
            list.add(start+1);

            start++; end++;
        }
      return list; }}
```

Approach - 3

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> list = new ArrayList<Integer>();
        int[] arr = new int[26];

        for(int i = 0;i < p.length(); i++)
            arr[p.charAt(i)-'a']++;

        for(int i = 0; i <= s.length()-p.length(); i++){
            String temp = s.substring(i, p.length() + i);
            int[] arr1 = new int[26];
            for(int j = 0;j < p.length(); j++){
                arr1[temp.charAt(j) - 'a']++;
```

```
            }
            if (Arrays.equals(arr,arr1))
                list.add(i);
        }
        return list;}}
```

6. Minimum Size Subarray Sum

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a contiguous subarray $[nums_l, nums_{l+1}, ..., nums_{r-1}, nums_r]$ of which the sum is greater than or equal to `target`. If there is no such subarray, return `0` instead.

Example 1:

```
Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem constraint.
```

Example 2:

```
Input: target = 4, nums = [1,4,4]
Output: 1
```

Example 3:

```
Input: target = 11, nums = [1,1,1,1,1,1,1,1]
Output: 0
```

METHOD - 1 (SLIDING WINDOW)

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int len = nums.length, start = 0, end = 0, minLen = len + 1;
        while (end < len) {
            target -= nums[end];
            end++;

            while (target <= 0) {
                minLen = Math.min(minLen, end - start);
                target += nums[start];
                start++;
            }
        }

        return minLen % (len + 1); }}
```

METHOD - 2 USING Two Pointer approach

```
public int minSubArrayLen(int s, int[] nums) {
        int sum = 0, from = 0, win = Integer.MAX_VALUE;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            while (sum >= s) {
                win = Math.min(win, i - from + 1);
                sum -= nums[from++];
            }
        }
        return (win == Integer.MAX_VALUE) ? 0 : win;
    }
```

METHOD - 3 USING HASHMAP

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int min = Integer.MAX_VALUE, cnt = target;
        HashMap<Integer, Integer> map = new HashMap();
        map.put(0, -1);
        for (int i = 0; i < nums.length; i++){
            if (cnt <= 0) {
                cnt = target;
            }
            cnt -= nums[i];
            if (map.containsKey(cnt) && cnt <= 0){
                min = Math.min(min, i - map.get(cnt));
            }else{
                map.put(cnt, i);
            }
        }
        return (min == Integer.MAX_VALUE) ? 0 : min; }}
```

Method - 4 diff. Approach

```
public int minSubArrayLen(int s, int[] A) {
        int i = 0, n = A.length, res = n + 1;
        for (int j = 0; j < n; ++j) {
            s -= A[j];
            while (s <= 0) {
                res = Math.min(res, j - i + 1);
                s += A[i++];
            }
        }
        return res % (n + 1);
    }
```

METHOD - 5 USING BINARY SEARCH

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int len = nums.length, start = 1, end = len, minLen = len + 1;

        while (start <= end) {
            int mid = start + (end - start) / 2;
            int foundWindowSize = windowExists(nums, target, mid);
            if (foundWindowSize > 0) {
                end = foundWindowSize - 1;
                minLen = foundWindowSize;
            } else {
                start = mid + 1;
            }
        }
        return minLen % (len + 1); }

    private int windowExists(int[] nums, int target, int maxWindowSize) {
        int foundWindowSize = 0;
        for (int i = 0; i < nums.length; i++) {
            target -= nums[i];
            foundWindowSize++;
            if (i >= maxWindowSize) {
                foundWindowSize--;
                target += nums[i - maxWindowSize];
            }

            if (target <= 0) {
                return foundWindowSize;
            }
        }

        return -1;} }
```

7. Contains Duplicate II

Given an integer array nums and an integer k, return true if there are two distinct indices i and j in the array such that nums[i] == nums[j] and abs(i - j) <= k.

Example 1:

```
Input: nums = [1,2,3,1], k = 3
```

```
Output: true
```

Example 2:

```
Input: nums = [1,0,1,1], k = 1
Output: true
```

Example 3:

```
Input: nums = [1,2,3,1,2,3], k = 2
Output: false
```

Method - 1 sliding window

```
class Solution1 {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        if (nums.length <= 1 || k == 0) {
            return false;
        }

        HashSet<Integer> window = new HashSet<>();
        for (int i = 0; i < nums.length; i++) {
            if (i > k) {
                window.remove(nums[i - k - 1]);
            }
            if (! window.add(nums[i])) { // not able to further add so it's true
                                         if element is present
                return true;
            }
        }
       // max size of set is k in this problem
        return false; }}
```

Method - 2 using hashmap

```
class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        HashMap<Integer, Integer> hm = new HashMap<>();
        for (int i = 0; i < nums.length; i++){
            if (hm.containsKey(nums[i])){
                if (i - hm.get(nums[i]) <= k)
                    return true;
            }
            hm.put(nums[i], i);
        }
        return false; }}
```

8. Permutation in String

Given two strings s1 and s2, return true *if* s2 *contains a permutation of* s1*, or* false *otherwise*.

In other words, return true if one of s1's permutations is the substring of s2.

Example 1:

```
Input: s1 = "ab", s2 = "eidbaooo"
Output: true
Explanation: s2 contains one permutation of s1 ("ba").
```

Example 2:

```
Input: s1 = "ab", s2 = "eidboaoo"
Output: false
```

Method - 1 using sliding window

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int[] counter1 = new int[26];
        int[] counter2 = new int[26];

        for (int i = 0; i < s1.length(); i++) {
            counter1[s1.charAt(i) - 'a'] += 1;
        }

        for (int i = 0; i < s2.length(); i++) {
            counter2[s2.charAt(i) - 'a'] += 1;

            if (i >= s1.length()) {
                counter2[s2.charAt(i - s1.length()) - 'a'] -= 1;
            }

            if (Arrays.equals(counter1, counter2)) {
                return true;
            }
        }

        return false;  }}
```

Method - 2 my method

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int[] arr = new int[26];
        for (int i = 0; i < s1.length(); i++)
            arr[s1.charAt(i) - 'a']++;
        int wind = s1.length();

        for (int i = 0; i <= s2.length() - s1.length(); i++){
            int[] newA = new int[26];
            for (int j = i; j < i + s1.length(); j++){
                newA[s2.charAt(j) - 'a']++;
```

```
            }
            if (check(newA, arr)) return true;
        }
        return false;
    }
    private boolean check(int[] A, int[] B){
        for (int i = 0; i < A.length; i++){
            if (A[i] != B[i]) return false;
        }
        return true; }}
```

9. Max Consecutive Ones III

Given a binary array nums and an integer k, return *the maximum number of consecutive* 1*'s in the array if you can flip at most* k 0's

Example 1:

```
Input: nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2
Output: 6
Explanation: [1,1,1,0,0,1,1,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
```

Example 2:

```
Input: nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1], k = 3
Output: 10
Explanation: [0,0,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
```

```
Method - 1 sliding window

class Solution{
    public int longestOnes(int[] A, int K) {
        int left = 0, max = 0, flips = 0;

        for (int i = 0; i < A.length; i++) {
            if (A[i] == 0) flips++;

            if (flips > K) {
                if (A[left] == 0) flips--;
                left++;
            }
```

```
            if (flips <= K) {
                max = Math.max(max, i - left + 1);
            }
        }
        return max;}}
```

Method - 2 two pointer approach

```
class Solution {
   public int longestOnes(int[] nums, int k) {
       int s = 0, end = 0, n = nums.length, ans = 0, flp1 = 0;
       while (end < n){
           if (nums[end] == 1){
               end++;
           }else {
               if (flp1 < k){
                   end++;
                   flp1++;
               }else{
                   while (flp1 >= k){
                       if (nums[s] == 0){
                           flp1--;
                       }
                       s++;
                   }
               }
           }
           ans = Math.max(ans, end - s);
       }
       return ans; }}
```

10.Minimum Window Substring

Given two strings `s` and `t` of lengths `m` and `n` respectively, return *the minimum window substring of* `s` *such that every character in* `t` *(including duplicates) is included in the window. If there is no such substring, return the empty string* `""`*.* The test cases will be generated such that the answer is unique. A substring is a contiguous sequence of characters within the string.

Examples:

```
Input: s = "ADOBECODEBANC", t = "ABC"
```

```
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from
string t.

Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

```java
class Solution {
    public String minWindow(String s, String t) {
        int l1 = s.length(), l2 = t.length(), i = 0, j = 0, count = l2, len = l1 + 1;
        String ans = "";
        HashMap<Character, Integer> map = new HashMap();
        for (int k = 0; k < l2; k++)
            map.put(t.charAt(k), map.getOrDefault(t.charAt(k), 0) + 1);
        char[] str = s.toCharArray();
        while (j < str.length){
            if (map.containsKey(str[j])){
                int freq = map.get(str[j]);
                if (freq > 0) count--;
                map.put(str[j], freq - 1);
            }
            j++;
            while (count == 0){
                if (len > j - i){
                    ans = s.substring(i, j);
                    len = j - i;
                }
                if (map.containsKey(str[i])){
                    int f = map.get(str[i]);
                    if (f == 0){  count++;  }
                    map.put(str[i], f + 1);
                }
                i++;
            }
        }
        return ans; }}
```

More Similar Sliding Window Problems

Here are some similar sliding window problems.

Also find more explanations.

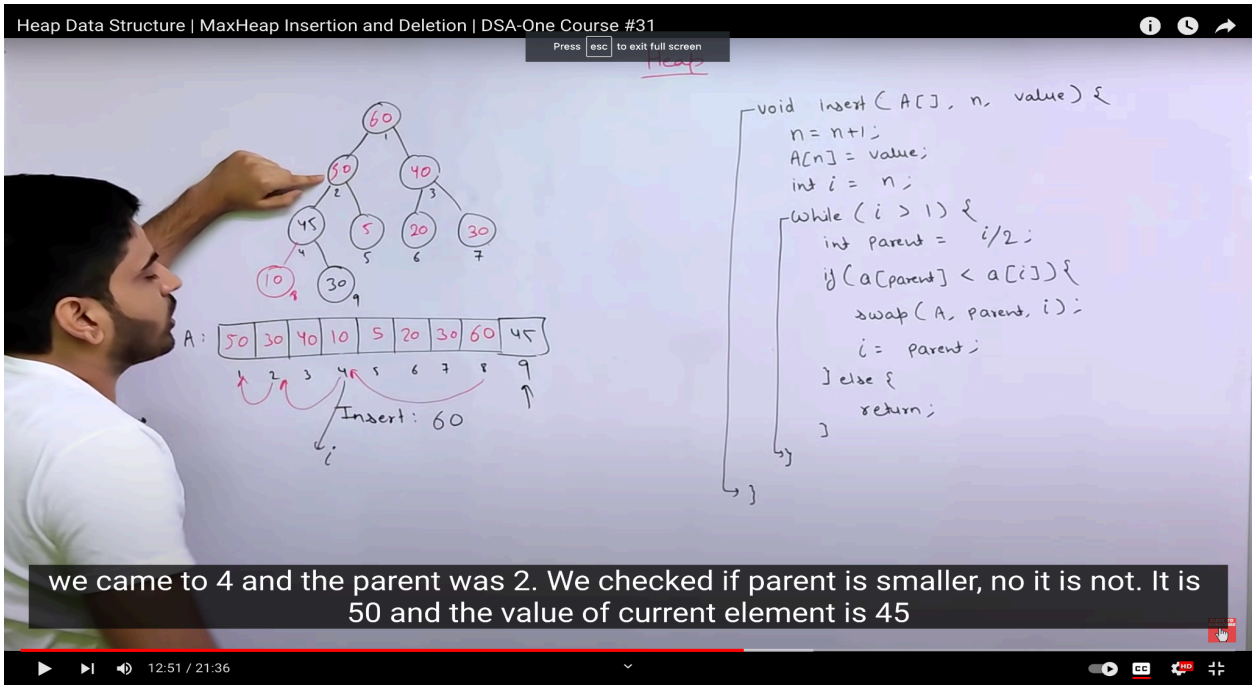
Good luck and have fun.

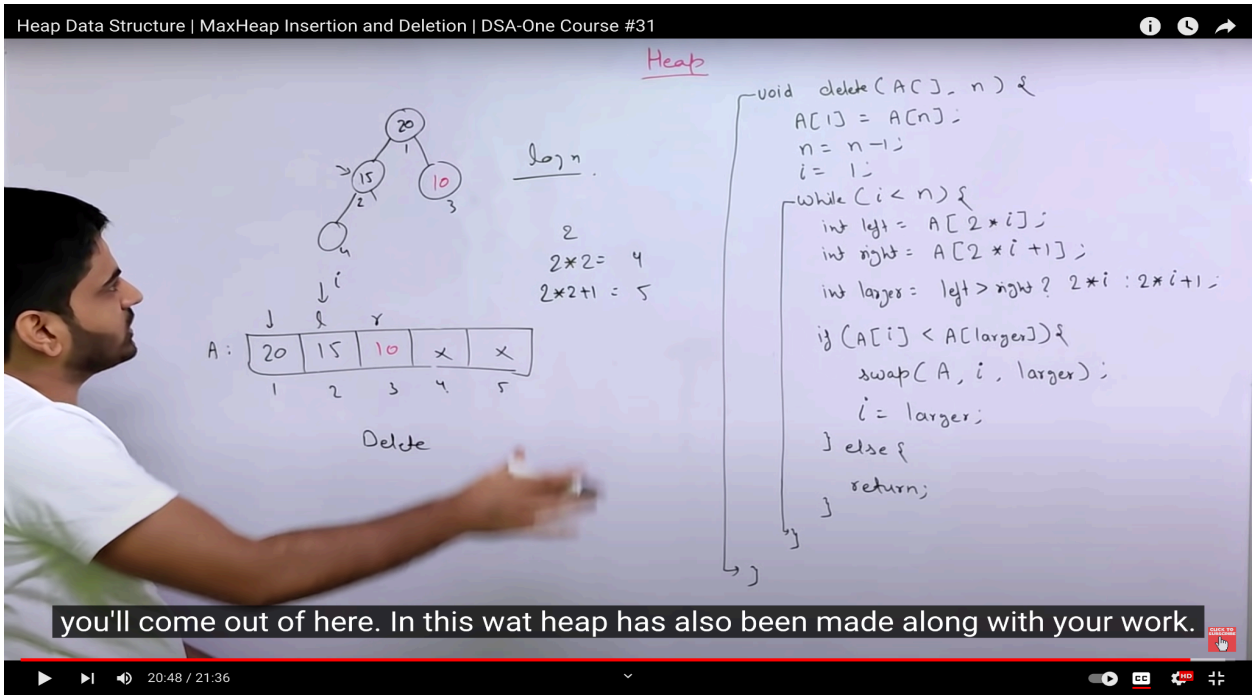
Sliding window:

862. Count Number of Nice Subarrays
863. Replace the Substring for Balanced String
864. Max Consecutive Ones III
865. Binary Subarrays With Sum
866. Subarrays with K Different Integers
867. Fruit Into Baskets
868. Shortest Subarray with Sum at Least K

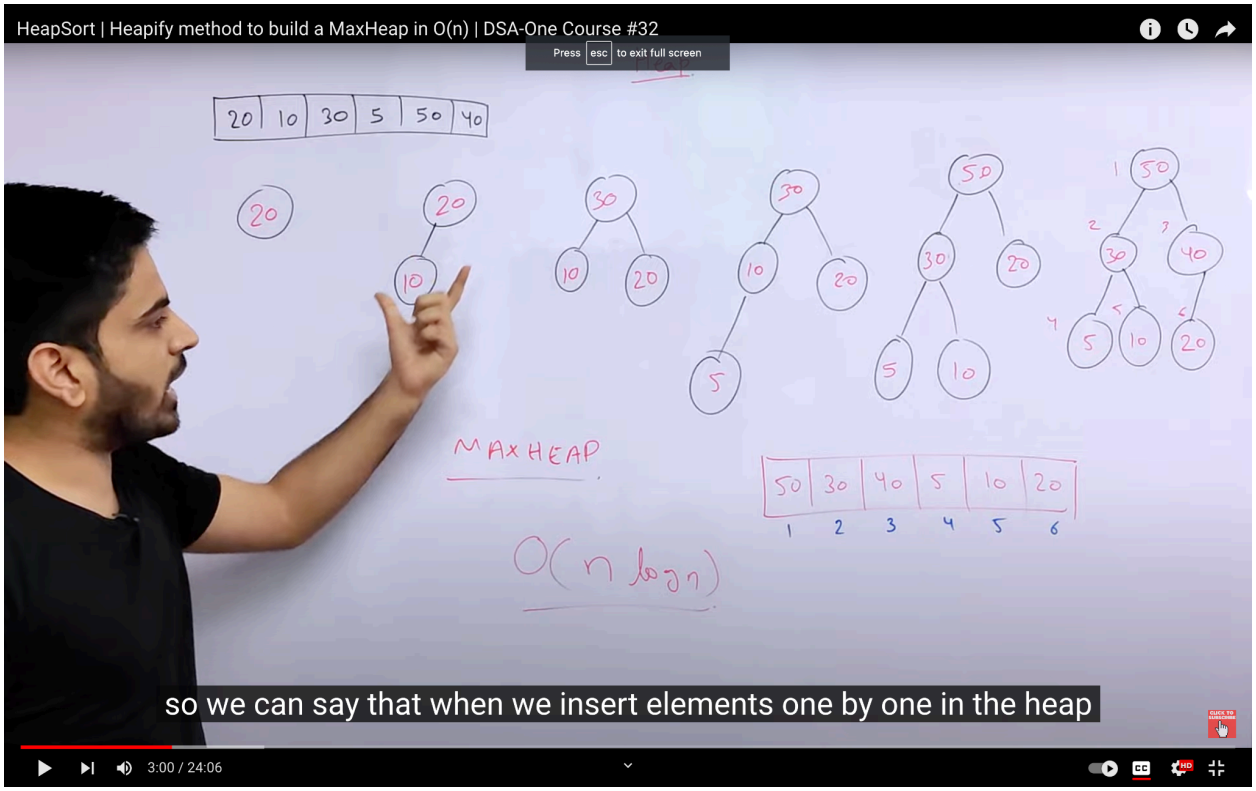
IV. STACKS, QUEUES, DeQue AND HEAPS(PRIORITY QUEUE):

Heap basics: INSERT, DELETE OPERATION: (https://www.youtube.com/watch?v=ywx-Onrdx4U)

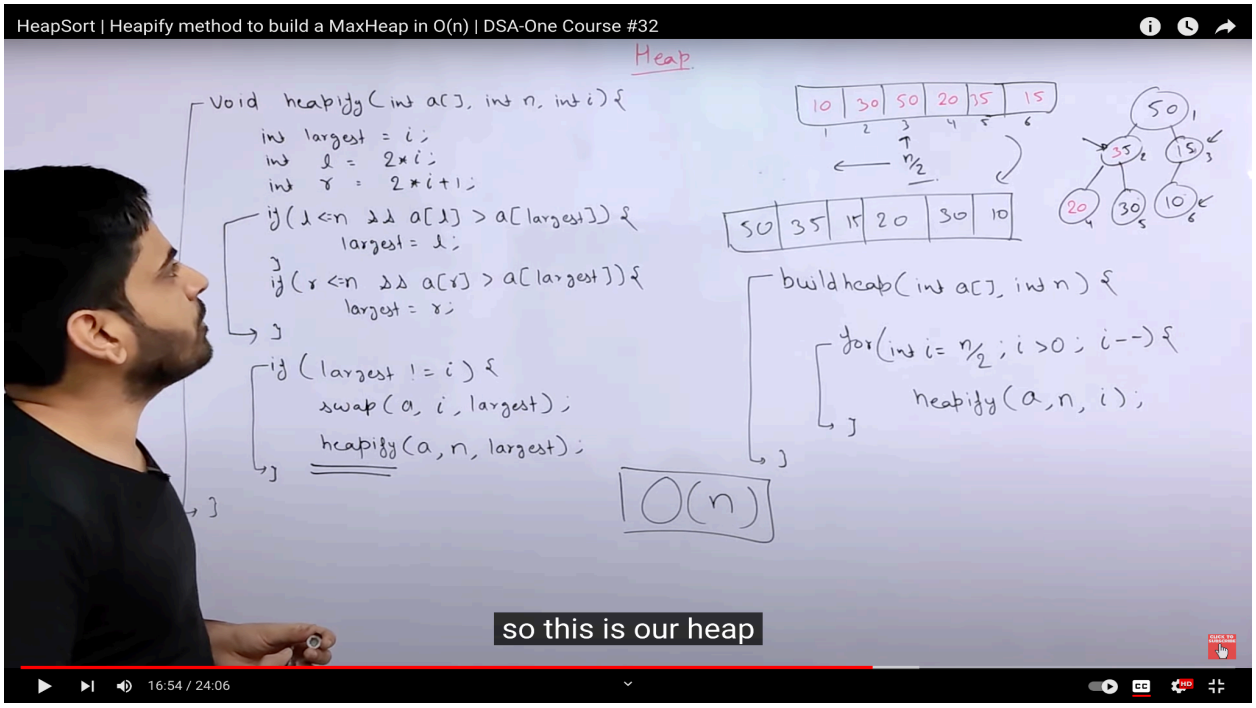




Heapify: (USING INSERT WILL TAKE N*LOG(N) TIME)



This method will take O(N) t.c. (https://www.youtube.com/watch?v=UVW0NfG_YWA)



1. Sliding window maximum

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position. Return *the max sliding window*.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3    Output: [3,3,5,5,6,7]
Explanation:
Window position                Max
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7      5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

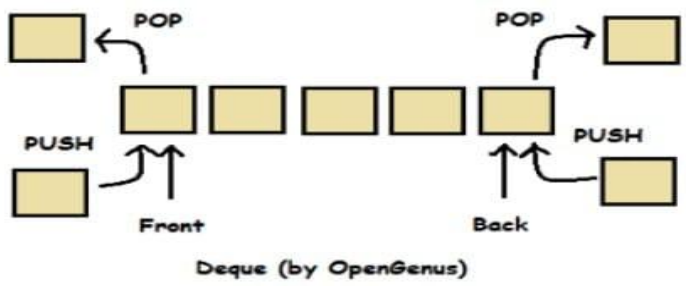
Example 2: `Input: nums = [1], k = 1  Output: [1]`

`Method - 1 giving TLE for PQ`

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (k == 1) return nums;
        int n = nums.length, prev = Integer.MIN_VALUE;
        int[] ans = new int[n - k + 1];
        PriorityQueue<Integer> pQ = new PriorityQueue(Collections.reverseOrder());
        for (int i = 0; i < n; i++){
            int st = i - k;
            if (st >= 0){
                pQ.remove(nums[st]);
            }
            pQ.add(nums[i]);
            if (pQ.size() == k) ans[i - k + 1] = pQ.peek();
        }
        return ans; }}
```

Method - 2 using Deque If we can add and remove elements from both sides of the sliding window, we can solve this problem in linear time. It turns out that we can use `Deque` to achieve the goal. In the `Deque`, we add and remove indices.

Basically, for each element `nums[i]` in the array that we are about to insert, we first check whether the leftmost index in the sliding window is out of bound. If so, we remove it by `pollFirst()` in constant time.



Then we continuously remove the rightmost indices if their corresponding elements are less than `nums[i]` (the element we are about to insert). The idea is that the elements that are less than the element we'll insert won't have any contributions to the maximum element of the sliding window. So it is safe to remove them. After removing `pollLast()` and insertion `offerLast(i)` (the element `nums[i]`), we can say that the leftmost element in the window is maximum. Think about it why. Notice that the maximum element could be the one we just insert.

Last but not least, adding the maximum elements to the result array when necessary.

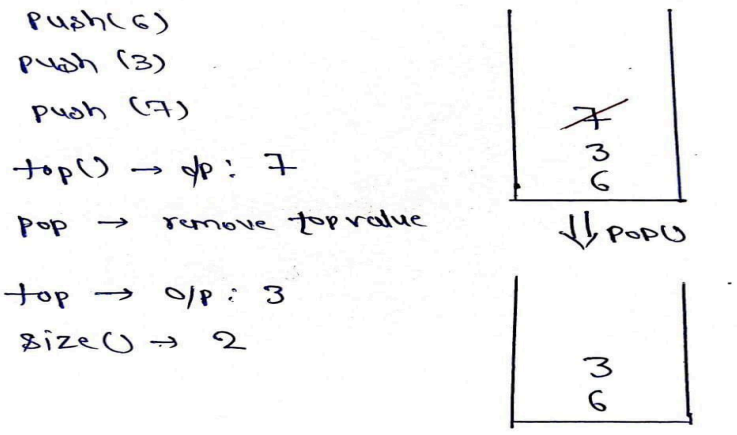
```
public int[] maxSlidingWindow(int[] nums, int k) {
  // assume nums is not null
  int n = nums.length;
  if (n == 0 || k == 0) return new int[0];
  int[] result = new int[n - k + 1]; // number of windows
  Deque<Integer> dQ = new ArrayDeque<>(); // stores indices

  for (int i = 0; i < n; ++i) {
    // remove indices that are out of bound
    while (dQ.size() > 0 && dQ.peekFirst() <= i - k) {
      dQ.pollFirst();
    }
    // remove indices whose corresponding values are less than nums[i]
    while (dQ.size() > 0 && nums[dQ.peekLast()] < nums[i]) {
      dQ.pollLast();
    }
    // add nums[i]
    dQ.offerLast(i);
    // add to result
    if (i >= k - 1) {
      result[i - k + 1] = nums[dQ.peekFirst()]; }}
  return result; }
```

Time: `O(N)` since each element is processed (add/remove) at most twice. Space: `O(k)`

2. Implement STACKS USING ARRAYS

Implement a stack using an array. Stack is a data structure that follows the **Last In First Out (LIFO)** rule.



Explanation:
push(): Insert the element in the stack.

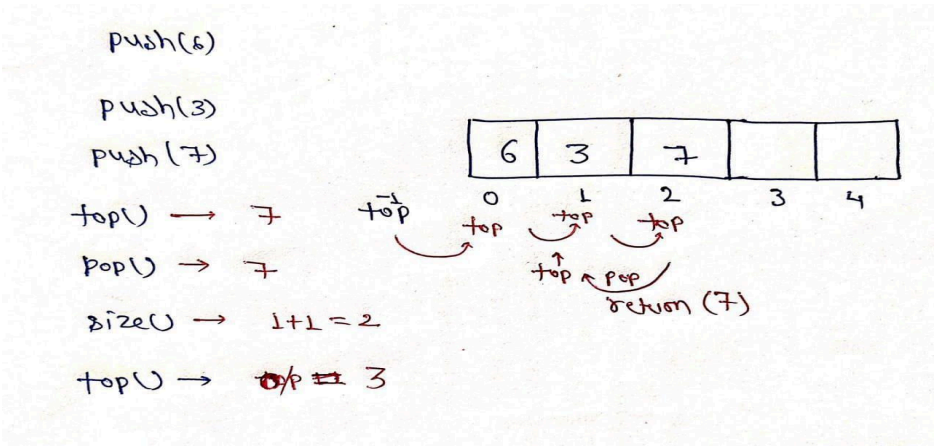
pop(): Remove and return the topmost element of the stack.

top(): Return the topmost element of the stack

size(): Return the number of remaining elements in the stack.

Solution
Intuition: As we know stack works on the principle of last in first out, so we have to put elements in an array such that it keeps track of the most recently inserted element. Hence we can think of using a Top variable which will help in keeping track of recent elements inserted in the array.
Approach:

- Declare an array of particular size.
- Define a variable “Top” and initialize it as -1.
- push(x): insert element is the array by increasing top by one.
- pop(): check whether top is not equal to -1 if it is so, return top and decrease its value by one.
- size(): return top+1.




```
class stack {
    int size = 10000;
    int arr[] = new int[size];
    int top = -1;
    void push(int x) {
        top++;
        arr[top] = x;
    }
    int pop() {
        int x = arr[top];
        top--;
        return x;
    }
    int top() {
        return arr[top];
    }
    int size() {
        return top + 1;
    }
}
```

3. Implement Queues Using Arrays

Input: push(4) push(14) push(24) push(34) top() size() pop()

Output:

The element pushed is 4 The element pushed is 14
The element pushed is 24 The element pushed is 34
The peek of the queue before deleting any element 4
The size of the queue before deletion 4 The first element to be deleted 4
The peek of the queue after deleting an element 14
The size of the queue after deleting an element 3

Intuition: The intuition is to fill the array in a circular manner, (ie) after popping from the front, rather than moving all the elements towards the front. We can have 2 variables to keep track of the start and end indexes of the sequence. Mod addition is done to handle boundary conditions.

Approach: The basic approach is to maintain two variables to point to the START and END of the filled elements in the array. START pointer is used to point to the starting index of the elements and the same case for the END pointer(ending index). Initially, both have value -1(indicating empty queue).
First, let's see the implementation of the push function. Push basically inserts a new element at the end. So only the END variable is going to be incremented.
Corner case 1: What if we have no empty places in the array? So, first check that, if we don't have we exit, in the other case we increment the START variable and put the new element.

Corner case 2: What if END reaches the last index? We are doing mod with addition. So, END goes back to index 0([0-(n-1)] will always be the range for END).

Second, let us see the pop function. In Queue pop removes and returns the front element. So, START needs to be modified. The general approach is to copy the current element pointed by START and increase the START variable to the next index.
Corner case 3: What if the Queue is empty? That's why we are checking the START variable. If it is -1, then the queue is empty, we just exit.

Corner case 4: What if START goes out of bound? As done for END, mod addition comes to the rescue.

Corner case 5: What happens after popping the last element? We check this state with the currSize variable. Queue returns to the initial state, both START and END are set to -1.
Third, let's see the top function. It behaves more like a pop function. We need to return the element pointed by the START variable. Since we are not actually removing any element, it's fine to ignore corner cases 4 and 5.
That's all about the Queue class implementation. In the main function, we just initialize the Queue class to check all corner cases.

```
class Queue {
    private int arr[];
    private int start, end, currSize, maxSize;
    public Queue() {
        arr = new int[16];
        start = -1;
        end = -1;
        currSize = 0;
    }
    public Queue(int maxSize) {
        this.maxSize = maxSize;
        arr = new int[maxSize];
        start = -1;
        end = -1;
        currSize = 0;
    }
    public void push(int newElement) {
        if (currSize == maxSize) {
            System.out.println("Queue is full\nExiting...");
            System.exit(1);
        }
        if (end == -1) {
```

```
            start = 0;
            end = 0;
        } else
            end = (end + 1) % maxSize;
        arr[end] = newElement;
        System.out.println("The element pushed is " + newElement);
        currSize++;
    }
    public int pop() {
        if (start == -1) {
            System.out.println("Queue Empty\nExiting...");
            System.exit(1);
        }
        int popped = arr[start];
        if (currSize == 1) {
            start = -1;
            end = -1;
        } else
            start = (start + 1) % maxSize;
        currSize--;
        return popped;
    }
    public int top() {
        if (start == -1) {
            System.out.println("Queue is Empty");
            System.exit(1);
        }
        return arr[start];
    }
    public int size() {
        return currSize; }}
```

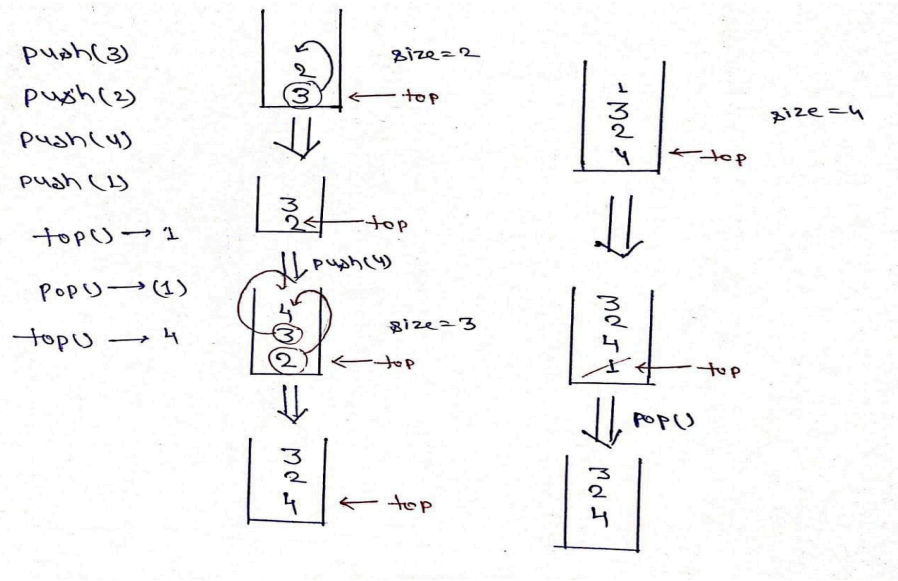
4. Implement Stack using Queue

Intuition: As we know stack follows last in first out, which means we get the most recently inserted element whenever we remove an element from the stack. But queue follows first in first out, it means we get that element which we inserted in the starting at each deletion, it means if we want to use the queue like a stack we have to arrange elements in the queue such that we get the most recent element at each deletion.

Approach:

- Take a single queue.
- push(x): Push the element in the queue.
- Use a for loop of size()-1, remove element from queue and again push back to the queue, hence the most recent element becomes the most former element and vice versa.
- pop(): remove the element from the queue.
- top(): show the element at the top of the queue.
- size(): size of the current queue.

Repeat step3 at every insertion of the element.

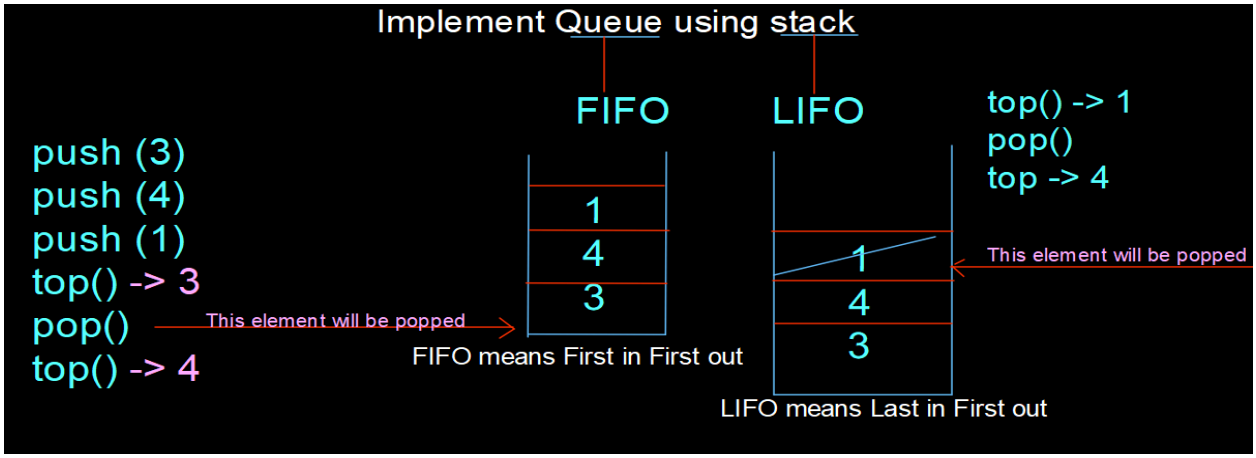



```
class stack {
    Queue < Integer > q = new LinkedList < > ();
    void push(int x) {
        q.add(x);
        for (int i = 0; i < q.size() - 1; i++) {
            q.add(q.remove());
        }
    }
    int pop() {
        return q.remove();
    }
    int top() {
        return q.peek();
    }
    int size() {
        return q.size();
    }
}
```

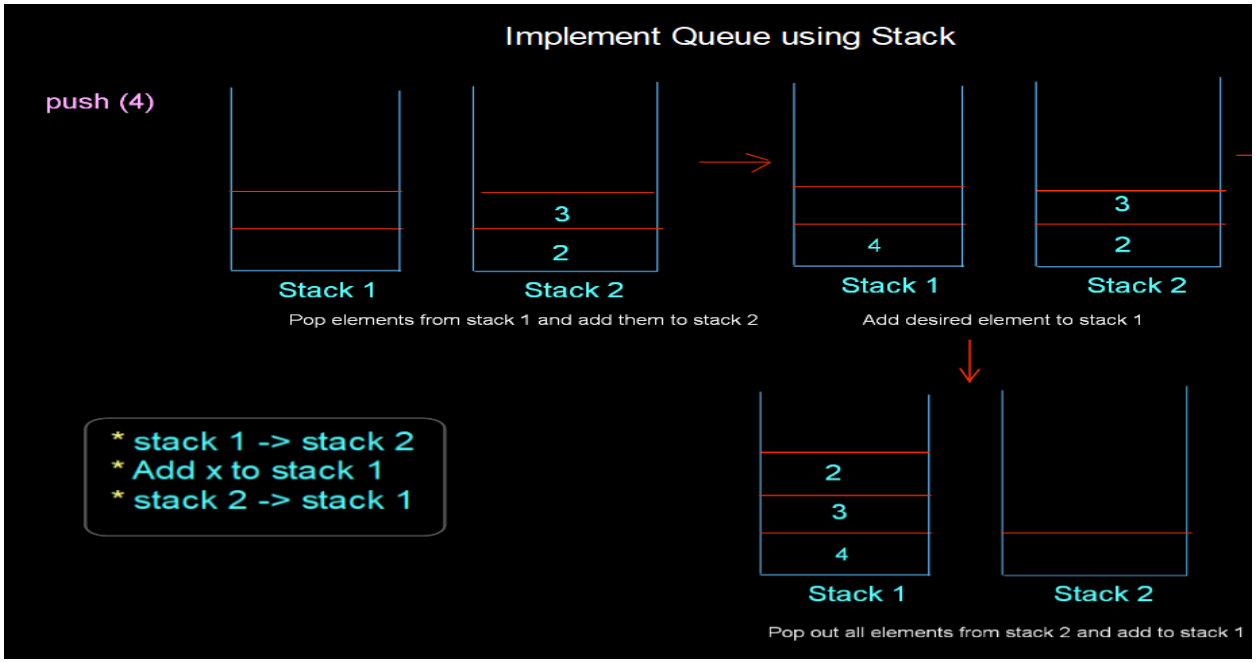
5. Implement Queue using Stack

Given a Stack having some elements stored in it. Can you implement a Queue using the given Stack?

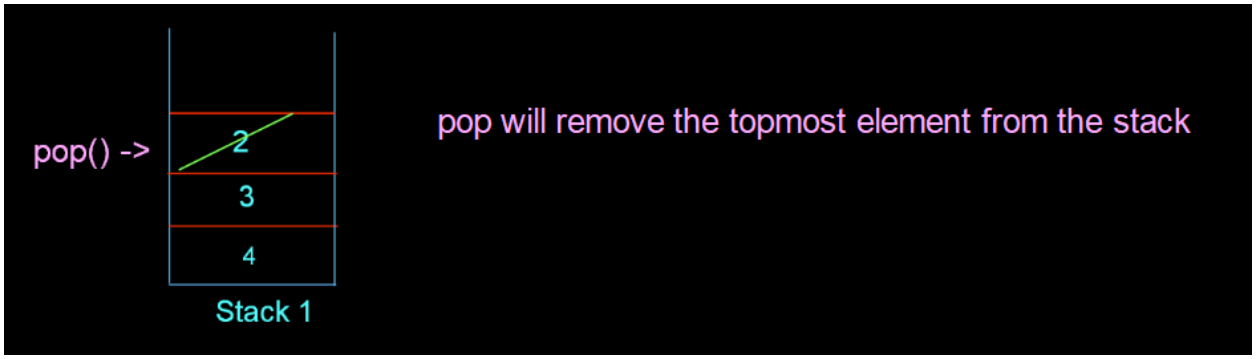
Approach:



push(x) ->



pop()->



top()->



size()->

size() operation is for returning the size of a queue which can be done by using the function Stack1. size(). It will actually return the total number of elements in the queue.

```
class MyQueue {
    Stack < Integer > input = new Stack < > ();
    Stack < Integer > output = new Stack < > ();
    /** Initialize your data structure here. */
    public MyQueue() {}
    /** Push element x to the back of queue. */
    public void push(int x) {
        while (input.empty() == false) {
            output.push(input.peek());
            input.pop();
        }
        // Insert the desired element in the stack input
        System.out.println("The element pushed is " + x);
        input.push(x);
        // Pop out elements from the stack output and push them into the stack input
        while (output.empty() == false) {
            input.push(output.peek());
            output.pop();
        } }

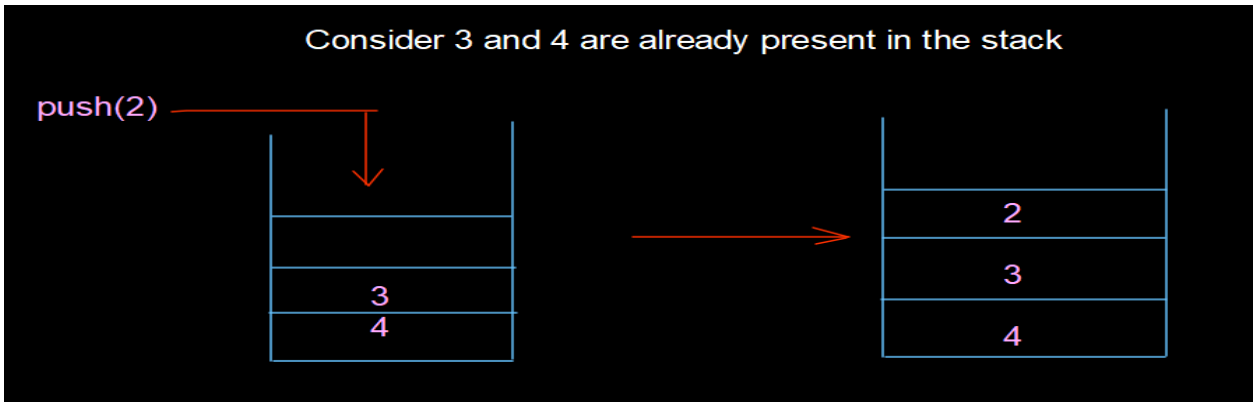
    /** Removes the element from in front of queue and returns that element. */
    public int pop() {
        // shift input to output
        if (input.empty()) {
            System.out.println("Stack is empty");
        }
        int val = input.peek();
        input.pop();
        return val;
    }
    /** Get the front element. */
    public int peek() {
        // shift input to output
```

```
        if (input.empty()) {
            System.out.println("Stack is empty");
        }
        return input.peek();
    }
    int size() {
        return input.size();
    }}
```

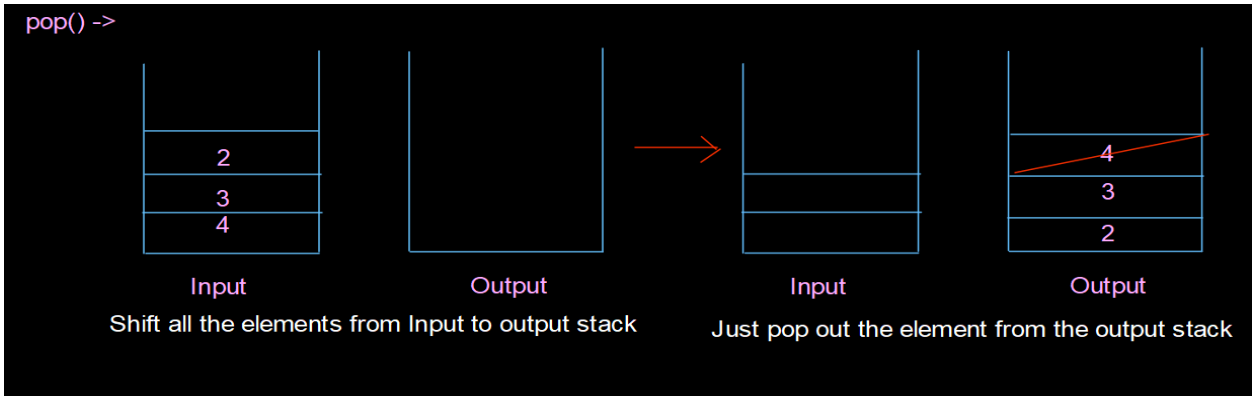
Solution 2: Using two Stacks where push operation is O(1)

Approach :

Push()->



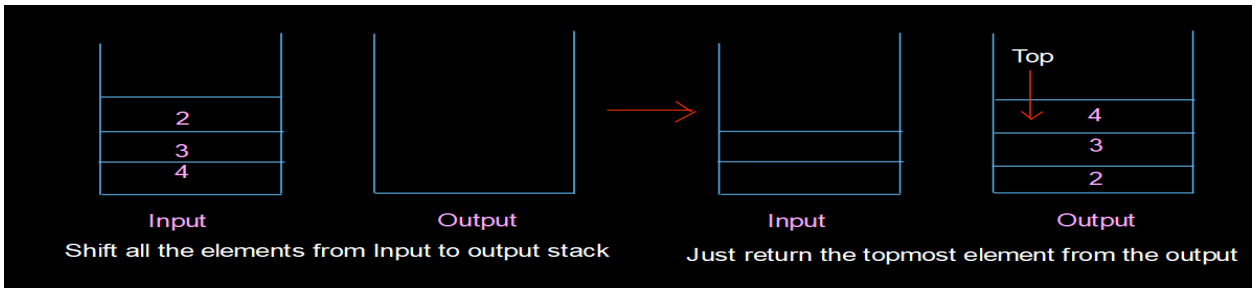
Pop()->



pop() -> pop can be O(1) or O(n) .

O(1) when all elements are already shifted . So we just need to delete the element . So here we can say that time complexity can be O(1) or amortised O(1)

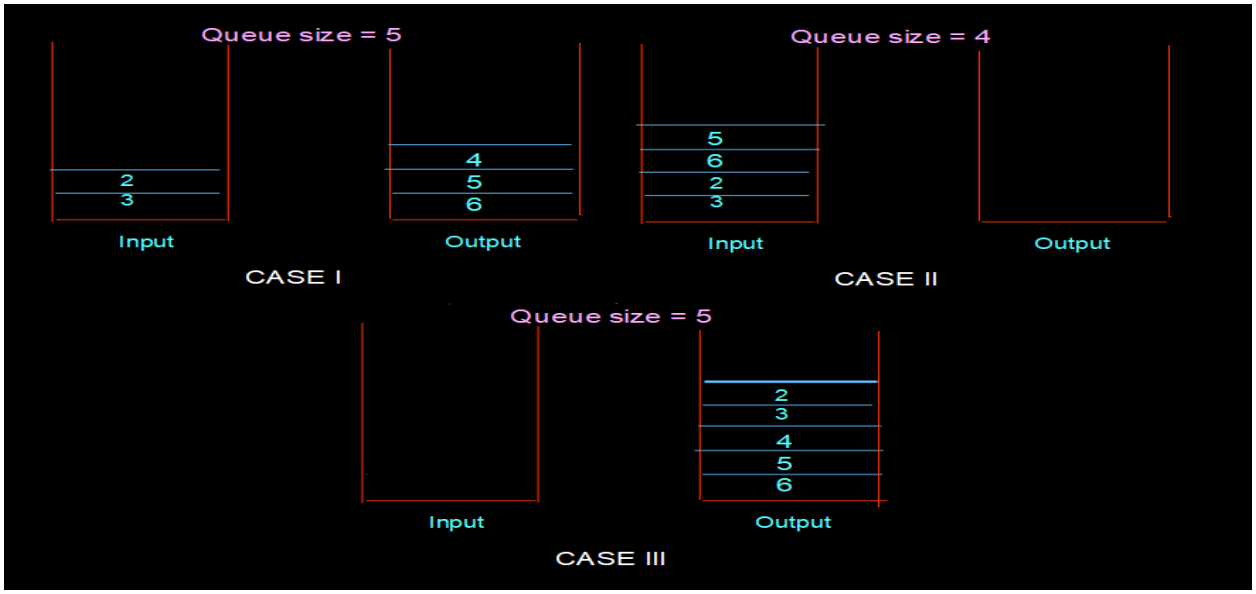
top()->



Top() -> Top can be O(1) or O(n) .

O(1) when all elements are already shifted . So we just need to return the element . So here we can say that time complexity can be O(1) or amortised O(1)

Size():




```
class MyQueue {
    Stack < Integer > input = new Stack < > ();
    Stack < Integer > output = new Stack < > ();
    /** Initialize your data structure here. */
    public MyQueue() {}
    /** Push element x to the back of queue. */
    public void push(int x) {
        System.out.println("The element pushed is " + x);
        input.push(x); }
    /** Removes the element from in front of queue and returns that element. */
    public int pop() {
        // shift input to output
        if (output.empty())
            while (input.empty() == false) {
                output.push(input.peek());
                input.pop(); }
        int x = output.peek();
        output.pop();
        return x;
    }
    /** Get the front element. */
    public int peek() {
        // shift input to output
        if (output.empty())
            while (input.empty() == false) {
                output.push(input.peek());
                input.pop();
            }
        return output.peek();
    }
    int size() {
        return (output.size() + input.size());
    }}
```

6. Check for balanced parentheses

Check Balanced Parentheses. Given string str containing just the characters '(', ')', '{', '}', '[' and ']', check if the input string is valid and return true if the string is balanced otherwise return false.

Note: string str is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.

Example 1:

Input: str = "( )[{ } ( )]" Output: True

Explanation: As every open bracket has its corresponding close bracket. Match parentheses are in correct order hence they are balanced.

Example 2:

Input: str = "[( )" Output: False

Explanation: As '[' does not have ']' hence it is not valid and will return false.

Solution

Intuition: We have to keep track of previous as well as most recent opening brackets and also keep in mind the sequence, as after opening of the bracket there should be opposite pairs of brackets. Also handle the corner cases like [ ) ( ] where closing bracket occurs first and opening bracket after it, which is an invalid sequence, as well as [( ]) where the most recent opening didn't get its opposite pair hence it will also not be valid.

So we have to use a data structure that will keep track of first in and last out, hence we will use the **stack**.

Approach:

1. Whenever we get the opening bracket we will push it into the stack. I.e '{', '[', '('.
2. Whenever we get the closing bracket we will check if the stack is non-empty or not.
3. If the stack is empty we will return false, else if it is nonempty then we will check if the topmost element of the stack is the opposite pair of the closing bracket or not.
4. If it is not the opposite pair of the closing bracket then return false, else move ahead.
5. After we move out of the string the stack has to be empty if it is non-empty then return it as invalid else it is a valid string.

```
public static boolean isValid(String s) {
        Stack<Character> st = new Stack<Character>();
        for (char it : s.toCharArray()) {
            if (it == '(' || it == '[' || it == '{')
                st.push(it);
            else {
                if(st.isEmpty()) return false;
```

```
                char ch = st.pop();
                if((it == ')' && ch == '(') ||  (it == ']' && ch == '[') ||
(it == '}' && ch == '{')) continue;
                else return false;
            }
        }
        return st.isEmpty();}}
```

7. Next greater element

Given a circular integer array **A**, return the next greater element for every element in A. The next greater element for an element x is the first element greater than x that we come across while traversing the array in a clockwise manner. If it doesn't exist, return -1 for this element.

Example 1:

Input: N = 11, A[] = {3,10,4,2,1,2,6,1,7,2,9} **Output:** 10,-1,6,6,2,6,7,7,9,9,10

Explanation: For the first element in A ,i.e, 3, the greater element which comes next to it while traversing and is closest to it is 10. Hence,10 is present on index 0 in the resultant array. Now for the second element,i.e, 10, there is no greater number and hence -1 is it's next greater element (NGE). Similarly, we got the NGEs for all other elements present in A.

Example 2:

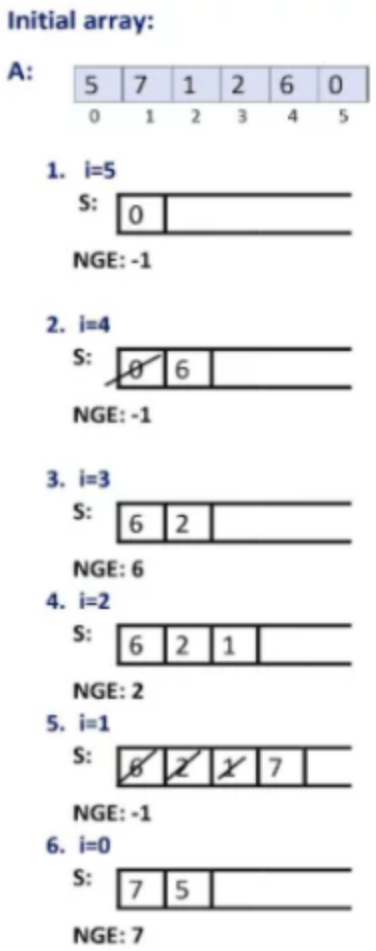
Input: N = 6, A[] = {5,7,1,7,6,0} **Output:** 7,-1,7,-1,7,5

Solution

Approach: This problem can be solved easily and efficiently by using the stack data structure as it is based on the First in First out (FIFO) principle.

To make it a bit easier let's first try to solve without considering the array as circular. To find the next greater element we start traversing the given array from the right. As for the rightmost element, there is no other element at its right. Hence, we assign -1 at its index in the resultant array. Since this can be the next greater element (NGE) for some other element, we push it in the stack S. We keep checking for other elements. Let's say we are checking for an element at index i. We keep popping from the stack until the element at the top of the stack is smaller than A[i]. The main intuition behind popping them is that these elements can never be the NGE for any element present at the left of A[i] because A[i] is greater than these elements. Now, if the top element of S is greater than A[i] then this is NGE of A[i] and we will assign it to res[i], where res is the resultant array. If the stack becomes empty then it implies that no element at the right of A[i] is greater than it and we assign -1. At last, we push A[i] in S.

Dry run: Let's apply this algorithm for A[] = {5,7,1,2,6,0}:



So, the resultant array is {7,-1,2,6,-1,-1}. Remember that we have considered the array to be non-circular. For a circular array, the resultant array should be {7,-1,2,6,7,5}.

Now we need to make this algorithm work for a circular array. The only difference between a circular and non-circular array is that while searching for the next greater element in a non-circular array we don't consider the elements left to the concerned element. This can be easily done by inserting the elements of the array A at the end of A, thus making its size double. But we actually don't require any extra space. We can just traverse the array twice. We actually run a loop 2*N times, where N is the size of the given array.

```
class TUF {
    public static int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int nge[] = new int[n];
        Stack < Integer > st = new Stack < > ();
        for (int i = 2 * n - 1; i >= 0; i--) {
            while (st.isEmpty() == false && st.peek() <= nums[i % n]) {
                st.pop();
            }

            if (i < n) {
                if (st.isEmpty() == false) nge[i] = st.peek();
                else nge[i] = -1;
            }

            st.push(nums[i % n]);
        }
        return nge;
    }}
```

8. Next smaller element

Given the array `prices` where `prices[i]` is the price of the `ith` item in a shop. There is a special discount for items in the shop, if you buy the `ith` item, then you will receive a discount equivalent to `prices[j]` where `j` is the minimum index such that `j > i` and `prices[j] <= prices[i]`, otherwise, you will not receive any discount at all. *Return an array where the `ith` element is the final price you will pay for the `ith` item of the shop considering the special discount.*

Example 1:

```
Input: prices = [8,4,6,2,3]
Output: [4,2,4,2,3]
Explanation:
For item 0 with price[0]=8 you will receive a discount equivalent to prices[1]=4,
therefore, the final price you will pay is 8 - 4 = 4.
For item 1 with price[1]=4 you will receive a discount equivalent to prices[3]=2,
therefore, the final price you will pay is 4 - 2 = 2.
For item 2 with price[2]=6 you will receive a discount equivalent to prices[3]=2,
therefore, the final price you will pay is 6 - 2 = 4.
For items 3 and 4 you will not receive any discount at all.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: [1,2,3,4,5]
Explanation: In this case, for all items, you will not receive any discount at all.
```

```
class Solution {
public int[] finalPrices(int[] prices) {
   Stack<Integer> st = new Stack<>();
    int len = prices.length;
    int [] res = new int[len];
      for(int i=len-1; i>=0; i--){
       if(st.isEmpty()){
           res[i] = prices[i];
       } else if(!st.isEmpty() && prices[i] >= st.peek())
           res[i] = prices[i] - st.peek();
       else if(!st.isEmpty() && prices[i] < st.peek()){
           while(!st.isEmpty() && prices[i] < st.peek())
               st.pop();
           if(st.isEmpty())
               res[i] = prices[i];
           else
               res[i] = prices[i] - st.peek();
       }
       st.push(prices[i]);
    }
    return res; }}
```

9. Sort a stack

You can do that using recursion. Here is the code and if you want to sort in descending order all you have to do is change ">" rather than "<" inside insideAtCorrectPosition method.

```
class StackSortRecursion {

public static Stack<Integer> stack = new Stack<>();

public static void main(String[] args) {
	stack.push(9);
	stack.push(2);
	stack.push(7);
	stack.push(4);
	stack.push(6);
	sort();
	System.out.println(stack);
}

private static void sort() {
	if (stack.isEmpty()) {
		return;
	}
	int temp = stack.pop();
	sort();
	insertAtCorrectPosition(temp);
}

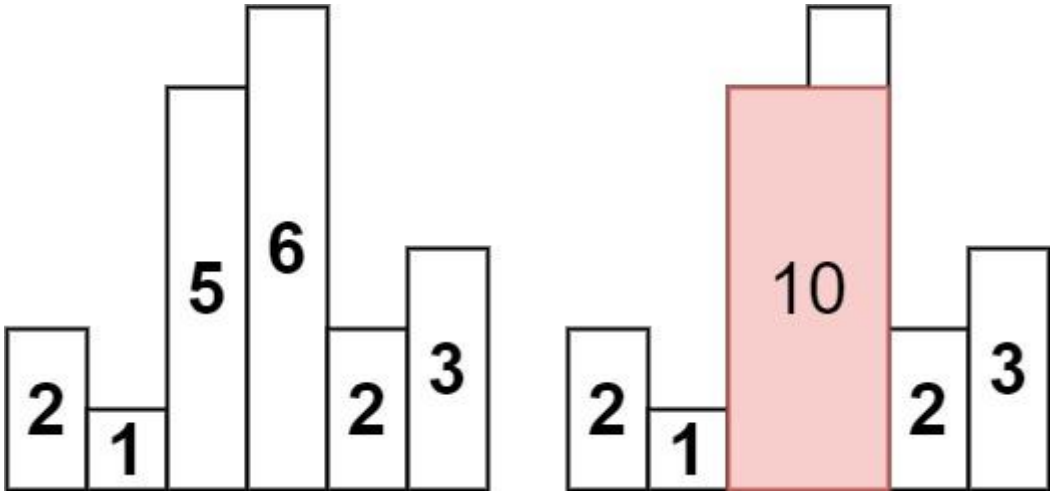
private static void insertAtCorrectPosition(int temp) {
	if (stack.isEmpty() || stack.peek() < temp) {
		stack.push(temp);
		return;
	}
	int top = stack.pop();
	insertAtCorrectPosition(temp);
	stack.push(top);
}

public void sortStack(Stack<Integer> stack) {
    Stack<Integer> helpStack = new Stack<>();
    while (!stack.isEmpty()) {
        int temp = stack.pop();
        while (!helpStack.isEmpty() && helpStack.peek() > temp) {
            stack.push(helpStack.pop());
        }
        helpStack.push(temp);
    }
    while (!helpStack.isEmpty()) stack.push(helpStack.pop());
}
```

10. Largest rectangle histogram

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.

Example 1:

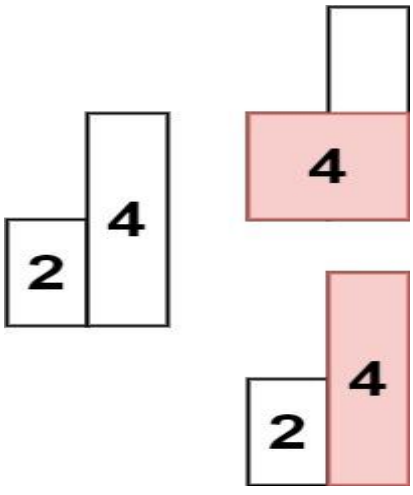


Input: heights = [2,1,5,6,2,3] Output: 10

Explanation: The above is a histogram where width of each bar is 1.

The largest rectangle is shown in the red area, which has an area = 10 units.

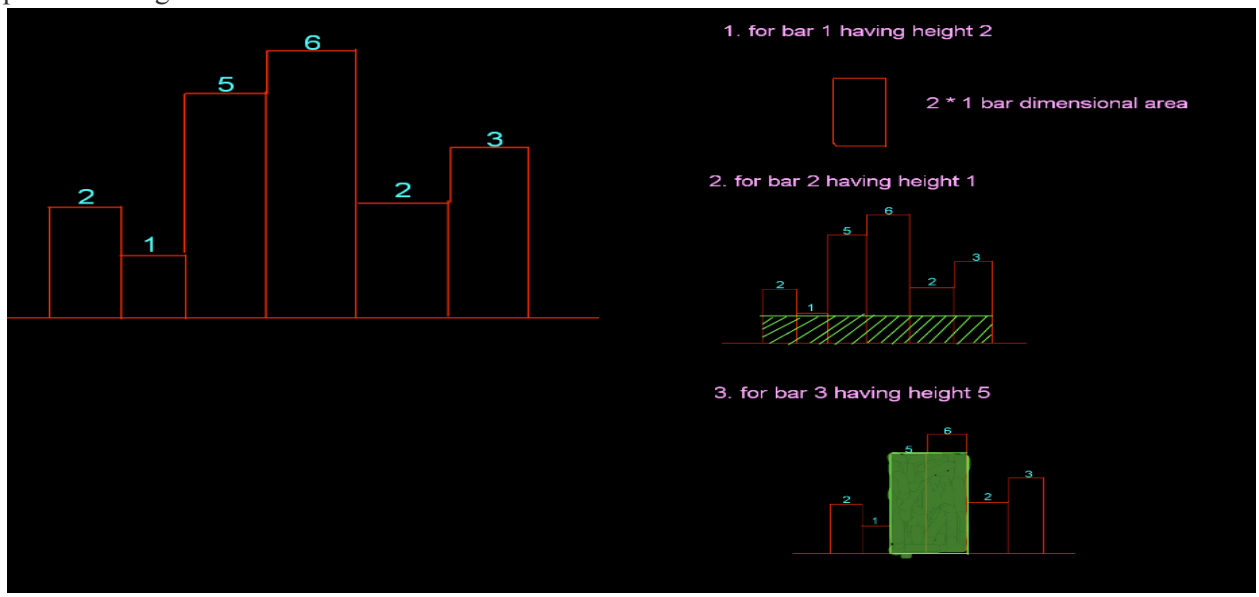
Example 2:



Input: heights = [2,4] Output: 4

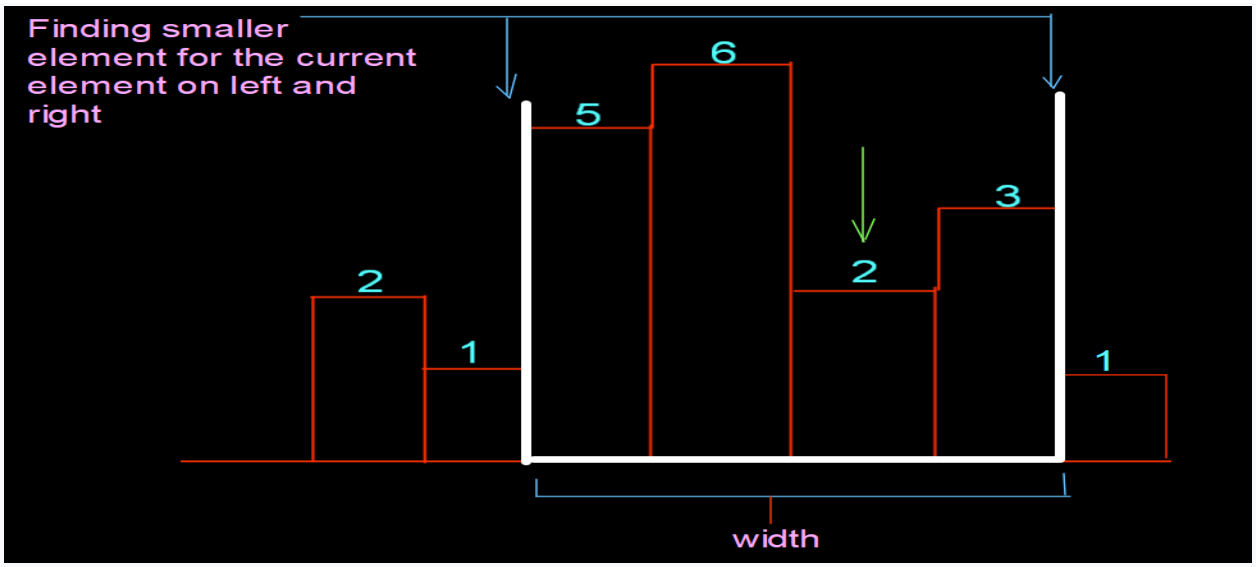
Solution 1: Brute Force Approach

Intuition: The intuition behind the approach is taking different bars and finding the maximum width possible using the bar.



Similarly for other bars, we will find the areas possible:-
Considering the width of each bar as 1 unit.
For first bar, area possible = 2* 1 =2 sq . units
For second bar, area possible = 1 * 6 =6 sq . units
For third bar , area possible = 5 *2 = 10 sq . units
For fourth bar , area possible = 6 * 1 = 6 sq . units
For Fifth bar , area possible = 2 * 4 = 8 sq . units
For Sixth bar , area possible = 3 * 1 =3 sq . units
So, the maximum area possible = 10 sq units.

Approach: The approach is to find the right smaller and left smaller element and find the largest Rectangle area in Histogram.



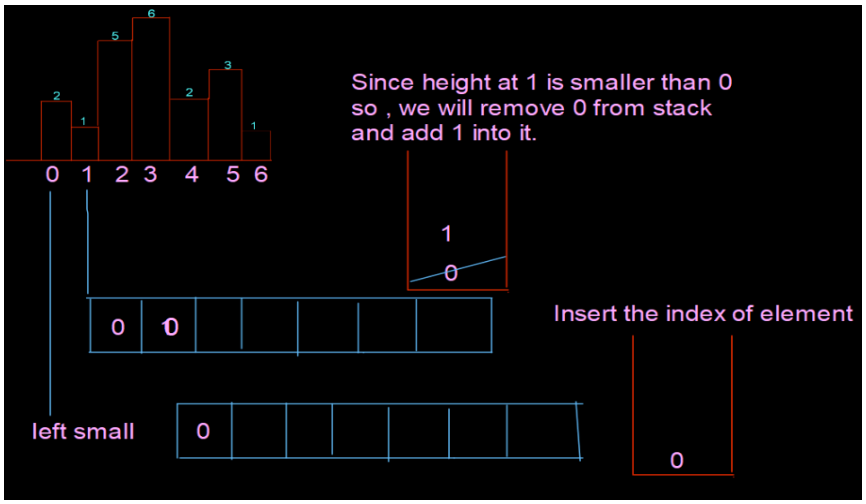
```java
static int largestarea(int arr[], int n) {
        int maxArea = 0;
        for (int i = 0; i < n; i++) {
            int minHeight = Integer.MAX_VALUE;
            for (int j = i; j < n; j++) {
                minHeight = Math.min(minHeight, arr[j]);
                maxArea = Math.max(maxArea, minHeight * (j - i + 1));
            }
        }
        return maxArea;}
```

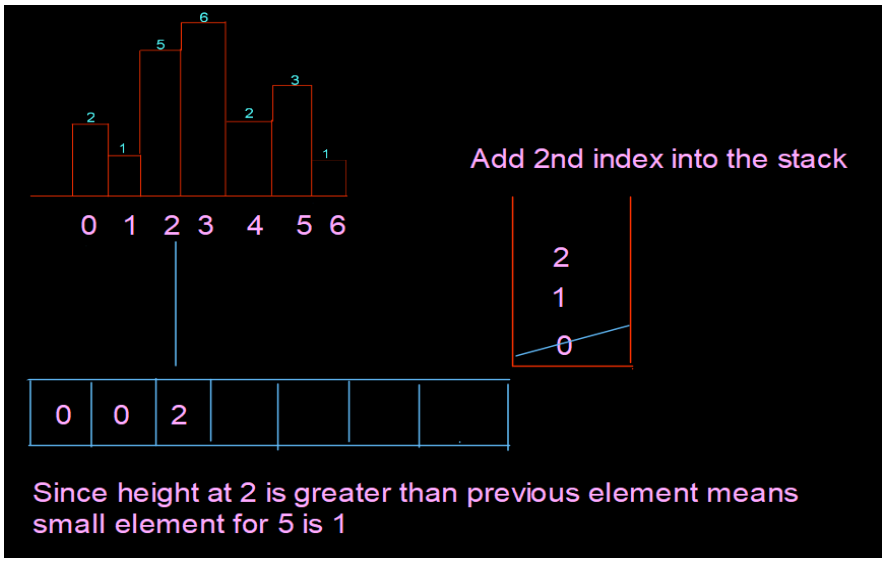
Solution 2: Optimised Approach 1

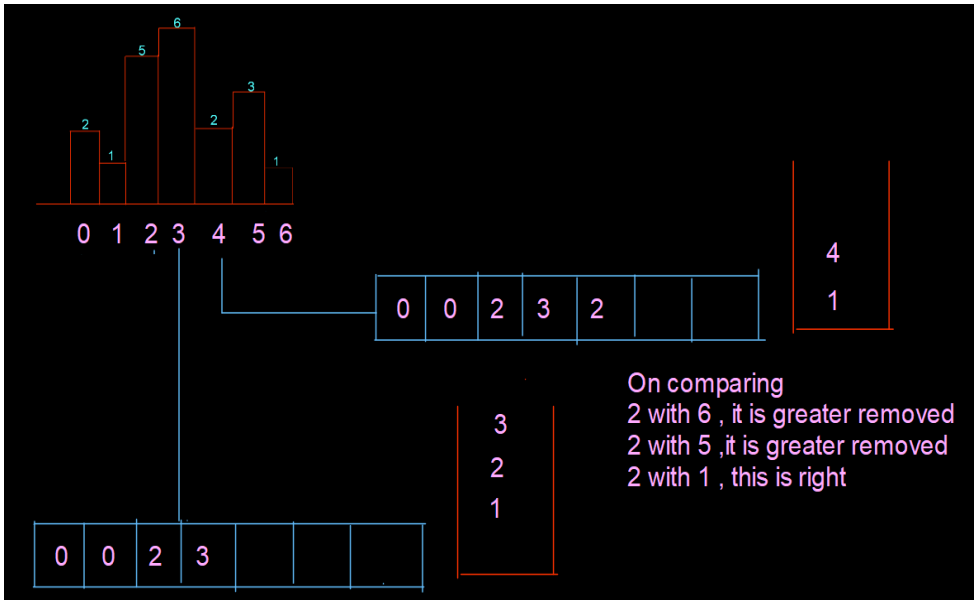
Intuition: The intuition behind the approach is the same as finding the smaller element on both sides but in an optimized way using the concept of the next greater element and the next smaller element.

Approach:

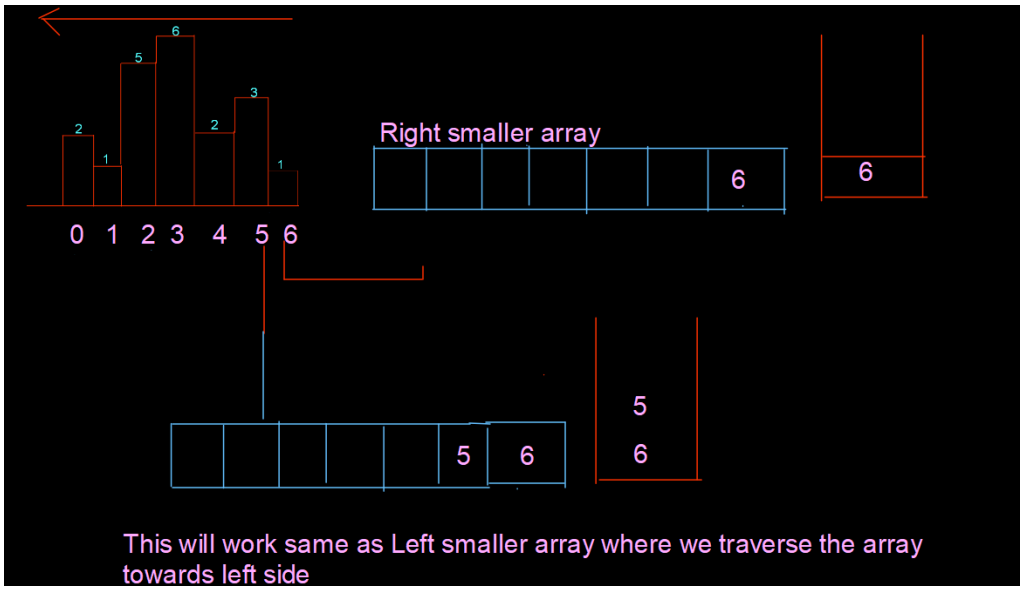
1. **Steps to be done for finding Left smaller element**



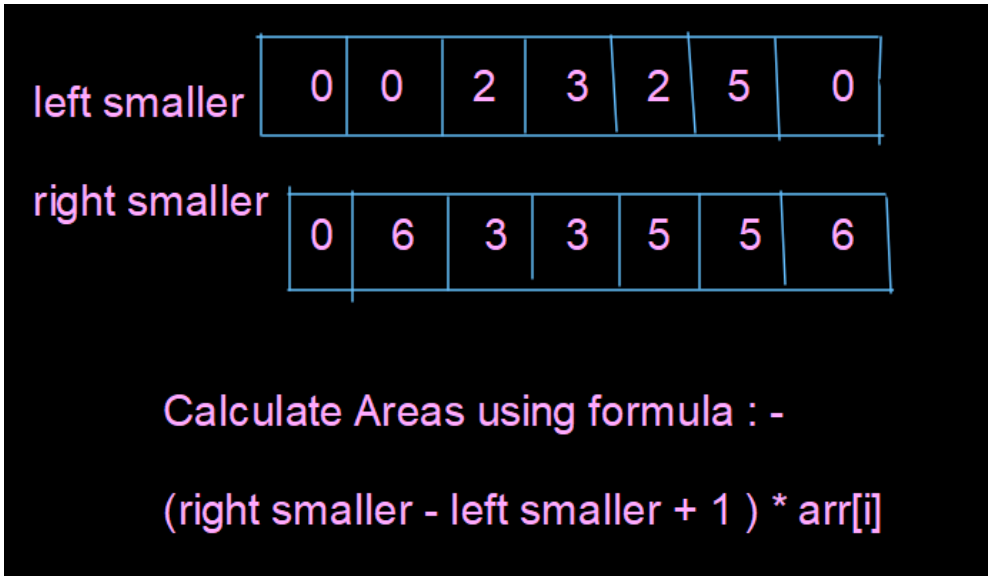




2. **Steps to be done for finding the Right smaller element**



After finding the right smaller and left smaller of each subsequent array elements, we



Area for first index – (0 – 0 +1) * 2 = 2
Area for second index – (6 – 0 + 1) * 1 = 6
Area for third index – (3 – 2 +1) * 5 = 10
Area for fourth index – (3 – 3 + 1) * 6 = 6
Area for fifth index – (5 – 2 +1) * 2 = 8
Area for sixth index – (5 – 5 + 1) * 3 = 3
Area for seventh index – (6 – 0 +1) * 1 = 7
So, the maximum area out of these is 10 sq units.

```
public static int largestRectangleArea(int[] heights) {
        int n = heights.length;
        Stack < Integer > st = new Stack < > ();
        int[] leftSmall = new int[n], rightSmall = new int[n];
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && heights[st.peek()] >= heights[i]) {
                st.pop();
            }
            if (st.isEmpty()) leftSmall[i] = 0;
            else leftSmall[i] = st.peek() + 1;
            st.push(i);
        }
        // clear the stack to be re-used
        while (!st.isEmpty()) st.pop();
        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && heights[st.peek()] >= heights[i]) {
                st.pop();
            }
            if (st.isEmpty()) rightSmall[i] = n - 1;
            else rightSmall[i] = st.peek() - 1;
            st.push(i);
        }

        int maxA = 0;
```

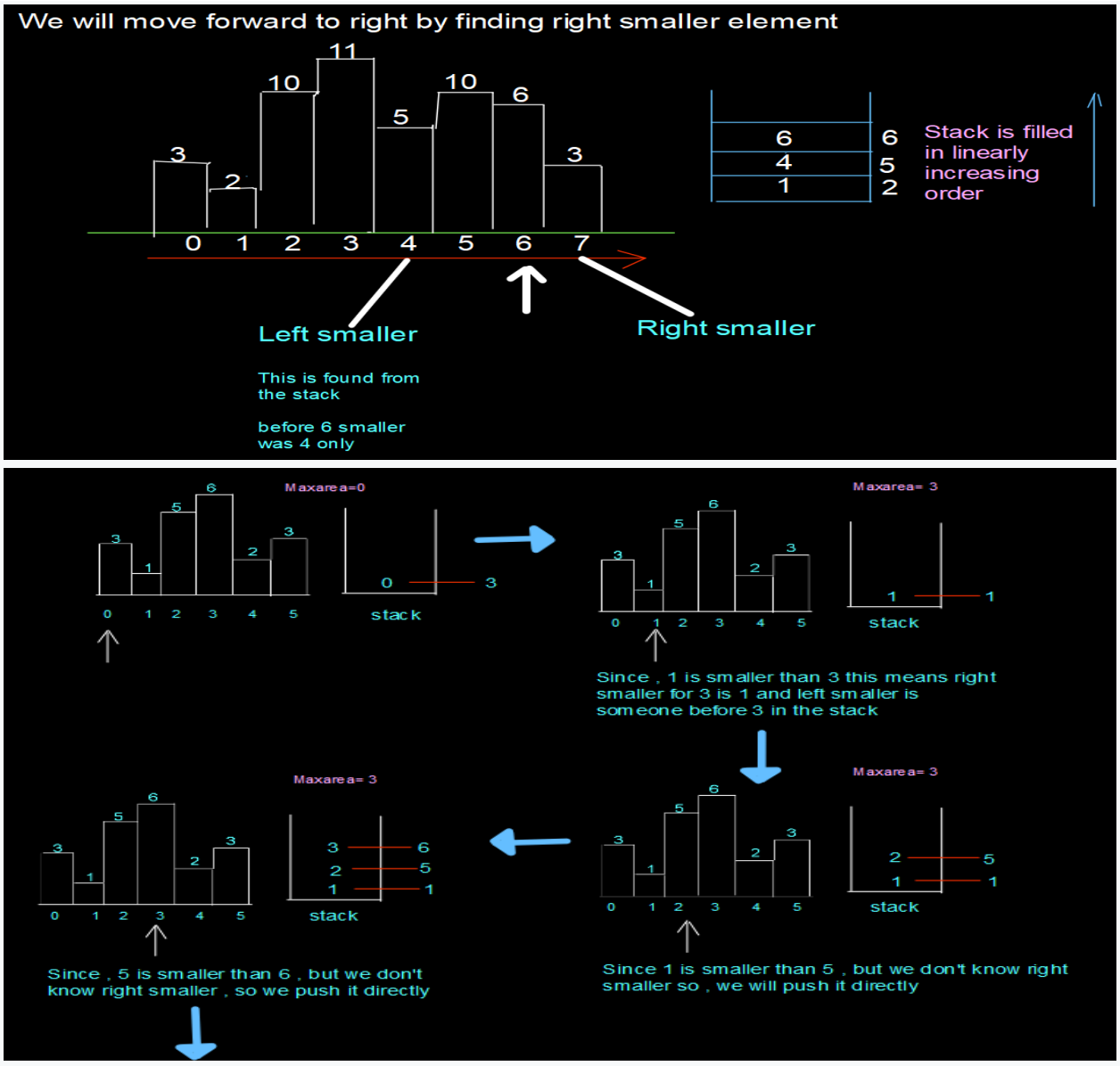
```
        for (int i = 0; i < n; i++) {
            maxA = Math.max(maxA, heights[i] * (rightSmall[i] - leftSmall[i] + 1));
        }
        return maxA; }}
```

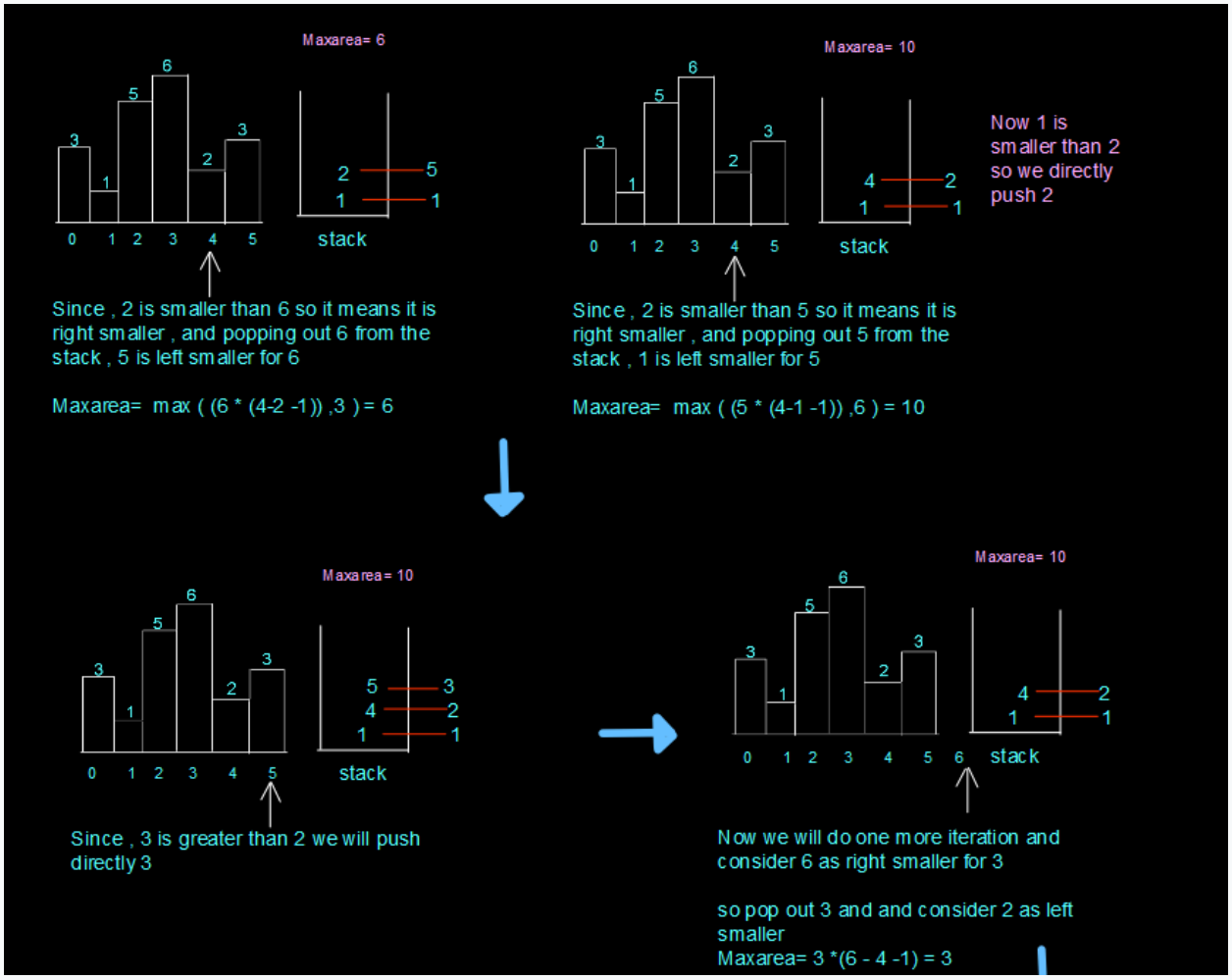
Solution 3: Optimised approach 2

Intuition:

This approach is a single pass approach instead of a two-pass approach. When we traverse the array by finding the next greater element, we found that some elements were inserted into the stack which signifies that after them the smallest element is themselves

So we can find the area of the rectangle by using **arr[i] * (right smaller – left smaller -1)**.




Maxarea= 6
stack
Since , 2 is smaller than 6 so it means it is right smaller , and popping out 6 from the stack , 5 is left smaller for 6
Maxarea= max ((6 * (4-2 -1)) ,3) = 6
Maxarea= 10
stack
Now 1 is smaller than 2 so we directly push 2
Since , 2 is smaller than 5 so it means it is right smaller , and popping out 5 from the stack , 1 is left smaller for 5
Maxarea= max ((5 * (4-1 -1)) ,6) = 10
Maxarea= 10
stack
Since , 3 is greater than 2 we will push directly 3
Maxarea= 10
stack
Now we will do one more iteration and consider 6 as right smaller for 3
so pop out 3 and and consider 2 as left smaller
Maxarea= 3 *(6 - 4 -1) = 3
Maxarea= 10
stack
consider 6 as right smaller for 4
so pop out 4 and and consider 1 as left smaller
Maxarea= 2 *(6 - 1 -1) = 8
Maxarea= 10
stack
Now pop out 1 remaining in the stack and calculate maxarea = 1 * (6 - 0 -1) = 6

Code:

```
 static int largestRectangleArea(int histo[]) {
        Stack < Integer > st = new Stack < > ();
        int maxA = 0;
        int n = histo.length;
        for (int i = 0; i <= n; i++) {
            while (!st.empty() && (i == n || histo[st.peek()] >= histo[i])) {
                int height = histo[st.peek()];
                st.pop();
                int width;
                if (st.empty())
                    width = i;
                else
                    width = i - st.peek() - 1;
                maxA = Math.max(maxA, width * height);
            }
            st.push(i);
        }
        return maxA; }}
```

My soln.

```
public int largestRectangleArea(int[] arr) {
        Stack<Integer> stack = new Stack();
        int n = arr.length, ans = 0;
        for (int i = 0; i <= n; i++){
            int cur = i == n ? 0 : arr[i];
            while (!stack.isEmpty() && cur < arr[stack.peek()]){
                int top = stack.pop();
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                ans = Math.max(ans, width*arr[top]);
            }
            stack.push(i);
        }
        return ans; }
```

11. Implement min stack

Implement Min Stack | O(2N) and O(N) Space Complexity. Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.
Examples:**Input Format**:["MinStack", "push", "push", "push", "getMin", "pop", "top", "getMin"]
[[ ], [-2], [0], [-3], [ ], [ ], [ ], [ ]]
Result: [null, null, null, null, -3, null, 0, -2]
Explanation:
stack < long long > st
st.push(-2); Push element in stack st.push(0); Push element in stack
st.push(-3); Push element in stack st.getMin(); Get minimum element fromstack
st.pop(); Pop the topmost element st.top(); Top element is 0
st.getMin(); Minimum element is -2

Solution 1: Using pairs to store the value and minimum element till now.

Approach: The first element in the pair will store the value and the second element will store the minimum element till now. When the first push operation comes in we will push the value and store it as minimum itself in the pair. In the second push operation, we will check if the top element's minimum is less than the new value. If it is then we will push the value with minimum as the previous top's minimum. To get the getMin element to take the top's second element.

```
class Pair {
    int x, y;
    Pair(int x, int y) {
        this.x = x;
        this.y = y;
    }}
class MinStack {
    Stack < Pair > st;
    public MinStack() {
        st = new Stack < > (); }
    public void push(int x) {
        int min;
        if (st.isEmpty()) {
            min = x;
        } else {
            min = Math.min(st.peek().y, x);
        }
        st.push(new Pair(x, min));
    }
    public void pop() {
        st.pop();
    }
    public int top() {
        return st.peek().x;
    }
    public int getMin() {
        return st.peek().y;
    }
}
```

Solution 2:

Approach:

Let's take a variable that stores the minimum number. So whenever a push operation comes in just take that number put it in the stack and update the variable to the number.

Push operation:

Now if there is a push operation just check whether that number is less than the min number. If it is smaller than min we will push a modified value which is a push(2 * Val – min) into the stack and will update min to the value of the original number. If it's not then we will just push it as it is.

getMin() operation:

We will just return the value of min.

Top operation:

While returning the top value we know that it is a modified value. We will check if the top value is lesser than min, If it is then we will return the min as the top value.

Pop operation:

While making pop we will check if the top value is lesser than min, If it is then we must update our min to its previous value. In order to do that min = (2 * min) – (modified value) and we will pop the element.

```
class MinStack {
    Stack < Long > st = new Stack < Long > ();
    Long mini;
    public MinStack() {
        mini = Long.MAX_VALUE;
    }
    public void push(int value) {
        Long val = Long.valueOf(value);
        if (st.isEmpty()) {
            mini = val;
            st.push(val);
        } else {
            if (val < mini) {
                st.push(2 * val - mini);
                mini = val;
            } else {
                st.push(val);
            }
        }}
    public void pop() {
        if (st.isEmpty()) return;
        Long val = st.pop();
        if (val < mini) {
            mini = 2 * mini - val;
        }
    }
    public int top() {
        Long val = st.peek();
        if (val < mini) {
            return mini.intValue();
        }
        return val.intValue();
    }
    public int getMin() {
        return mini.intValue();
    }}
```

12. Maximum Number of Eaten Apples

There is a special kind of apple tree that grows apples every day for `n` days. On the `ith` day, the tree grows `apples[i]` apples that will rot after `days[i]` days, that is on day `i + days[i]` the apples will be rotten and cannot be eaten. On some days, the apple tree does not grow any apples, which are denoted by `apples[i] == 0` and `days[i] == 0`.You decided to eat at most one apple a day (to keep the doctors away). Note that you can keep eating after the first `n` days.Given two integer arrays `days` and `apples` of length `n`, return *the maximum number of apples you can eat.*

Example 1: **`Input`**: `apples = [1,2,3,5,2], days = [3,2,1,4,2]` **`Output`**: `7`
`Explanation`: `You can eat 7 apples:`
`- On the first day, you eat an apple that grew on the first day.`
`- On the second day, you eat an apple that grew on the second day.`

- On the third day, you eat an apple that grew on the second day. After this day, the apples that grew on the third day rot.
- On the fourth to the seventh days, you eat apples that grew on the fourth day.

Example 2: **Input**: apples = [3,0,0,0,0,2], days = [3,0,0,0,0,2] **Output**: 5
Explanation: You can eat 5 apples:
- On the first to the third day you eat apples that grew on the first day.
- Do nothing on the fouth and fifth days.
- On the sixth and seventh days you eat apples that grew on the sixth day.

```
class Solution {
    class Apple{
        int no, expires;
        Apple(int no, int expires){
            this.no = no;
            this.expires = expires;
        }}
    public int eatenApples(int[] apples, int[] days) {
        int n = apples.length, count = 0;
        PriorityQueue<Apple> pq = new PriorityQueue<>(n,
                                                (a, b) -> a.expires - b.expires);
        for (int i = 0; i < n || !pq.isEmpty(); i++){
            if (i < n && apples[i] != 0){
                pq.add(new Apple(apples[i], days[i] + i));
            }
            while (! pq.isEmpty() && pq.peek().expires <= i) {  pq.poll(); }
            if (! pq.isEmpty()){
                Apple eatUp = pq.poll();
                eatUp.no--;
                count++;
                if (eatUp.no > 0){
                    pq.add(eatUp);
                }
            }}
        return count;
    }}
```

13. Maximum Product After K Increments

You are given an array of non-negative integers `nums` and an integer `k`. In one operation, you may choose any element from `nums` and increment it by `1`. Return *the maximum product of* `nums` *after at most* `k` *operations.* Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1: **Input**: nums = [0,4], k = 5 **Output**: 20

Explanation: Increment the first number 5 times.
Now nums = [5, 4], with a product of 5 * 4 = 20.
It can be shown that 20 is maximum product possible, so we return 20.

Example 2: `Input: nums = [6,3,3,2], k = 2 Output: 216`

`Explanation: Increment the second number 1 time and increment the fourth number 1 time. Now nums = [6, 4, 3, 3], with a product of 6 * 4 * 3 * 3 = 216. It can be shown that 216 is maximum product possible, so we return 216.`

`Note that there may be other ways to increment nums to have the maximum product.`

Think about a simple case, assume you have [4, 1].The current product is 4 * 1 = 4.

1. If we increment the 4, the array will be [5, 1] and the result will be 5. ❌
2. If we increment 1, the array will be [4, 2] and the result will be 8. ✅
   Option 2 wins.

The current product after update is 4 * 2 = 8.

1. If we increment the 4, the array will be [5, 2] and the result will be 10. ❌
2. If we increment the 2, the array will be [4, 3] and the result will be 12. ✅
   Option 2 wins. And so on ...

If you notice it, what's actually happening with each increment is that we are adding the other number in the array to the current product. You're basically adding `1`, and this `1` is multiplied by all numbers other than the number you've added this `1` to. The result of the multiplication is added to the current product.

In the above example, the array had only two elements, that's why we are always adding the other element to the current product. If the array is for example [6, 5, 3], the current product is 6 * 5 * 3 = 90, updating the array by adding 1 to 3 is just as simple as adding all other numbers' product (6 * 5) to current product (90), i.e., the result becomes 120.

That's why the solution is to always increment the minimum of the array. This ensures that the product of all other numbers except the minimum is max.A min heap is the easiest way to track the minimum.

```
class Solution {
    public int maximumProduct(int[] nums, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue();

        for (int N: nums) pq.offer(N);

        while (k-- > 0){
            pq.offer(pq.poll() + 1);
        }

        long ans = 1, mod = 1_000_000_007;
        while (!pq.isEmpty()){
            ans = (ans*pq.poll())%mod;
```

```
        }
        return (int) ans; }}
```

14. Find K Pairs with Smallest Sums

You are given two integer arrays `nums1` and `nums2` sorted in ascending order and an integer `k`.

Define a pair `(u, v)` which consists of one element from the first array and one element from the second array.

Return *the* `k` *pairs* $(u_1, v_1), (u_2, v_2), ..., (u_k, v_k)$ *with the smallest sums*.

Example 1: `Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3 Output: [[1,2],[1,4],[1,6]]`
`Explanation: The first 3 pairs are returned from the sequence:`
`[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]`

Example 2: `Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2 Output: [[1,1],[1,1]]`
`Explanation: The first 2 pairs are returned from the sequence:`
`[1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]`

Example 3: `Input: nums1 = [1,2], nums2 = [3], k = 3 Output: [[1,3],[2,3]]`
`Explanation: All possible pairs are returned from the sequence: [1,3],[2,3]`

```
class Solution {
    class pair{
        int f, s, j;
        pair(int f, int s, int j){
            this.f = f;  this.s = s; this.j = j;
        }
    }
    public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        int n = nums1.length, m = nums2.length;
        List<List<Integer>> ans = new ArrayList();
        PriorityQueue<pair> pQ = new PriorityQueue<pair>((a, b) ->
                                            ((a.f + a.s) - (b.f + b.s)));

        for (int i = 0; i < n && i < k; i++){
            pQ.add(new pair(nums1[i], nums2[0], 0));
        }

        for (int i = 0; i < k && !pQ.isEmpty(); i++){
            pair p = pQ.poll();
            ans.add(List.of(p.f, p.s));
            if (p.j + 1 < m){
                pQ.add(new pair(p.f, nums2[p.j + 1], p.j + 1));
            }
```

```
        }
        return ans;
    }}
```

15. **Rotten oranges**
16. **Stock Span problem**
17. **Find maximum of minimum of every window size**
18. **The celebrity problem**

VII. SORTING ALGOS:

A. Binary Search

1. Ways to Split Array Into Three Subarrays

A split of an integer array is good if:

- The array is split into three non-empty contiguous subarrays - named `left`, `mid`, `right` respectively from left to right.
- The sum of the elements in `left` is less than or equal to the sum of the elements in `mid`, and the sum of the elements in `mid` is less than or equal to the sum of the elements in `right`.

Given `nums`, an array of non-negative integers, return *the number of good ways to split* `nums`. As the number may be too large, return it modulo $10^9 + 7$.

Example 1:

```
Input: nums = [1,1,1]
Output: 1
Explanation: The only good way to split nums is [1] [1] [1].
```

Example 2:

```
Input: nums = [1,2,2,2,5,0]
Output: 3
Explanation: There are three good ways of splitting nums:
```

```
[1] [2] [2,2,5,0]
[1] [2,2] [2,5,0]
[1,2] [2,2] [5,0]
```

```java
class Solution {
    public int waysToSplit(int[] nums) {
        int MOD = (int) (1e9 + 7), N = nums.length;

        int[] A = new int[N];
        A[0] = nums[0];
        for (int i = 1; i < N; ++i) A[i] = A[i - 1] + nums[i];

        int res = 0;
        for (int i = 1; i < N - 1; ++i) {

            if (A[i - 1] > (A[N - 1] - A[i - 1]) / 2) break;  // early termination

            int left = helper(A, A[i - 1], i, true);
            int right = helper(A, A[i - 1], i, false);

            if (left == -1 || right == -1) continue;  // none is satisfied

            res = (res + (right - left + 1) % MOD) % MOD;
        }
        return res;
    }

    private int helper(int[] A, int leftSum, int index, boolean searchLeft) {
        int N = A.length, l = index, r = N - 2, res = -1;
        while (l <= r) {

            int m = l + (r - l)/2;
            int midSum = A[m] - A[index - 1];
            int rightSum = A[N - 1] - A[m];

            if (leftSum <= midSum && midSum <= rightSum) {
                res = m;
                if (searchLeft) r = m - 1;
                else l = m + 1;
            } else if (leftSum > midSum) {  // shrink left
                l = m + 1;
            } else {  // shrink right
                r = m - 1;
            }
        }
        return res;
    }
}
```

IMPORTANT PROBLEMS TO BE ADDED

`Array problems`
https://takeuforward.org/data-structure/count-inversions-in-an-array/
https://www.geeksforgeeks.org/counting-inversions/
REverse pairs
https://leetcode.com/problems/reverse-pairs/

`Sliding window`

- 220. Contains Duplicate III
- 239. Sliding Window Maximum
- 480. Sliding Window Median

`DYNAMIC PROG.`
https://leetcode.com/problems/combination-sum-iv/
https://leetcode.com/problems/longest-arithmetic-subsequence/
Maximum profit in Job scheduling

Rod Cutting

https://leetcode.com/problems/ones-and-zeroes/
https://leetcode.com/problems/palindrome-partitioning-ii/

`RECURSION`

N queens Problem

Sudoku Solver

Rat in a Maze

Word Break (print all ways)

`DFS`
https://leetcode.com/problems/path-with-maximum-gold/

`Trees`

Morris Inorder Traversal

Morris Preorder Traversal

`Stack problems`

Stock Span problem

Find maximum of minimum of every window size

The celebrity problem

`Grid problems`

Rotten Orange (Using BFS)

Similar questions based on "Binary search on required answer range" on Leetcode ->

1. **Magnetic Force Between Two Balls**
2. **Capacity To Ship Packages Within D Days**
3. **Split Array Largest Sum**
4. **Find the Smallest Divisor Given a Threshold**
5. **Minimum Number of Days to Make m Bouquets**
6. **Minimized Maximum of Products Distributed to Any Store**

1870. Minimum Speed to Arrive on Time

https://leetcode.com/problems/koko-eating-bananas/

https://leetcode.com/problems/minimum-limit-of-balls-in-a-bag/

https://leetcode.com/problems/maximum-candies-allocated-to-k-children/